Fusion HCI Multi-Architecture Integration

None

OpenShift Integration Team

None

Table of contents

1. Fusion HCI Multi-Architecture Integration 4
   - 1.1 What is This? 4
   - 1.2 Architecture Overview 4
   - 1.3 Quick Links 4
   - 1.4 Use Cases 5
   - 1.5 Features 6
   - 1.6 Deployment Variants 6
   - 1.7 Quick Start 6
   - 1.8 Support & Community 7
   - 1.9 License 7
   - 1.10 Acknowledgments 7
2. Getting Started 8
   - 2.1 Prerequisites 8
   - 2.2 Getting Started 18
   - 2.3 Deployment Variants 28
3. Architecture 36
   - 3.1 Solution Overview 36
   - 3.2 Architecture Overview 41
   - 3.3 Network Design 51
   - 3.4 VM Provisioning Architecture 59
   - 3.5 Provider/Consumer Model for HCI Integration 68
   - 3.6 HCI Provisioning Model — Lifecycle Operator Design 72
   - 3.7 Setup Concept — Direction C: `integration-setup.yaml`-Driven Day-0 Setup 78
4. Configuration 82
   - 4.1 HCI Cluster Configuration 82
   - 4.2 Main Cluster Configuration 90
   - 4.3 Kustomize Overlays Guide 99
5. Operations 106
   - 5.1 Troubleshooting Guide 106
   - 5.2 Monitoring Guide 120
6. Development 130
   - 6.1 Developer Notes 130
   - 6.2 Building and Deploying the Operators 135
   - 6.3 Operator Optimization and Go Migration Guide 140
   - 6.4 Contributing to Fusion HCI Multi-Architecture Integration 143

6.5 Operator Roadmap 148

1. Fusion HCI Multi-Architecture Integration

Welcome to the Fusion HCI Multi-Architecture Integration documentation. This project enables seamless integration of Fusion HCI x86 worker nodes into s390x/LinuxONE OpenShift clusters, creating true multi-architecture deployments.

Architecture Flexibility

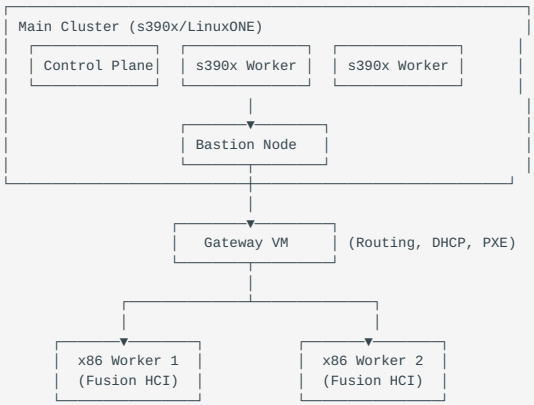
While this project focuses on s390x/LinuxONE as the main cluster architecture, the solution is designed to be architecture-agnostic. The same principles and components can be adapted for ARM or Power-based main clusters. The network integration, VM provisioning, and configuration management patterns are applicable across different CPU architectures.

1.1 What is This?

This integration solution allows you to:

- **Extend s390x clusters** with x86 compute capacity from Fusion HCI
- **Run multi-architecture workloads** in a single OpenShift cluster
- **Leverage both architectures** for optimal workload placement
- **Maintain unified management** across different CPU architectures

1.2 Architecture Overview



1.2.1 Key Components

- **Main Cluster**: Your primary OpenShift cluster (s390x/LinuxONE, ARM, or Power)
- **HCI Cluster**: Fusion HCI cluster providing x86 compute resources
- **Gateway VM**: Network bridge providing DHCP, PXE, and routing
- **Worker VMs**: x86 RHCOS nodes that join the main cluster

1.3 Quick Links

1.3.1 Getting Started

- Prerequisites - What you need before starting
- Getting Started Guide - Step-by-step deployment

- Deployment Variants - Choose Mediated or Direct topology

1.3.2 Architecture

- Architecture Overview - System design
- Network Design - Network topology details
- VM Provisioning - How VMs are deployed

1.3.3 Configuration

- HCI Cluster Setup - Configure Fusion HCI
- Main Cluster Setup - Configure main cluster
- Kustomize Overlays - Variant configuration

1.3.4 Operations

- Troubleshooting - Common issues and solutions
- Monitoring - Monitor your deployment

1.3.5 Development

- Developer Notes - Technical decisions and learnings
- Contributing - How to contribute
- Operator Roadmap - Automation roadmap and implementation status

1.3.6 Operators

- hci-consumer Operator - Steady-state worker VM lifecycle operator
- hci-provisioner Operator - Bootstrap and teardown of gateway VM, UDN, and ignition
- Provider/Consumer Model - Three-actor architecture design

1.4 Use Cases

1.4.1 Multi-Architecture Workloads

Run applications that require both architectures:

- Legacy mainframe applications alongside modern x86 microservices
- CI/CD pipelines building for multiple architectures
- Hybrid workloads leveraging strengths of each architecture

1.4.2 Capacity Expansion

Add x86 compute capacity to your main cluster:

- Scale out with cost-effective x86 hardware
- Maintain primary architecture for critical workloads
- Flexible resource allocation

1.4.3 Development & Testing

Test multi-architecture deployments:

- Validate application portability
- Test architecture-specific optimizations
- Develop cross-platform solutions

1.5 Features

✅ **Production Ready**: Comprehensive testing and documentation ✅ **Flexible Deployment**: Supports Mediated and Direct network topologies ✅ **Operator-Driven Provisioning**: Gateway VM, UDN, ignition, and worker VM lifecycle fully managed by `hci-provisioner` and `hci-consumer` operators ✅ **Network Integration**: Transparent networking via OVN-Kubernetes ✅ **Security**: Network isolation and access control ✅ **Architecture Agnostic**: Adaptable to ARM, Power, or other architectures 🚧 **HCI Provider Operator**: Epics 3–4 (HCI-side automation, negotiation flow) in backlog

1.6 Deployment Variants

This solution supports two network topology variants:

1.6.1 Mediated Topology (Private Network)

- Main cluster nodes in private network behind bastion
- Bastion provides routing and DNS
- Common in z/VM, LPAR, or highly secured environments

1.6.2 Direct Topology (Public Network)

- Main cluster nodes on shared public network
- Direct connectivity to HCI gateway
- Requires static route on master nodes
- Common in KVM, bare metal, or less restricted environments

See Deployment Variants for detailed comparison.

1.7 Quick Start

1. **Prepare Main Cluster**

```
oc adm upgrade --to-multiarch
oc apply -k main-cluster/configuration/overlays/mediated/
```

2. **Prepare HCI Cluster (cluster-admin, one-time)**

```
oc apply -f hci-cluster/cluster-network-setup.yaml
oc label namespace <namespace> hci-gateway-enabled=true
```

3. **Set up credentials**

```
./operator/setup-provider-credentials.sh
```

4. **Apply a `WorkerPool` CR** — the operators provision everything automatically:

```
oc apply -f operator/hci-consumer/config/samples/workerpool-sample.yaml
```

The `hci-consumer` operator creates the `WorkerPoolProvisioning` CR for you. The `hci-provisioner` runs all bootstrap steps, then `hci-consumer` scales up.

See Getting Started for detailed instructions.

1.8 Support & Community

- **Documentation**: You're reading it!
- **Issues**: GitHub Issues
- **Discussions**: GitHub Discussions
- **Source Code**: GitHub Repository

1.9 License

This project is licensed under the Apache License 2.0.

1.10 Acknowledgments

This project builds on:

- OpenShift's multi-architecture capabilities
- OpenShift Virtualization
- OVN-Kubernetes networking
- Kustomize for configuration management
- Ansible Operator (operator-sdk Ansible plugin)

Ready to get started? Head to the Prerequisites page to begin your journey!

2. Getting Started

2.1 Prerequisites

This page outlines the requirements and preparation steps needed before deploying the Fusion HCI multi-architecture integration.

2.1.1 System Requirements

Main Cluster (s390x/LinuxONE)

OpenShift Version: - OpenShift 4.20 or later (tested on 4.20+) - Multi-architecture support enabled

Cluster Configuration: - At least 3 control plane nodes - Existing s390x worker nodes (optional but recommended) - Cluster-admin access required

Network Requirements: - Routable network for cluster nodes - Bastion host with external connectivity - DNS server (typically on bastion) - Available IP addresses for HCI workers

Storage: - No persistent storage required on main cluster - Worker node data stored on HCI cluster VMs

HCI Cluster (Fusion HCI)

OpenShift Version: - OpenShift 4.20 or later (tested on 4.20+) - OpenShift Virtualization operator installed

Cluster Configuration: - Worker nodes with virtualization capabilities - Sufficient compute resources for VMs - Cluster-admin access required

Network Requirements: - OVN-Kubernetes CNI (default) - Physical network access (localnet support) - Available IP addresses in physical network range

Storage: - Fusion Data Foundation (FDF) built-in with S3 storage (NooBaa) - Storage class for VM disks - Sufficient capacity for worker VMs (50GB+ per VM) - FDF provides persistent S3 storage for HCI cluster's image registry

Network Connectivity

Between Clusters: - Network connectivity between HCI and main cluster - Routing configured (via bastion or direct) - Firewall rules allowing required traffic

Required Ports: - **6443**: Kubernetes API server - **22623**: Machine config server - **80/443**: Application routes - **9000-9999**: Node ports (if used) - **DHCP (67/68)**: For PXE boot - **TFTP (69)**: For PXE boot - **HTTP (8080)**: For ignition files

2.1.2 Software Requirements

On Your Workstation

Required Tools:

```
# OpenShift CLI
oc version
# Client Version: 4.20.0 or later

# Kubernetes CLI (optional, oc includes kubectl)
kubectl version --client

# Kustomize (or use kubectl kustomize)
kustomize version
# v5.0.0 or later
```

Optional Tools:

```
# For documentation
mkdocs --version
# mkdocs, version 1.5.0 or later

# For YAML validation
yamllint --version

# For shell script linting
shellcheck --version
```

Installation

Linux macOS Windows

```
# OpenShift CLI
curl -LO https://mirror.openshift.com/pub/openshift-v4/clients/ocp/latest/openshift-client-linux.tar.gz
tar xzf openshift-client-linux.tar.gz
sudo mv oc kubectl /usr/local/bin/

# Kustomize
curl -s "https://raw.githubusercontent.com/kubernetes-sigs/kustomize/master/hack/install_kustomize.sh" | bash
sudo mv kustomize /usr/local/bin/

# Using Homebrew
brew install openshift-cli kustomize

# Using Chocolatey
choco install kubernetes-cli kustomize

# Download oc separately from:
# https://mirror.openshift.com/pub/openshift-v4/clients/ocp/latest/
```

2.1.3 Access Requirements

Kubernetes Context Setup

Context Management

Using named contexts makes it easy to switch between clusters. We recommend naming your contexts `main-cluster` and `hci-cluster` for consistency with documentation examples and helper scripts.

Main Cluster Access

1. **Login and Create Context**

```
# Login to main cluster
oc login https://api.cluster.example.com:6443

# Get the current context name
oc config current-context
# Output: default/api-cluster-example-com:6443/admin

# Rename context for easy reference
oc config rename-context $(oc config current-context) main-cluster

# Verify
oc config get-contexts
# Should show 'main-cluster' with a '*' indicating it's active

# Test access
oc whoami
# Should show your username

oc auth can-i '*' '*' --all-namespaces
# Should return "yes"
```

2. **Bastion Host Access**

```
# SSH access to bastion
ssh user@bastion.cluster.example.com

# On bastion, also setup context (if using oc there)
```

```
oc login https://api.cluster.example.com:6443
oc config rename-context $(oc config current-context) main-cluster

# Verify DNS service
systemctl status named

# Verify routing capability
ip route show
```

HCI Cluster Access

1. **Login and Create Context**

```
# Login to HCI cluster
oc login https://api.hci-cluster.example.com:6443

# Rename context for easy reference
oc config rename-context $(oc config current-context) hci-cluster

# Verify
oc config get-contexts
# Should show both 'main-cluster' and 'hci-cluster'

# Test access
oc whoami

oc auth can-i '*' '*' --all-namespaces
# Should return "yes"
```

2. **OpenShift Virtualization**

```
# Ensure you're using HCI cluster context
oc config use-context hci-cluster

# Verify virtualization operator
oc get csv -n openshift-cnv

# Check for HyperConverged CR
oc get hyperconverged -n openshift-cnv
```

Switching Between Clusters

Once contexts are configured, switching is simple:

```
# Switch to main cluster
oc config use-context main-cluster
oc get nodes

# Switch to HCI cluster
oc config use-context hci-cluster
oc get nodes

# View all contexts
oc config get-contexts

# View current context
oc config current-context
```

> **Context in Scripts**
>
> The helper scripts in `main-cluster/scripts/` use the centralized `setup_env.sh` configuration which allows you to specify context names. Update `setup_env.sh` with your context names:
>
> ```
> MAIN_CLUSTER_CONTEXT="main-cluster"
> HCI_CLUSTER_CONTEXT="hci-cluster"
> ```

2.1.4 Environment Configuration

Before starting deployment, capture all cluster-pair decisions in a single configuration file — `workspace/integration-setup.yaml`. This is the **only** file you edit by hand. The `generate-setup` script derives every other environment artifact from it: shell environment variables, operator config, kustomize overlays, DNS zone files, and the initial `WorkerPool` CR. All integration-specific

assets land in `workspace/`; `setup_env.sh` is written to the repo root so every script in the repository can source it with a relative path.

Setting Up `workspace/integration-setup.yaml`

```
# 1. Create the workspace directory (skip if it already exists)
mkdir -p workspace

# 2. Copy the annotated example into the workspace
cp templates/integration-setup.example.yaml workspace/integration-setup.yaml

# 3. Open it in your editor — every field has an inline comment explaining
#    what it is and how to find the right value
vi workspace/integration-setup.yaml
```

The file has three top-level sections with clear ownership:

Section	Owner	What goes here
`consumer.*`	Main cluster (s390x/LinuxONE) admin	Cluster context, domain, operator namespace, bastion, topology, DNS server
`provider.*`	HCI cluster admin	Cluster context, HCI namespace, gateway IP, MAC prefix, internal subnet
`registry.*`	Both admins (agreed once)	Registry model (`local-s2i`, `cross-cluster`, `external`) and model-specific fields

> **Two-admin handover**
>
> In a two-team setup the main cluster admin fills in `consumer.*` and the HCI admin fills in `provider.*`. The finished file is the negotiated record of the cluster-pair relationship — keep it alongside your Ansible inventory.

All fields have inline comments in the example file. The required fields are:

```
# ── Consumer cluster (s390x / LinuxONE) ──────────────────────────────────
consumer:
  context: main-cluster              # oc config get-contexts
  domain: cluster.example.com        # oc get dns cluster -o jsonpath='{.spec.baseDomain}'
  operatorNamespace: hci-node-manager
  bastion:
    host: bastion.cluster.example.com
    user: root
  networking:
    topology: mediated               # direct | mediated
    dnsServer: 172.16.0.1            # main cluster bastion IP
    # mediated topology only:
    internalNetwork: 10.0.0.0/24    # main cluster's internal network
    internalGateway: 172.16.0.1     # HCI-side gateway to that network

# ── Provider cluster (x86 / Fusion HCI) ──────────────────────────────────
provider:
  context: hci-cluster               # oc config get-contexts
  apiEndpoint: https://api.hci.example.com:6443
  registryHostname: default-route-openshift-image-registry.apps.hci.example.com
  namespace: main-cluster-workers    # one namespace per main cluster on this HCI
  maxPoolSize: 10                    # optional, default: 10; also sizes DNS zones
  gateway:
    ip: 172.16.1.100                 # assigned from HCI physical network
    prefixLength: 16
    defaultRouter: 172.16.0.1
  networking:
    macPrefix: "52:54:01"            # unique per pool on this HCI; optional, default: 52:54:00
    internalSubnet: 192.168.100.0/24

# ── Image registry ────────────────────────────────────────────────────────
registry:
  model: cross-cluster               # local-s2i | cross-cluster | external
  # cross-cluster model only:
  proxyPort: 5443
  saName: registry-pusher
  secretName: hci-registry-push
```

Generating All Configuration Artifacts

Once `workspace/integration-setup.yaml` is complete, run the generator from the repo root:

```
./generate-setup workspace/integration-setup.yaml
```

This writes every derived artifact in one shot:

Output	Purpose
`setup_env.sh`	Shell environment — source this in every terminal session
`workspace/workerpool-hci-workers.yaml`	Initial `WorkerPool` CR (`workers.count: 0`)
`operator/build/config.env`	Makefile variables for `make build` / `make deploy`
`main-cluster/configuration/overlays/<topology>/`	Kustomize overlays (direct topology only)
`workspace/<domain>.zone`	Forward DNS zone section (A records for gateway + workers)
`workspace/<subnet>.in-addr.arpa.zone`	Reverse DNS zone (PTR records)

```
# After generating, source the environment in your current shell
source setup_env.sh
```

> **Re-run after any change**
>
> `generate-setup` is idempotent — re-running it after editing `workspace/integration-setup.yaml` regenerates all artifacts consistently. The `WorkerPool` CR in `workspace/` is overwritten on every run; to preserve the live state of a running pool use `oc get workerpool hci-workers -o yaml` instead.

Configuration Validation

Before proceeding to deployment, validate your configuration:

- [ ] All IP addresses are available and not in use
- [ ] DNS entries planned for worker nodes
- [ ] Cluster contexts are configured and accessible
- [ ] Network connectivity between clusters verified
- [ ] Firewall rules planned
- [ ] Storage capacity sufficient for worker VMs
- [ ] `workspace/integration-setup.yaml` created and `./generate-setup` run
- [ ] `setup_env.sh` sourced in the current shell

DNS Configuration

> **DNS zone files are generated automatically**
>
> Once you have run `./generate-setup workspace/integration-setup.yaml`, ready-to-use zone file sections are written into `workspace/`:
>
> - `workspace/<domain>.zone` — forward A records for the gateway VM and all worker nodes up to `provider.maxPoolSize`
> - `workspace/<subnet>.in-addr.arpa.zone` — reverse PTR records for worker nodes
>
> Copy or append those files to your bastion DNS server instead of writing entries by hand. Re-run `generate-setup` after any change to `workspace/integration-setup.yaml` to keep the zone files in sync with the WorkerPool configuration. Step-by-step instructions for applying them are in the Getting Started guide — step C.1.

The generated forward zone section looks like this:

```
; Forward zone entries — produced by generate-setup
hci-gateway.cluster.example.com.     IN  A  172.16.1.100
worker-x86-1.cluster.example.com.    IN  A  192.168.100.10
worker-x86-2.cluster.example.com.    IN  A  192.168.100.11
worker-x86-3.cluster.example.com.    IN  A  192.168.100.12
```

Allow recursive queries from the gateway and worker subnet

The bastion DNS server is also the upstream resolver for the gateway VM and all worker VMs. By default a BIND server only allows recursion from `localhost` and its own networks, so queries from those VMs are refused. Add the gateway's external IP (`provider.gateway.ip`) and the worker subnet (`provider.networking.internalSubnet`) to the recursion ACL — for example in a typical BIND configuration:

```
# /etc/named.conf
acl "internal_nets" {
    ...existing entries...;
    172.16.1.100/32;        # provider.gateway.ip
    192.168.100.0/24;       # provider.networking.internalSubnet
};
options {
    allow-query       { localhost; internal_nets; };
    allow-recursion   { localhost; internal_nets; };
    allow-query-cache { localhost; internal_nets; };
};
```

The Getting Started guide — step C.1 covers this in detail, including the stop/start requirement for `named-chroot`.

Routing Configuration

Routing is handled by scripts driven by `setup_env.sh`

You do not set IP addresses or routes by hand. After running `generate-setup` (which writes `setup_env.sh`), the scripts and kustomize overlays in the Getting Started guide pick up all values automatically:

- **Mediated topology** — `main-cluster/scripts/01-add-route.sh` and `02-firewall-setup.sh` configure the bastion route and firewall rules (step C.2). Both scripts source `setup_env.sh` — no manual editing needed.
- **Direct topology** — `generate-setup` writes a ready-to-apply route MachineConfig into `main-cluster/configuration/overlays/direct/` (applied with `oc apply -k` in step C.4).

2.1.5 Registry Strategy Selection

Record your choice in `workspace/integration-setup.yaml`, then follow the setup guide

Set `registry.model` to your chosen strategy before running `generate-setup`. The script wires the correct registry URLs and credentials into `setup_env.sh` and `operator/build/config.env` automatically. The actual HAProxy proxy and credential setup is then handled by scripts in the Getting Started guide (steps C.5 and C.6) that source `setup_env.sh` — no manual variable editing needed.

Choose your container image registry strategy:

Option 1: Bastion Registry Proxy (Recommended) — `registry.model: cross-cluster`

Use when: - You want to build multi-architecture images - HCI workers on main cluster cannot directly access HCI cluster registry - You need a unified registry accessible from both clusters

Architecture: - HCI cluster has built-in Fusion Data Foundation (FDF) with S3 storage (NooBaa) - HCI cluster's image registry uses FDF S3 for persistent storage - HAProxy on main cluster's bastion proxies registry traffic to HCI cluster - Enables all workers (s390x and x86) to access HCI registry

Setup: Set `registry.model: cross-cluster` in `workspace/integration-setup.yaml`. The actual HAProxy and credential setup is handled by `setup-registry-proxy.sh` (step C.5) and `setup-registry-access.sh` (step C.6) — both source `setup_env.sh`.

Documentation: Bastion Registry README

Pros: - ✅ Persistent storage via HCI cluster's built-in FDF (no external S3 needed) - ✅ Works for all network topologies - ✅ Enables cross-cluster builds - ✅ Single registry for all architectures - ✅ No persistent storage required on main cluster

Cons: - ❌ Requires bastion HAProxy configuration - ❌ Additional infrastructure component (proxy) - ❌ Self-signed certificate (insecure registry config needed)

Option 2: Public Registry (quay.io, ICR, etc.) — `registry.model: external`

Use when: - You have access to external registry - You don't want to manage internal registry infrastructure - Internet connectivity is reliable

Setup: Set `registry.model: external` and `registry.pullSecretName` in `workspace/integration-setup.yaml`. Pre-create the pull secret on the main cluster before following the setup guide.

Pros: - ✅ No infrastructure to manage - ✅ High availability - ✅ Valid SSL certificates - ✅ Often includes image scanning

Cons: - ❌ Requires internet connectivity - ❌ May have cost implications - ❌ Data egress from clusters - ❌ Dependency on external service

Option 3: Main Cluster Internal Registry (Not Recommended) — `registry.model: local-s2i`

Use when: - Testing only - Temporary deployments - No persistent storage requirements

Setup: Set `registry.model: local-s2i` in `workspace/integration-setup.yaml`. No additional infrastructure needed — images are built and stored in the main cluster's internal ImageStream registry.

Pros: - ✅ No additional setup - ✅ Already available

Cons: - ❌ **Ephemeral storage** - images lost on pod restart - ❌ Not suitable for production - ❌ May not be accessible from HCI workers

Decision Matrix

Requirement	Bastion Proxy (`cross-cluster`)	Public Registry (`external`)	Internal Registry (`local-s2i`)
Persistent Storage	✅ Yes	✅ Yes	❌ No (ephemeral)
Multi-arch Builds	✅ Yes	✅ Yes	⚠️ Limited
Production Ready	✅ Yes	✅ Yes	❌ No
Setup Complexity	Medium	Low	None
External Dependency	No	Yes	No
Cost	Infrastructure	Service fees	None

Recommendation: Use `cross-cluster` (bastion registry proxy) for production deployments, or `external` if you prefer a managed registry service.

2.1.6 Deployment Variant Selection

Record your choice in `workspace/integration-setup.yaml`

Set `consumer.networking.topology` to `mediated` or `direct`. `generate-setup` uses this to generate the correct kustomize overlays and to include the mediated-topology fields (`internalNetwork`, `internalGateway`) in `setup_env.sh`. See Deployment Variants for a detailed comparison.

Choose your deployment variant based on your main cluster network topology:

Mediated Topology

Use when: - Main cluster nodes are in a private network not directly reachable from the HCI cluster - Bastion host provides external connectivity

Requirements: - Bastion must route to HCI internal network - DNS provided by bastion - Gateway VM connects via bastion's public IP

Direct Topology

Use when: - Main cluster nodes are on the same routable network as the HCI cluster - Direct connectivity between main cluster nodes and HCI

Requirements: - Master nodes need a static route to the HCI internal subnet — generated by `generate-setup` into `main-cluster/configuration/overlays/direct/` - Gateway VM on same network as main cluster - Direct DNS resolution

See Deployment Variants for detailed comparison.

2.1.7 Pre-Deployment Checklist

Before proceeding with deployment, verify:

Main Cluster

- [ ] OpenShift 4.20+ installed
- [ ] Multi-architecture support enabled (`oc adm upgrade --to-multiarch`)
- [ ] Cluster-admin access confirmed
- [ ] Context renamed to `main-cluster`
- [ ] Bastion host accessible via SSH
- [ ] DNS server running and accessible
- [ ] IP addresses allocated for HCI workers and gateway VM
- [ ] `workspace/integration-setup.yaml` filled in and `./generate-setup` run
- [ ] **Registry model chosen** and set in `workspace/integration-setup.yaml`

HCI Cluster

- [ ] OpenShift 4.20+ installed
- [ ] OpenShift Virtualization operator installed
- [ ] Cluster-admin access confirmed
- [ ] Context renamed to `hci-cluster`
- [ ] Storage class available for VMs
- [ ] Sufficient compute resources (CPU, memory)

- [ ] Network connectivity to main cluster verified
- [ ] IP addresses allocated in physical network

Workstation

- [ ] `oc` CLI installed and configured
- [ ] `kustomize` installed (or `kubectl kustomize` available)
- [ ] Access to both clusters configured
- [ ] Contexts properly named (`main-cluster`, `hci-cluster`)
- [ ] Repository cloned locally

Network

- [ ] Connectivity between clusters verified
- [ ] Firewall rules configured
- [ ] IP address ranges allocated
- [ ] DNS configuration planned
- [ ] Routing strategy determined (z/VM or KVM)

2.1.8 Next Steps

Once all prerequisites are met:

1. **Review Architecture**: Read Architecture Overview
2. **Choose Variant**: Review Deployment Variants
3. **Start Deployment**: Follow Getting Started Guide

2.1.9 Troubleshooting Prerequisites

Cannot Access Cluster

```
# Check kubeconfig
echo $KUBECONFIG
cat ~/.kube/config

# Test connectivity
curl -k https://api.cluster.example.com:6443/version

# Check credentials
oc login --token=<token> --server=https://api.cluster.example.com:6443
```

Context Issues

```
# List all contexts
oc config get-contexts

# View current context
oc config current-context

# Rename a context
oc config rename-context <old-name> <new-name>

# Delete a context
oc config delete-context <context-name>
```

OpenShift Virtualization Not Available

```
# Check operator installation
oc get csv -n openshift-cnv
```

```
# Install if missing (via OperatorHub in web console)
# Or check OpenShift documentation for your version
```

Network Connectivity Issues

```
# Test from HCI cluster to main cluster
oc debug node/<hci-node> -- chroot /host ping <main-cluster-bastion-ip>

# Test DNS resolution
oc debug node/<hci-node> -- chroot /host nslookup api.cluster.example.com
```

2.1.10 Support

If you encounter issues with prerequisites:

- Check Troubleshooting Guide
- Review Developer Notes
- Open an issue on GitHub

2.2 Getting Started

This guide walks you through the full lifecycle from a bare HCI cluster to a running multi-architecture integration. The process is organised in three independent tiers that have different cadences:

Tier	Who	How often
A — HCI cluster-wide setup	HCI admin	Once per HCI cluster
B — HCI per-integration setup	HCI admin	Once per main-cluster you want to connect
C — Connect a main cluster	Main-cluster admin	Once per main cluster (re-run for re-connections)

Reconnecting an existing cluster?

If the bastion host is already configured and the main cluster has previously been integrated, **tiers A and B are already done**. Jump straight to Tier C and check each step against your existing state — several steps will be no-ops.

Prerequisites Required

Before starting, ensure you've completed all steps in Prerequisites, including setting up contexts named `main-cluster` and `hci-cluster` and choosing your registry strategy.

2.2.1 Workspace Setup

If you have not done so already, complete the Environment Configuration section of the prerequisites: copy the example, fill in `workspace/integration-setup.yaml`, and run `generate-setup` once to produce all artifacts including `setup_env.sh`.

```
# If generate-setup has not been run yet:
mkdir -p workspace
cp templates/integration-setup.example.yaml workspace/integration-setup.yaml
vi workspace/integration-setup.yaml          # fill in your values
./generate-setup workspace/integration-setup.yaml
```

Source `setup_env.sh` to load the environment for this shell session:

```
# Source it (required for all commands in this guide)
source setup_env.sh

# Verify
echo "Main Cluster: ${MAIN_CLUSTER_CONTEXT}"
echo "HCI Cluster:  ${HCI_CLUSTER_CONTEXT}"
```

Every new shell

Source `setup_env.sh` from the repo root in every new terminal — all commands in this guide use the `${VARIABLES}` it provides.

2.2.2 Tier A — HCI Cluster-Wide Setup

Run once per HCI cluster. Subsequent integrations skip this tier entirely.

These steps require HCI cluster-admin access. They create shared infrastructure that every main-cluster integration will use.

A.1 — Enable the HCI Image Registry

The script configures the registry for external access. It is idempotent — safe to re-run.

What it does in one shot: - Configures ODF/NooBaa S3 storage and enables the external default route - Sets `disableRedirect: true` — required for cross-cluster push; without it the registry issues S3 pre-signed redirects that clients cannot follow across the bastion boundary

```
oc config use-context ${HCI_CLUSTER_CONTEXT}
./hci-cluster/enable-image-registry-s3.sh
```

Verify:

```
oc get route default-route -n openshift-image-registry
curl -k https://$(oc get route default-route \
  -n openshift-image-registry -o jsonpath='{.spec.host}')/v2/
# UNAUTHORIZED = registry is up and reachable

oc get configs.imageregistry.operator.openshift.io/cluster \
  -o jsonpath='{.spec.disableRedirect}'
# true
```

A.2 — Apply Cluster-Scoped Network Configuration

The `ClusterUserDefinedNetwork` provides the external network attachment that every gateway VM will use. Apply it once — it is cluster-scoped and integration-namespace agnostic.

```
oc config use-context ${HCI_CLUSTER_CONTEXT}
oc apply -f hci-cluster/cluster-network-setup.yaml

# Verify
oc get clusteruserdefinednetwork
# router-external-network    Active
```

2.2.3 Tier B — HCI Per-Integration Setup

Run once per main cluster you connect. Reconnecting the same main cluster does not require re-running these steps unless the namespace or RBAC was deleted.

These steps create the HCI-side namespace, credentials, and RBAC for one specific main-cluster ↔ HCI pairing.

B.1 — Create and Label the Integration Namespace

```
oc config use-context ${HCI_CLUSTER_CONTEXT}

oc create namespace ${HCI_INTEGRATION_NAMESPACE}

# Label enables the ClusterUserDefinedNetwork controller to create the NAD
oc label namespace ${HCI_INTEGRATION_NAMESPACE} hci-gateway-enabled=true

# Verify the NetworkAttachmentDefinition was created automatically
sleep 5
oc get network-attachment-definition -n ${HCI_INTEGRATION_NAMESPACE}
# router-external-network
```

> ⚠️ **NAD must exist before deploying VMs**
>
> If the NAD is absent after labeling, check the namespace label and the `ClusterUserDefinedNetwork` status before continuing.

B.2 — Set Up Registry RBAC on the HCI Cluster

This script creates the `registry-pusher` ServiceAccount in `${HCI_INTEGRATION_NAMESPACE}` on the HCI cluster — the only credential that cannot be minted from the main cluster side. Its long-lived token is later read by `setup-registry-access.sh` (step C.6) to create the registry pull secret on the main cluster, without ever mutating the HCI cluster again.

```
# Run from the repo root with setup_env.sh already sourced
./hci-cluster/setup-consumer-rbac.sh
```

> **Recreating RBAC**
>
> If you need to rotate credentials or the SA was deleted, pass `--delete` first and then re-run:
>
> ```
> ./hci-cluster/setup-consumer-rbac.sh --delete
> ./hci-cluster/setup-consumer-rbac.sh
> ```

2.2.4 Tier C — Connect a Main Cluster

Run for each new main cluster, and again when reconnecting a previously integrated cluster.

> **Assumed execution context for Tier C**
>
> All commands in this tier are run **on the bastion host of the main cluster** as an unprivileged but `sudo`-capable user, with the repository cloned and `setup_env.sh` sourced. Commands that need elevated privileges call `sudo` explicitly — do not run an entire shell session as root.

Each step starts with a **check** command. Run the check first — if it passes, the step is already done and you move on.

C.1 — Configure DNS on the Bastion

Check: all worker hostnames resolve correctly.

```
# Run for each planned worker name
nslookup worker-x86-1.<main-cluster-domain>
nslookup 192.168.100.10   # reverse lookup
```

If any lookup fails, add the missing forward (A) and reverse (PTR) entries on the bastion DNS server.

`generate-setup` automatically derives these zone files and writes them to the `workspace/` directory:

- `workspace/<your-domain>.zone` — forward zone section with A records for the gateway VM (`hci-gateway.<domain>`) and all worker nodes up to `maxPoolSize`
- `workspace/<subnet>.in-addr.arpa.zone` — reverse zonefile with PTR records for worker nodes

Copy or append these entries on the bastion DNS server:

```
ssh user@${BASTION_HOST}

sudo vi /var/named/<your-domain>.zone
# Append contents from workspace/<your-domain>.zone:
# hci-gateway.cluster.example.com.           IN  A    172.16.1.100
# worker-x86-1.cluster.example.com.          IN  A    192.168.100.10
# worker-x86-2.cluster.example.com.          IN  A    192.168.100.11
# worker-x86-3.cluster.example.com.          IN  A    192.168.100.12

sudo vi /var/named/192.168.100.in-addr.arpa.zone
# Append/create with contents from workspace/<subnet>.in-addr.arpa.zone:
# 10    IN  PTR  worker-x86-1.cluster.example.com.
# 11    IN  PTR  worker-x86-2.cluster.example.com.
# 12    IN  PTR  worker-x86-3.cluster.example.com.

# Zone-file edits alone are picked up by a reload, but use stop/start whenever
# /etc/named.conf was touched (see the warning below)
sudo systemctl stop named-chroot
sudo systemctl start named-chroot
```

> **Use stop/start for `named-chroot`, not `reload`**
>
> On a chrooted BIND (`named-chroot`) the config file is bind-mounted into `/var/named/chroot/etc/`. An editor that saves by writing a new file gives `/etc/named.conf` a new inode, while the chroot keeps pointing at the old, unlinked one. `systemctl reload` then silently re-reads the **stale** config, so ACL changes have no effect and nothing in the log says so. A stop/start re-creates the mount. After editing `named.conf`, check that the copies match:
>
> ```
> sudo ls -li /etc/named.conf /var/named/chroot/etc/named.conf   # same size and mtime, link count 1
> ```

> **DNS is critical**
>
> Without correct forward and reverse DNS, the `csr-approver` sidecar cannot validate worker node names and CSRs will not be approved.

> **Keep DNS in sync with your WorkerPool**
>
> The zone files are generated from the same `integration-setup.yaml` as the WorkerPool CR, so they stay in sync as long as you regenerate both.
>
> | DNS config | `integration-setup.yaml` field |
> |---|---|
> | Worker name prefix (`worker-x86`) | `consumer.workers.namePrefix` |
> | Internal subnet (`192.168.100.x`) | `provider.networking.internalSubnet` |
> | Number of entries | pre-added up to `provider.maxPoolSize` |
> | Cluster domain | `consumer.domain` |

ALLOW RECURSION FOR THE GATEWAY AND THE WORKER NETWORK

The bastion DNS server is not only authoritative for the cluster zone. It is also the only resolver the HCI components use, so it must resolve **external** names (for example `mirror.openshift.com`) on their behalf. The default BIND configuration only allows recursion for `localhost` and the bastion's own networks, so queries from the gateway VM and the worker VMs are refused.

Client	Source address seen by the bastion	Needs recursion for
`hci-gateway`	gateway external IP (`provider.gateway.ip`, e.g. `172.16.1.100`)	RHCOS boot file download (`rhcos-sync`) from `mirror.openshift.com`
Worker VMs	worker IPs in `spec.provider.networking.internalSubnet` (e.g. `192.168.100.0/24`)	Name resolution during first boot, before and after joining the cluster

Check: run this **on the gateway VM** (`ssh core@<gateway-ip>`).

```
dig @<dnsServer> mirror.openshift.com
# status: NOERROR with an ANSWER section  ← pass, nothing to do
# status: REFUSED, or no answer           ← recursion is not allowed, fix below
```

> **Test from the right host**
>
> A `dig` from the bastion or from any other host that is already allowed proves nothing. The ACL is evaluated against the source address, so the test must run on the gateway (and, for the worker network, from a host in the internal subnet).

If the lookup is refused, add both networks to the recursion ACL on the bastion (file and ACL name depend on your `named.conf`):

```
sudo vi /etc/named.conf
# acl "internal_nets" {
#     ...existing entries...;
#     172.16.1.100/32;       # hci-gateway external IP (provider.gateway.ip)
#     192.168.100.0/24;      # HCI internal worker subnet (provider.networking.internalSubnet)
# };
#
# options {
#     allow-query       { localhost; internal_nets; };
#     allow-recursion   { localhost; internal_nets; };
#     allow-query-cache { localhost; internal_nets; };   # set explicitly, do not rely on defaults
# };

sudo named-checkconf

# stop/start, NOT reload — see the warning above
sudo systemctl stop named-chroot
sudo systemctl start named-chroot
sudo ls -li /etc/named.conf /var/named/chroot/etc/named.conf   # must match
```

Where the refusal shows up

A refused recursive query is logged on the bastion as `query (cache) '<name>/A/IN' denied` for the client address. The denial is reported against the *cache* ACL, so `allow-query-cache` must include the gateway as well.

A missing entry fails silently

If the gateway is refused, `rhcos-sync.service` on the gateway fails every 5 minutes with `curl: (6) Could not resolve host: mirror.openshift.com` (see `journalctl -u rhcos-sync.service`). On the operator side this only shows up as the scale-up being deferred until the gateway syncs the boot files, and `current-version.txt` never appears under `/rhcos/`.

The gateway's DNS settings are applied once

The gateway VM receives its upstream DNS server (`spec.consumer.networking.dnsServer`) through cloud-init on first boot only. Changing the recursion ACL on the bastion takes effect immediately and needs no change on the gateway, but changing `dnsServer` in the WorkerPool afterwards does not update an already-running gateway.

C.2 — Configure Bastion Routing and Firewall (Mediated Topology only)

Check: the route to the worker subnet exists on the bastion.

```
ip route show | grep "${INTERNAL_NETWORK}"
# 192.168.100.0/24 via <gateway-ip>  ← pass, nothing to do
# (no output)                        ← run the scripts below
```

If the route or firewall rules are missing, run the scripts with `sudo` (the scripts automatically locate and source `setup_env.sh` from the repository root):

```
sudo ./main-cluster/scripts/01-add-route.sh
sudo ./main-cluster/scripts/02-firewall-setup.sh
```

C.3 — Enable Multi-Architecture Support

Check: the cluster payload already supports multiple architectures.

```
oc --context ${MAIN_CLUSTER_CONTEXT} \
  get clusterversion -o jsonpath='{.items[0].status.desired.image}' | grep -i multi \
  && echo "Multi-arch already enabled — nothing to do"
```

If the check produces no output:

```
oc adm --context ${MAIN_CLUSTER_CONTEXT} upgrade --to-multi-arch=true

# Wait for completion (30–60 minutes)
oc --context ${MAIN_CLUSTER_CONTEXT} get clusterversion -w
```

C.4 — Apply MachineConfigs

Check: the `hci-vm` MachineConfigPool exists.

```
oc config use-context ${MAIN_CLUSTER_CONTEXT}
oc get mcp hci-vm 2>/dev/null && echo "MCP exists — nothing to do"
```

If absent, apply the overlay that matches your network topology (see Deployment Variants):

Mediated Topology | Direct Topology

```
oc apply -k main-cluster/configuration/overlays/mediated/

oc get mc | grep hci
# machineconfig-pool-hci-vm
# machineconfig-hci-vm-kargs

oc get mcp hci-vm
# MACHINECOUNT: 0  (no workers yet — expected)

oc apply -k main-cluster/configuration/overlays/direct/

oc get mc | grep hci
# machineconfig-pool-hci-vm
# machineconfig-hci-vm-kargs
# 99-master-s390x-hci-network-route

# Wait for master pool to finish rolling before continuing
oc get mcp master -w
```

C.5 — Configure Bastion Registry Proxy

Check: the bastion is registered as an insecure registry on the main cluster.

```
oc config use-context ${MAIN_CLUSTER_CONTEXT}
oc get image.config.openshift.io cluster \
  -o jsonpath='{.spec.registrySources.insecureRegistries}' \
  | grep -o "${BASTION_HOST}:${REGISTRY_PROXY_PORT}" \
  && echo "Registry proxy configured — nothing to do"
```

If absent, run the setup script directly on the bastion **as your normal user (not with `sudo`)**. It automatically sources `setup_env.sh`, wraps the HAProxy, firewall, and certificate commands in `sudo` itself, and runs all `oc` commands unprivileged with your own login against `${MAIN_CLUSTER_CONTEXT}`:

```
./main-cluster/bastion-registry/setup-registry-proxy.sh
```

> ⚠ **MachineConfigPool rollout**
>
> This triggers a cluster-wide MachineConfig update — all worker nodes restart in sequence. Allow 10–30 minutes.

Verify connectivity once the rollout is complete:

```
# From the bastion
curl -sk https://localhost:${REGISTRY_PROXY_PORT}/v2/ && echo ok
```

C.6 — Create the Registry Access Secret

Check: the push/pull secret exists in the operator namespace.

```
oc config use-context ${MAIN_CLUSTER_CONTEXT}
oc get secret ${REGISTRY_SECRET_NAME} -n hci-node-manager 2>/dev/null \
  && echo "Secret exists — nothing to do"
```

If absent, run the setup script. It reads the long-lived token from the `registry-pusher` SA in `${HCI_INTEGRATION_NAMESPACE}` on the HCI cluster (created in Tier B step B.2 — this script is read-only on the HCI side) and stores it as a docker-registry Secret on the main cluster:

```
# Run from repo root with setup_env.sh sourced
./main-cluster/bastion-registry/setup-registry-access.sh
```

The script creates `${REGISTRY_SECRET_NAME}` (default: `hci-registry-push`) in the `hci-node-manager` namespace.

C.7 — Create Operator Credentials

The operators need their HCI-side credential Secrets to exist before they start reconciling. Create them now, before deploying.

Check: the credentials Secrets exist in the operator namespace.

```
oc --context ${MAIN_CLUSTER_CONTEXT} \
  get secret hci-provisioner-credentials hci-provider-credentials \
  -n ${MAIN_INTEGRATION_NAMESPACE}
# Both names listed without error → credentials exist, nothing to do
# Error: secrets "..." not found  → run the script below
```

If absent, run the script from the repo root with `setup_env.sh` sourced. It connects to **both clusters** using the contexts from `setup_env.sh`, creates the `hci-consumer-operator` and `hci-provisioner-operator` ServiceAccounts with their Roles on the HCI cluster, mints long-lived tokens, and writes two kubeconfig Secrets to the main cluster — all in one shot:

```
./operator/setup-provider-credentials.sh
```

Two Secrets are created in `${MAIN_INTEGRATION_NAMESPACE}`:

Secret	Consumed by	Scope
`hci-provider-credentials`	`hci-consumer` operator	VM + DataVolume CRUD in the HCI namespace — steady-state reconcile
`hci-provisioner-credentials`	`hci-provisioner` operator	Gateway VM, UDN, SA/token management — bootstrap and teardown only

> **Rotating credentials**
>
> Re-run the script at any time to rotate tokens. Pass `--consumer` or `--provisioner` to rotate only one of the two.

C.8 — Build and Deploy the Operators

Check: both operator pods are running.

```
oc config use-context ${MAIN_CLUSTER_CONTEXT}
oc get pods -n hci-node-manager
# hci-consumer-operator-xxxx    3/3  Running  ← pass
# hci-provisioner-operator-xxxx 1/1  Running  ← pass
```

Whether deploying for the first time or rebuilding after a code change, the steps are the same — see the dedicated page for the full workflow:

Building and Deploying the Operators →

Quick reference for the bastion-proxy registry case:

```
# REGISTRY_PUSH, REGISTRY_PULL, REGISTRY_SECRET_NAME are already exported
# from setup_env.sh — no config.env editing needed.

pushd operator
make apply-buildconfigs   # one-time: namespace, ImageStreams, BuildConfigs
                          # auto-selects external-registry overlay when REGISTRY_PUSH is set
make build                # builds both operator images in-cluster
make install              # applies WorkerPool and WorkerPoolProvisioning CRDs
make deploy               # generates active overlay from environment, runs oc apply -k
                          # MAIN_CLUSTER_DOMAIN is derived automatically — nothing to set
popd

oc get pods -n hci-node-manager
# hci-consumer-operator-xxxx    3/3  Running
# hci-provisioner-operator-xxxx 1/1  Running
```

C.9 — Apply the WorkerPool CR

> **One CR — fully automated**
>
> Applying the `WorkerPool` CR is all that is needed. The operators handle everything: gateway VM, ignition, boot files, worker join, CSR approval, and node labeling.

`generate-setup` has already rendered an initial WorkerPool CR from `integration-setup.yaml` into `workspace/workerpool-hci-workers.yaml`. It has `workers.count: 0`, so applying it provisions the HCI gateway only. Workers are added afterwards by editing the live resource.

```
oc config use-context ${MAIN_CLUSTER_CONTEXT}
oc apply -f workspace/workerpool-hci-workers.yaml

# Once the pool is Ready, add workers:
oc -n ${MAIN_INTEGRATION_NAMESPACE} edit workerpool hci-workers   # set spec.consumer.workers.count
```

The generated file is a starting point and is overwritten on every `generate-setup` run. To keep the current state of a live pool, save it with `oc get workerpool hci-workers -o yaml`.

Watch both operators progress from a single terminal:

```
# WorkerPool: Provisioning → Ready → ScalingUp → Ready
oc get workerpool ${WORKERPOOL_NAME} -w

# WorkerPoolProvisioning: created automatically, shows per-step progress
oc get workerpoolprovisioning -w
```

The deployment is complete when `PHASE=Ready` and `WORKERS`, `DEPLOYED`, and `READY` are all equal with `PENDING=0`:

```
oc get workerpool
# NAME          PHASE   WORKERS   DEPLOYED   READY   PENDING   AGE
# hci-workers   Ready   3         3          3       0         12m
```

If the pool does not reach `Ready`, see Troubleshooting.

2.2.5 Scaling the Worker Pool

Once the pool is `Ready`, scaling is a single-field edit — the operator handles cordon, drain, and VM lifecycle automatically.

Scale Up

```
oc patch workerpool ${WORKERPOOL_NAME} \
  --type=merge \
  -p '{"spec":{"consumer":{"workers":{"count": 5}}}}'

oc get workerpool ${WORKERPOOL_NAME} -w
# Transitions: ScalingUp → Ready
```

Ensure DNS entries exist for every new worker IP before scaling up.

Scale Down

Scale down by reducing `workers.count`:

```
oc patch workerpool ${WORKERPOOL_NAME} \
  --type=merge \
  -p '{"spec":{"consumer":{"workers":{"count": 2}}}}'
```

The operator cordons and drains removed nodes before deleting their VMs. Watch progress:

```
oc get workerpool ${WORKERPOOL_NAME} -w
# Transitions: ScalingDown → Ready
```

2.2.6 Verify Integration

Check Node Status

```
oc get nodes -l node-role.kubernetes.io/hci-vm=""
# All HCI worker nodes should be Ready
```

Note: If the x86 workers are `Ready` but missing from this list, and the WorkerPool stays in `ScalingUp` with `readyWorkers: 0`, see Troubleshooting: WorkerPool Stuck in ScalingUp.

Verify MachineConfigPool

```
oc get mcp hci-vm
# MACHINECOUNT and READYMACHINECOUNT should match workers.count
# DEGRADED: False
```

Verify Network Configuration

```
# Check routes on a worker node (Mediated Topology)
oc debug node/<hci-worker-node> -- chroot /host ip route
# 192.168.100.0/24 via <gateway-ip>

# Check UDN on HCI cluster
oc config use-context ${HCI_CLUSTER_CONTEXT}
oc get userdefinednetwork -n ${HCI_INTEGRATION_NAMESPACE}
```

Test Workload Scheduling

```
# Schedule a test pod on an HCI worker
oc run test-pod --image=registry.access.redhat.com/ubi9/ubi-minimal:latest \
  --overrides='{"spec":{"nodeSelector":{"node-role.kubernetes.io/hci-vm":""}}}' \
  --restart=Never --command -- sleep 60

oc get pod test-pod -o wide
# Should show an HCI worker node in the NODE column

oc delete pod test-pod
```

2.2.7 Troubleshooting

Registry Proxy Issues

```
# Verify HAProxy is running on bastion
ssh user@${BASTION_HOST} systemctl status haproxy

# Check insecure-registries in cluster config
oc get image.config.openshift.io cluster \
  -o jsonpath='{.spec.registrySources.insecureRegistries}' | tr ',' '\n'

# Test push/pull secret
oc get secret ${REGISTRY_SECRET_NAME} -n hci-node-manager -o yaml
```

Gateway VM Not Starting

```
# Check provisioner logs
oc logs deployment/hci-provisioner-operator -c operator -n hci-node-manager | tail -40

# Check VM status on HCI cluster
oc config use-context ${HCI_CLUSTER_CONTEXT}
oc get vm -n ${HCI_INTEGRATION_NAMESPACE}
oc describe vm gateway-vm -n ${HCI_INTEGRATION_NAMESPACE}
```

Workers Not Booting

```
# Check that DNS resolves
nslookup worker-x86-1.${MAIN_CLUSTER_CONTEXT}

# Check DHCP/PXE logs on gateway
ssh -J user@${BASTION_HOST} cloud-user@<gateway-ip>
```

```
sudo journalctl -u dhcpd -f
sudo journalctl -u tftp -f
```

CSRs Not Being Approved

```
# Check csr-approver sidecar logs
oc logs deployment/hci-consumer-operator -c csr-approver -n hci-node-manager | tail -20

# List pending CSRs
oc get csr | grep Pending
```

Workers Not Joining Cluster

```
# Check consumer operator logs
oc logs deployment/hci-consumer-operator -c operator -n hci-node-manager | tail -40

# Check node-labeler sidecar
oc logs deployment/hci-consumer-operator -c node-labeler -n hci-node-manager | tail -20
```

2.2.8 Tear Down

```
# Delete all WorkerPool objects first (triggers drain + VM cleanup)
oc delete workerpool --all -n hci-node-manager
# Wait for finalizers to complete, then:

# Remove operators
cd operator/
make undeploy

# Remove CRDs (also deletes any remaining CRs)
make uninstall

# Remove namespace
oc delete namespace hci-node-manager
```

> ⚠️ **Uninstalling CRDs while WorkerPools exist**
>
> Running `make uninstall` while active `WorkerPool` CRs exist bypasses the finalizer that drains nodes and deletes VMs. Always delete all `WorkerPool` objects and wait for completion first.

2.2.9 Next Steps

- Configuration Reference — Main Cluster
- Configuration Reference — HCI Cluster
- Building and Deploying the Operators
- Operations and Monitoring
- Troubleshooting Guide

2.3 Deployment Variants

This document explains the two network topology variants supported by the Fusion HCI multi-architecture integration and helps you choose the right one for your environment.

2.3.1 Overview

The integration supports two deployment variants based on your main cluster's network topology:

- **Mediated Topology**: Main cluster nodes in private network behind bastion
- **Direct Topology**: Main cluster nodes on public network with direct connectivity

The choice depends on how your main cluster is deployed and networked, not on the CPU architecture itself.

2.3.2 Variant Comparison

Quick Comparison Table

Aspect	Mediated Topology	Direct Topology
Main Cluster Network	Private (10.x.x.x)	Public (172.16.x.x)
Bastion Role	Routes all traffic	Routes only if needed
Master Node Config	Standard	Requires static route
S390x Worker Config	Standard	Requires static route
Gateway VM Access	Via bastion	Direct
DNS	Bastion provides	Bastion or direct
Complexity	Medium	Low-Medium
Common In	z/VM environments	KVM environments
Setup Time	~30 min	~20 min

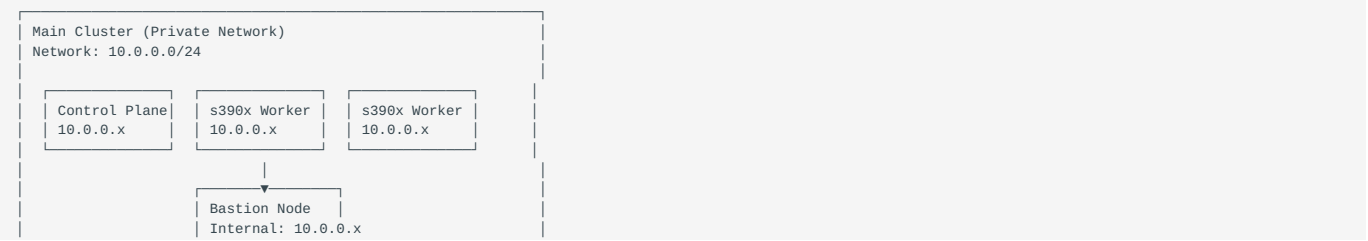
2.3.3 Mediated Topology (Private Network)

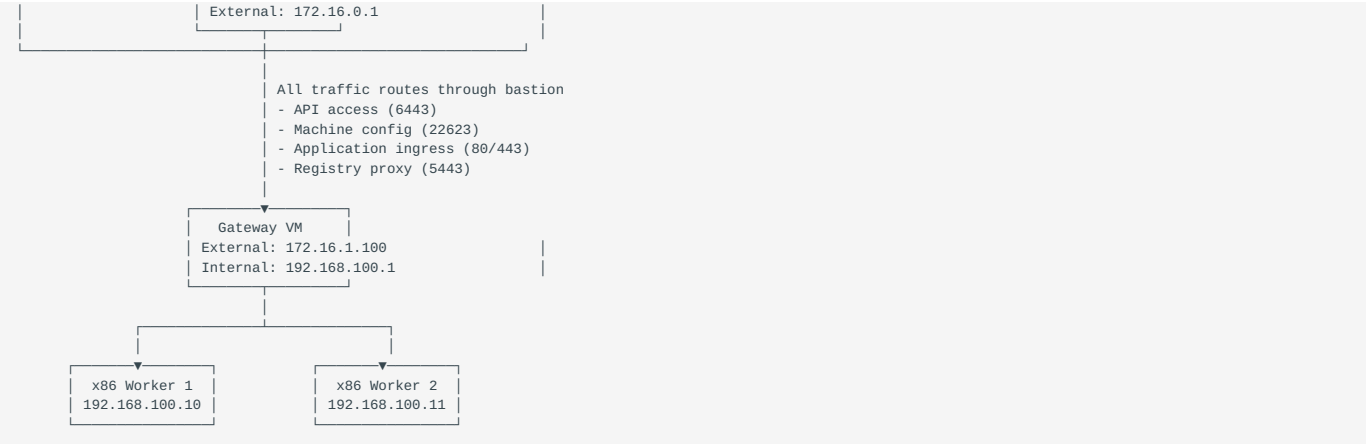
When to Use

Choose the Mediated Topology when:

- ✅ Main cluster nodes are in a private network (e.g., 10.x.x.x)
- ✅ Bastion host provides external connectivity
- ✅ Main cluster nodes cannot directly access external networks
- ✅ Common in z/VM, LPAR, or highly secured environments

Network Topology





Key Characteristics

Routing: - All cluster traffic goes through bastion - Bastion routes to HCI internal network: `192.168.100.0/24 via 172.16.1.100` - Gateway VM connects to bastion's external IP

DNS: - Bastion provides DNS for entire cluster - HCI workers use bastion as DNS server - Forward and reverse zones on bastion

Security: - Private network provides isolation - Bastion is single point of entry/exit - Firewall rules on bastion control access

Configuration: - Apply: `oc apply -k main-cluster/configuration/overlays/mediated/` - Includes: MachineConfigPool + kernel args - No master node changes needed - No s390x worker changes needed

Setup Steps

1. **Configure Bastion Routing**

```
# On bastion host
sudo ip route add 192.168.100.0/24 via 172.16.1.100

# Make persistent
sudo tee /etc/sysconfig/network-scripts/route-eth0 <<EOF
192.168.100.0/24 via 172.16.1.100
EOF
```

2. **Configure DNS on Bastion**
3. Add forward zone entries for HCI workers
4. Add reverse zone for 192.168.100.0/24
5. Ensure recursive queries allowed from HCI network
6. **Apply MachineConfigs**

```
oc apply -k main-cluster/configuration/overlays/mediated/
```

7. **Deploy Gateway and Workers**
8. Gateway VM uses bastion IP for DNS and routing
9. Workers automatically use bastion DNS

Verification

```
# From bastion, verify route
ip route | grep 192.168.100

# Test connectivity to gateway
ping 172.16.1.100  # Gateway external IP
ping 192.168.100.1   # Gateway internal IP

# Test DNS
nslookup worker-x86-1.cluster.example.com
nslookup 192.168.100.10
```

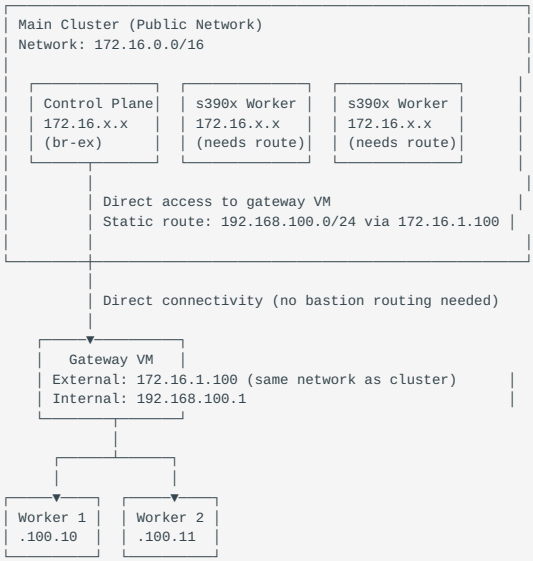
2.3.4 Direct Topology (Public Network)

When to Use

Choose the Direct Topology when:

- ✅ Main cluster nodes are on public/shared network (e.g., 172.16.x.x)
- ✅ Main cluster nodes can directly access external networks
- ✅ Gateway VM will be on same network as main cluster
- ✅ Common in KVM, bare metal, or less restricted environments

Network Topology



Key Characteristics

Routing: - Master nodes need static route to HCI internal network - S390x workers need static route to HCI internal network - Routes added via MachineConfigs: `192.168.100.0/24 via 172.16.1.100` - HCI x86 workers route through gateway VM as usual

DNS: - Can use bastion DNS or direct DNS - More flexible DNS configuration - HCI workers can use bastion or external DNS

Security: - Public network requires careful firewall rules - Direct access means more exposure - Network segmentation via firewall policies

Configuration: - Apply: `oc apply -k main-cluster/configuration/overlays/direct/` - Includes: MachineConfigPool + kernel args + **master route** + **worker route** - Master nodes will restart to apply route - S390x workers will restart to apply route

What's Included

The direct overlay includes two additional MachineConfigs for routing:

1. **99-master-s390x-hci-network-route.yaml**
2. Applies to: Master nodes
3. Purpose: Cluster management (CSR approval, scheduling, health checks)
4. Route: `192.168.100.0/24 via 172.16.1.100 dev br-ex`
5. **99-worker-s390x-hci-network-route.yaml**
6. Applies to: S390x worker nodes (via `node-platform: s390x` label)
7. Purpose: Pod-to-pod communication across architectures
8. Route: `192.168.100.0/24 via 172.16.1.100`

Setup Steps

1. **Apply MachineConfigs (includes both routes)**

```
oc apply -k main-cluster/configuration/overlays/direct/

# This applies:
# - MachineConfigPool for HCI workers
# - Kernel args for HCI workers
# - 99-master-s390x-hci-network-route (for masters)
# - 99-worker-s390x-hci-network-route (for s390x workers)
```

2. **Wait for MachineConfigPool Updates**

```
# Masters will restart one by one
watch oc get mcp master

# S390x workers will also restart
watch oc get mcp worker

# Wait for all pools:
# - UPDATED: True
# - UPDATING: False
# - DEGRADED: False
```

3. **Verify Routes**

```
# Check route on a master node
oc debug node/<master-node>
chroot /host
ip route | grep 192.168.100
# Should show: 192.168.100.0/24 via 172.16.1.100 dev br-ex

# Check route on s390x worker
oc debug node/<s390x-worker-node>
chroot /host
ip route | grep 192.168.100
# Should show: 192.168.100.0/24 via 172.16.1.100
```

4. **Deploy Gateway and Workers**
5. Gateway VM on same network as cluster
6. Workers route through gateway as usual

Verification

```
# From any master node
oc debug node/<master-node>
chroot /host

# Verify route exists
ip route | grep 192.168.100

# Test connectivity to gateway
ping 172.16.1.100  # Gateway external IP
ping 192.168.100.1   # Gateway internal IP (via route)

# From s390x worker
oc debug node/<s390x-worker>
```

```
chroot /host
ip route | grep 192.168.100
ping 192.168.100.1

# Test from HCI worker node
oc debug node/worker-x86-1
chroot /host
ping api.cluster.example.com
```

Why Both Routes Are Needed

Master Route: - Masters need to communicate with HCI workers for: - CSR approval - Node status updates - Pod scheduling decisions - Health checks

S390x Worker Route: - S390x workers need to communicate with HCI workers for: - Pod-to-pod communication across architectures - Service endpoints spanning both worker types - Network policies involving both architectures - Multi-architecture application deployments

2.3.5 Decision Guide

Choose Mediated Topology If:

- ✅ Your main cluster uses private networking (10.x.x.x)
- ✅ You have a bastion host handling all external traffic
- ✅ Your environment is z/VM, LPAR, or highly secured
- ✅ You're comfortable with bastion-centric architecture
- ✅ You want centralized traffic control

Choose Direct Topology If:

- ✅ Your main cluster uses public/shared networking (172.16.x.x)
- ✅ Cluster nodes can directly access external networks
- ✅ Your environment is KVM, bare metal, or less restricted
- ✅ You prefer direct connectivity over bastion routing
- ✅ You're comfortable with master and worker node configuration changes

Still Not Sure?

Ask yourself:

1. **Can my main cluster nodes ping 172.16.1.100 directly?**
2. Yes → Direct Topology
3. No → Mediated Topology
4. **Does all my cluster traffic go through a bastion?**
5. Yes → Mediated Topology
6. No → Direct Topology
7. **Are my cluster nodes on a 10.x.x.x network?**
8. Yes → Likely Mediated Topology
9. No → Likely Direct Topology

2.3.6 Registry Implications

Both variants work with all registry strategies, but there are considerations:

Bastion Registry Proxy

- **Mediated**: Natural fit, bastion already routes all traffic
- **Direct**: Works fine, bastion proxies registry traffic

Public Registry

- **Mediated**: Requires bastion to allow outbound internet
- **Direct**: Direct internet access, no bastion dependency

Internal Registry

- **Mediated**: Registry accessible via bastion
- **Direct**: Registry directly accessible

See Prerequisites - Registry Strategy for details.

2.3.7 Switching Between Variants

> ⚠ **Not Recommended**
>
> Switching variants after deployment requires redeploying workers and reconfiguring the main cluster. Plan your variant choice carefully.

If you must switch:

1. **Remove existing workers**

```
# Scale the WorkerPool to 0 and wait for drain + VM deletion
oc patch workerpool hci-workers \
  --type merge -p '{"spec":{"consumer":{"workers":{"count":0}}}}'
# Then delete the WorkerPool CR — the operator cleans up remaining nodes
oc delete workerpool hci-workers
```

2. **Remove old MachineConfigs**

```
oc delete mcp hci-vm
oc delete mc 99-hci-vm-kargs 99-master-s390x-hci-network-route 99-worker-s390x-hci-network-route --ignore-not-found
```

3. **Apply new variant**

```
# Update integration-setup.yaml with the new topology and run:
./generate-setup

# Or apply manually via overlay:
# For Mediated Topology:
oc apply -k main-cluster/configuration/overlays/mediated/

# For Direct Topology:
oc apply -k main-cluster/configuration/overlays/direct/
```

4. **Reconfigure networking** (bastion routes, DNS, etc.)
5. **Redeploy workers** by applying a new `WorkerPool` CR with the updated variant settings

2.3.8 Troubleshooting by Variant

Mediated Topology Issues

Workers can't reach main cluster API: - Check bastion routing: `ip route | grep 192.168.100` - Verify bastion firewall allows traffic - Test from bastion: `ping 192.168.100.1`

DNS resolution fails: - Check bastion DNS configuration - Verify recursive queries allowed from 192.168.100.0/24 - Test: `nslookup worker-x86-1.cluster.example.com`

Direct Topology Issues

Masters can't reach workers: - Check master route: `oc debug node/<master> -- chroot /host ip route | grep 192.168.100` - Verify MachineConfig applied: `oc get mc 99-master-s390x-hci-network-route` - Check master pool status: `oc get mcp master`

S390x workers can't reach HCI workers: - Check worker route: `oc debug node/<s390x-worker> -- chroot /host ip route | grep 192.168.100` - Verify MachineConfig applied: `oc get mc 99-worker-s390x-hci-network-route` - Check worker pool status: `oc get mcp worker`

Route not persisting: - Check systemd service: `systemctl status hci-network-route.service` - View logs: `journalctl -u hci-network-route.service` - For masters, verify br-ex interface exists: `ip link show br-ex`

Cross-architecture pod communication fails: - Verify routes on both master and s390x workers - Test connectivity: `oc debug node/<s390x-worker> -- chroot /host ping 192.168.100.10` - Check network policies aren't blocking traffic

2.3.9 Architecture Flexibility

> **Other Architectures**
>
> While this documentation focuses on s390x main clusters, both variants work with ARM or Power-based main clusters. The network topology (private vs public) is independent of CPU architecture.

2.3.10 Summary

Aspect	Mediated Topology	Direct Topology
Best For	Private networks, mediated access	Public networks, direct access
Bastion Role	Central router	Optional
Master Changes	None	Static route added
Worker Changes	None	Static route added (s390x)
Complexity	Medium	Low-Medium
Flexibility	Less (bastion-dependent)	More (direct access)
Security	Higher (isolated)	Medium (exposed)

Recommendation: Choose based on your existing network topology, not on preference. The variant should match how your main cluster is already deployed.

2.3.11 Next Steps

After choosing your variant:

1. Review Prerequisites for your chosen variant
2. Follow Getting Started guide
3. Apply the appropriate overlay: `overlays/mediated/` or `overlays/direct/`

2.3.12 Support

- **Mediated Topology**: Mediated overlay README

- **Direct Topology**: Direct overlay README
- **Issues**: GitHub Issues

3. Architecture

3.1 Solution Overview

A generic, architecture-level map of the core concepts and components. No cluster names, IP addresses, or low-level configuration details.

3.1.1 1 · Problem This Solves

Two independently managed OpenShift clusters — each running a different CPU architecture — are combined into a single, unified cluster. Worker nodes are sourced as virtual machines from one cluster and automatically joined to the other as fully managed, architecture-aware workers.

The integration is fully **declarative**: an administrator describes the desired number of workers in a single Custom Resource; all infrastructure provisioning and lifecycle management flows from that intent.

3.1.2 2 · The Two Clusters

	Main Cluster	HCI Cluster
Role	Destination cluster. Runs the control plane and native worker nodes. Extended with additional architecture workers sourced from the HCI cluster.	Infrastructure source cluster. Runs OpenShift Virtualization and hosts the Gateway VM and all Worker VMs.
Requirements	OpenShift 4.20+ with multi-architecture enabled. Separate **MachineConfigPool** for HCI-sourced workers.	OpenShift 4.20+ with OpenShift Virtualization. Shared storage for VM boot volumes.

3.1.3 3 · Core Infrastructure Components

Gateway VM

A dedicated virtual machine that acts as the network bridge between the two clusters. It is automatically created and managed by the provisioning operator and provides:

- **IP routing** — forwards traffic between the isolated worker network and the main cluster
- **Selective NAT** — masquerades traffic depending on destination (topology-dependent)
- **DHCP** — static reservations (MAC, IP, hostname) for the worker VMs
- **Network boot** — iPXE: loader via TFTP, boot script, RHCOS files and ignition via HTTP
- **DNS forwarding** — resolves main-cluster hostnames for workers

Worker VMs

RHCOS virtual machines created on the HCI cluster. They boot via iPXE from the gateway VM, install RHCOS from an ignition configuration, then join the main cluster as worker nodes. Fully managed by the consumer operator (scale up / scale down).

Dual OVN Networks

Two logical networks back the integration. They differ in scope:

- **External network** (localnet) — maps to the physical underlay and carries the gateway VM's external interface. It is a cluster-scoped `ClusterUserDefinedNetwork` that exists **once per HCI cluster** and is shared by all integrations. It is created by the HCI cluster admin as part of the one-time cluster setup; each integration namespace gets access to it through the label `hci-gateway-enabled=true`.
- **Internal network** (Layer 2 OVN overlay) — the isolated worker network. It is a namespace-scoped `UserDefinedNetwork` created **per main-cluster integration**, in that integration's namespace, with its own subnet (`provider.networking.internalSubnet`). The provisioning operator creates it during bootstrap and removes it on teardown.

3.1.4 4 · The Three-Operator Model

The solution is decomposed into three operators with separate responsibilities. Each operator handles one phase of the lifecycle.

Operator	Runs on	Responsibility	Status
HCI Consumer Operator	Main cluster	Steady-state worker lifecycle: scale VMs up/down, cordon/drain before removal.	✅ Complete
HCI Provisioning Operator	Main cluster	Infrastructure bootstrap & teardown: creates the UDN and Gateway VM, generates the provisioner SSH key and distributes ignition to the gateway.	✅ Complete
HCI Provider Operator	HCI cluster	HCI-side namespace & quota provisioning; generates time-limited onboarding tickets for new consumers.	🗓️ Planned

> ✏️ **Supporting Component: HCI Node Manager**
>
> A lightweight permanent process (not an operator) running on the main cluster. Uses a stream-watch (millisecond reaction time) to handle two time-sensitive operations that cannot wait for an operator reconcile cycle:
>
> - Automatic CSR approval for joining worker nodes
> - Immediate node labeling (assigns to the HCI MachineConfigPool)

3.1.5 5 · Key Custom Resources

WorkerPool

The primary control surface. An administrator applies one CR specifying:

- Desired worker count
- Network topology variant
- Target namespace on the HCI cluster

All provisioning and scaling flows from this single declaration.

WorkerPoolProvisioning

An internal coordination board, auto-created by the `WorkerPool` operator. Tracks per-step status for each bootstrap or teardown step (UDN, Gateway VM, MachineConfig, Ignition). Not managed by the administrator; short name `wpp`.

3.1.6 6 · Two Deployment Topologies

The topology choice is driven by how the main cluster is attached to the network — not by CPU architecture. It is set once at cluster-pair setup time.

Mediated Topology **Direct Topology**

For main clusters on a **private network**. A bastion node acts as an additional routing hop between the main cluster and the gateway VM.

- No changes to master or worker node routing tables
- Bastion provides routing and DNS for the main cluster side
- Simpler main-cluster configuration

```
Main Cluster → Bastion Node → Gateway VM → Worker VMs
```

For main clusters on a **routable (public) network**. Master and worker nodes have static routes applied via MachineConfig to reach the worker VM network directly through the gateway.

- No bastion required as a routing middleman
- Static routes pushed via MachineConfig (managed in Git)
- Required on both master nodes and same-arch worker nodes

```
Main Cluster → Gateway VM → Worker VMs
```

3.1.7 7 · Key Lifecycle Flows

Bootstrap Flow (one-time per consumer)

```
WorkerPool CR applied
  → WorkerPoolProvisioning CR created
  → MachineConfig (MCP and routes) validated on the main cluster
  → Provisioner creates internal UDN
  → Gateway VM created & SSH-verified
  → Ignition distributed
  → Consumer phase → Ready
```

VM Provisioning Flow (per worker node)

```
DataVolume created
  → VirtualMachine created
  → iPXE boot via Gateway DHCP/TFTP/HTTP
  → RHCOS install via Ignition
  → Kubelet starts, bootstrap + serving CSRs generated
  → csr-approver sidecar approves both CSRs
  → Node labeled → joins MachineConfigPool
  → MachineConfig applied → Ready  (~20–30 min)
```

Scale-Down Flow

```
WorkerPool count reduced
  → Node cordoned & drained
  → Operator waits for NotReady
  → VM & DataVolume deleted
```

3.1.8 8 · Privilege & Credential Model

The operators on the main cluster reach the HCI cluster with a single ServiceAccount token. A setup script creates the ServiceAccount, its Role and RoleBinding on the HCI cluster and mints a token once per integration; the token is stored as a kubeconfig Secret on the main cluster. The script needs administrative access to the HCI integration namespace, which is why the operators never create credentials themselves.

Credential	Scope	Lifecycle
HCI operator ServiceAccount token (Secret `hci-provisioner-credentials`)	Namespace-scoped Role in the integration namespace on HCI: VMs, DataVolumes, PVCs, UDN, Secrets, ConfigMaps. Read-only access to the cluster-scoped `ClusterUserDefinedNetwork`. The Role also allows ServiceAccount, Role and RoleBinding management, reserved for a credentials step that is not implemented yet.	Valid for 1 year, rotated by re-running the script. Stored in the operator namespace on the main cluster and used by both operators.
Optional: registry push token (`registry-pusher` ServiceAccount)	Push images to the HCI image registry from the integration namespace. Needed only for the bastion registry proxy (`registry.model: cross-cluster`).	Non-expiring token created with the integration RBAC, read once and stored as a docker-registry Secret on the main cluster.
HCI cluster-admin	Full HCI admin: one-time cluster setup (network, registry), integration namespace, RBAC and token creation	Human/admin only, never stored in-cluster

One Secret for both operators

`WorkerPool.spec.provider.credentialsSecret` is a single field, passed on to the `WorkerPoolProvisioning` CR, so the consumer and the provisioning operator read the same Secret. The setup script can also create a second, narrower consumer-only ServiceAccount and Secret (`hci-provider-credentials`), but the operators do not use it in the default configuration.

Planned: onboarding tickets

Short-lived, elevated onboarding tickets generated by the HCI Provider Operator are part of the planned design (backlog item HCI-04). They are not implemented.

3.1.9 9 · Configuration & Variant Management

Kustomize Overlays

Main-cluster MachineConfigs are managed as a **base + overlay** set. The base contains configuration common to all variants. Each topology overlay adds only what differs. New topologies can be added without touching the base.

Cluster-Pair Setup File (`integration-setup.yaml`)

A single YAML input file captures all cluster-pair decisions once: the two clusters, network topology, MachineConfigPool and image registry model. The `generate-setup` script derives all deployment artifacts from it — shell environment, build configuration, MachineConfigs, DNS zones and an initial WorkerPool skeleton — eliminating scattered hard-coded values. See Setup Concept for the design rationale.

3.1.10 10 · Guiding Design Principles

- **Declarative intent** — a single WorkerPool CR is the sole operational control surface.

- **Idempotent reconciliation** — the operator can be restarted at any point and resume correctly; all state is derived from Kubernetes resources.
- **Least privilege on HCI** — the operators reach the HCI cluster only through a namespace-scoped ServiceAccount; no HCI cluster-admin credential is ever stored in-cluster.
- **Observe → Diff → Act** — the standard Kubernetes controller pattern applied consistently across all operator roles.
- **Topology abstraction** — the same operator code handles mediated and direct deployments via configuration, not branching code paths.
- **Safe scale-down** — a node must reach `NotReady` before its VM is deleted; no data loss or abrupt eviction.
- **Namespace isolation** — each consumer gets a dedicated namespace and network on the HCI cluster; resources cannot leak across tenants.

3.1.11 Closest Analogy in the OpenShift Ecosystem

The solution mirrors the **ODF Storage Provider / Consumer model**: one cluster contributes a resource (storage / compute), another cluster consumes it, mediated by a pair of operators that manage the cross-cluster lifecycle. The difference here is that the resource is *compute* (virtual machines as worker nodes) rather than storage.

3.2 Architecture Overview

This document provides a comprehensive overview of the Fusion HCI multi-architecture integration architecture, explaining how x86 worker nodes running on a Fusion HCI cluster can be integrated into a main OpenShift cluster running on a different architecture (s390x, ARM, or Power).

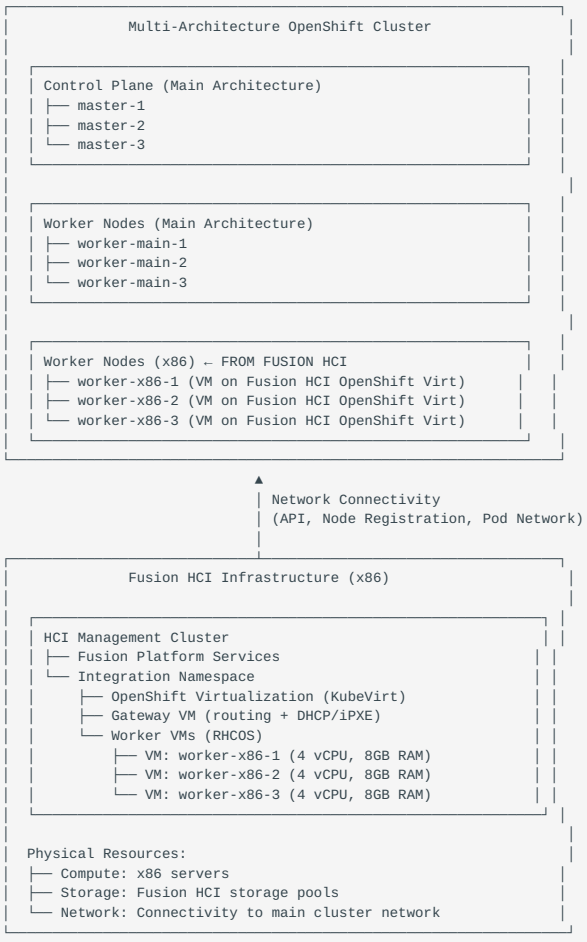
3.2.1 Executive Summary

The Fusion HCI multi-architecture integration enables:

- **Main cluster** runs on one architecture (e.g., s390x/LinuxONE, ARM, or Power)
- **Additional x86 worker nodes** provided as VMs from Fusion HCI via OpenShift Virtualization
- **VMs created** in the HCI management cluster within a dedicated namespace
- **Single unified cluster** with mixed architecture worker nodes

This architecture enables workloads requiring x86 architecture to run alongside the main cluster's native architecture workloads, with unified management and orchestration.

3.2.2 High-Level Architecture



3.2.3 Key Concepts

Multi-Architecture Cluster

OpenShift supports running worker nodes with different CPU architectures in the same cluster:

- **Control plane** must be single architecture (e.g., s390x, ARM, or Power)
- **Worker nodes** can be mixed architectures (main + x86)
- **Container images** must be multi-arch manifests or architecture-specific
- **Workload scheduling** uses node selectors and tolerations for architecture targeting

Why This Architecture?

Benefits:

1. **Unified Management**: Single cluster for all workloads
2. **Resource Optimization**: Use x86 hardware for x86-specific workloads
3. **Flexibility**: Run workloads on optimal architecture
4. **Cost Efficiency**: Leverage existing Fusion HCI investment
5. **Simplified Operations**: One cluster to manage, monitor, and secure

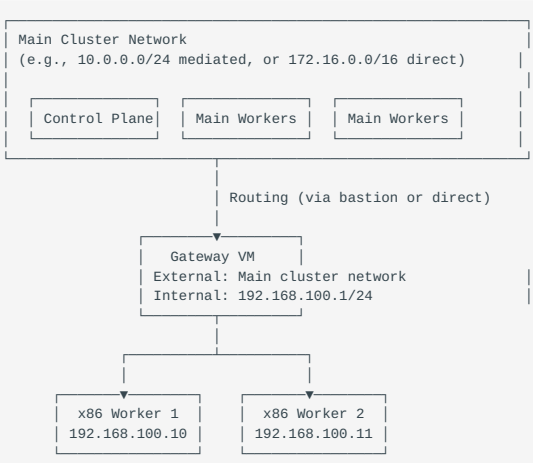
Use Cases:

- Legacy x86 applications that can't be recompiled for other architectures
- Third-party software only available for x86
- Development/testing of multi-arch applications
- Gradual migration strategies
- Workload-specific performance optimization

3.2.4 Network Architecture

Network Topology

The integration uses a dual-network approach:



Network Components

1. EXTERNAL NETWORK (LOCALNET)

- **Purpose**: Connects gateway VM to main cluster network

- **Technology**: OVN-Kubernetes ClusterUserDefinedNetwork with localnet topology
- **Configuration**: Maps to physical network interface on HCI nodes
- **IP Assignment**: Static IP on the gateway VM, set via cloud-init

2. INTERNAL NETWORK (LAYER2)

- **Purpose**: Isolated network for HCI worker VMs
- **Technology**: OVN-Kubernetes UserDefinedNetwork with Layer2 topology
- **Network**: 192.168.100.0/24 (set by `provider.networking.internalSubnet`)
- **Gateway**: 192.168.100.1 (network address + `.1`, the gateway VM)
- **IP Assignment**: Gateway VM static (cloud-init); workers via static DHCP reservations on the gateway

3. GATEWAY VM

The gateway VM provides critical services:

- **Routing**: Forwards traffic between internal and external networks
- **DHCP**: Static reservations (MAC, IP, hostname) for the worker VMs
- **Network boot**: iPXE (TFTP + HTTP) for RHCOS installation
- **DNS Forwarding**: Resolves main cluster hostnames
- **NAT**: Selective masquerading for internet traffic

Gateway VM Specifications (defaults; override with `provider.gateway.resources`): - OS: CentOS Stream 9 (or similar) - vCPU: 2 - Memory: 4GB - Storage: 30GB - Network Interfaces: 2 (external + internal)

Routing Strategies

The integration supports two routing variants based on main cluster network topology:

MEDIATED TOPOLOGY (PRIVATE NETWORK)

- Main cluster nodes in a private network (e.g., 10.0.0.0/24, `consumer.networking.internalNetwork`)
- Bastion host routes all traffic
- Gateway VM connects to bastion's external IP
- No changes to main cluster nodes

DIRECT TOPOLOGY (PUBLIC NETWORK)

- Main cluster nodes on a public/shared network (e.g., 172.16.0.0/16)
- Direct connectivity between nodes and gateway VM
- Static routes added to master and worker nodes
- More flexible but requires node configuration

See Deployment Variants for detailed comparison.

3.2.5 Component Architecture

HCI Cluster Components

1. CLUSTER-ADMIN RESOURCES (CLUSTER-SCOPED)

Applied once by cluster administrator:

- **ClusterUserDefinedNetwork**: External network definition
- **Namespace Labels**: Enable network access per namespace

2. INTEGRATION NAMESPACE RESOURCES

Deployed per integration namespace:

- **UserDefinedNetwork**: Internal worker network
- **Gateway VM**: Routing and services
- **Worker VMs**: x86 worker nodes
- **Secrets**: Gateway cloud-init data, SSH keys
- **ConfigMaps**: Network configuration

Main Cluster Components

1. MACHINECONFIGPOOL

Separate pool for HCI workers:

- **Name**: `hci-vm`
- **Selector**: `node-role.kubernetes.io/hci-vm: ""`
- **Purpose**: Isolate HCI worker configuration

2. MACHINECONFIGS

Configuration for HCI workers:

- **Kernel Arguments**: Boot parameters, multipath settings
- **Network Configuration**: Interface management (variant-specific)
- **Kernel arguments**: x86 VM-specific boot parameters (disable multipath, configure interfaces)

3. HCI NODE MANAGER (SIDECARS)

Deployed as sidecar containers inside the `hci-consumer-operator` pod:

- **CSR Auto-Approval** (`csr-approver`): stream-watch, millisecond reaction time
- **Node Labeling** (`node-labeler`): applies `node-role.kubernetes.io/hci-vm: ""` before MCO renders ignition
- **Permanent**: re-reads config from `hci-node-manager-config` ConfigMap every ~30 s

4. HCI-CONSUMER OPERATOR ✅ COMPLETE

Kubernetes operator for steady-state worker VM lifecycle (Epics 1 and 2 complete, Epics 3–4 in backlog):

- **CRD**: `WorkerPool` (`hci-integration.openshift.io/v1alpha1`) — declarative desired worker count
- **Scale-up**: creates `DataVolume` + `VirtualMachine` on HCI when `workers.count` increases
- **Scale-down**: deletes VM and node when `workers.count` decreases, gated on node NotReady state
- **Observe/diff/act**: all API calls isolated in `observe_world.yml`; `diff_desired_actual.yml` is pure set arithmetic
- `maxPoolSize` **enforcement**: CEL admission rule on CRD + runtime clamping with CR condition

See the hci-consumer operator README and the Operator Roadmap for full implementation status.

3.2.6 Deployment Flow

Configuration Input

All cluster-pair decisions are captured once in a single file, `workspace/integration-setup.yaml`, filled in jointly by the HCI admin (`provider.*`) and the main cluster admin (`consumer.*`). The `generate-setup` script derives everything else from it: `setup_env.sh`, `operator/build/config.env`, the direct-topology route MachineConfigs, the DNS zone files and an initial `WorkerPool` CR with `workers.count: 0`. See Setup Concept for the design and Prerequisites for the how-to.

The concrete values shown as examples throughout the architecture pages map to these fields:

Example value in these pages	**`integration-setup.yaml` field**
`mediated` or `direct` routing	`consumer.networking.topology`
Main cluster domain `cluster.example.com`	`consumer.domain`
DNS server `172.16.0.1` (the bastion)	`consumer.networking.dnsServer`
Main cluster internal network `10.0.0.0/24` (mediated only)	`consumer.networking.internalNetwork`
Gateway towards that network `172.16.0.1` (mediated only)	`consumer.networking.internalGateway`
MachineConfigPool `hci-vm`	`consumer.machineConfigPool` (default `hci-vm`)
Operator namespace `hci-node-manager`	`consumer.operatorNamespace`
Worker name prefix `worker-x86`	`consumer.workers.namePrefix`
Worker VM resources (4 vCPU, 8Gi, 50Gi)	`consumer.workers.resources` (operator defaults)
HCI namespace `main-cluster-workers`	`provider.namespace`
Gateway external IP `172.16.1.100`, prefix `16`, default router `172.16.0.1`	`provider.gateway.ip`, `.prefixLength`, `.defaultRouter`
Gateway VM resources (2 vCPU, 4Gi, 30Gi)	`provider.gateway.resources` (operator defaults)
Internal worker network `192.168.100.0/24`	`provider.networking.internalSubnet`
MAC address prefix `52:54:01`	`provider.networking.macPrefix` (default `52:54:00`)
Maximum number of workers `10`	`provider.maxPoolSize`
Image registry strategy	`registry.model`

Phase 1: Prerequisites

1. Main cluster operational with OpenShift 4.20+
2. HCI cluster operational with OpenShift Virtualization
3. Network connectivity between clusters
4. Registry strategy selected (`registry.model`)
5. `workspace/integration-setup.yaml` completed and `generate-setup` run — this also produces the DNS zone files to be added on the bastion

Phase 2: Main Cluster Preparation

1. Create MachineConfigPool for HCI workers
2. Apply MachineConfigs (base + variant overlay; the direct-topology route MachineConfigs are generated by `generate-setup`)
3. Deploy HCI node manager (optional but recommended)
4. Configure registry access (if using bastion proxy)

Phase 3: HCI Cluster Preparation

1. Cluster admin applies ClusterUserDefinedNetwork
2. Create and label integration namespace
3. Run `setup-provider-credentials.sh` to create operator credential Secrets

Phase 4: Deployment

1. Apply the generated `WorkerPool` CR (`workspace/workerpool-hci-workers.yaml`, `workers.count: 0`) — `hci-consumer` creates the `WorkerPoolProvisioning` CR automatically
2. `hci-provisioner` creates UDN, gateway VM, ignition, and bootfiles; sets `WorkerPool` phase → `Ready`
3. Set `workers.count` on the live `WorkerPool`; `hci-consumer` scales up and worker VMs network-boot (iPXE) and install RHCOS
4. Workers join main cluster; CSRs approved automatically by the `csr-approver` sidecar; nodes become Ready

Phase 5: Verification

1. Verify nodes appear in main cluster
2. Check node labels and MachineConfigPool membership
3. Test workload deployment on x86 workers
4. Verify cross-architecture pod communication

3.2.7 Security Considerations

Namespace Isolation

- HCI workers deployed in dedicated namespace
- Network policies control traffic flow
- RBAC limits access to integration resources
- Separate service accounts for components

Network Segmentation

- Internal worker network isolated from HCI pod network
- Gateway VM controls all external access
- Firewall rules on gateway VM
- Optional network policies on main cluster

Certificate Management

- Main cluster CA trusted by HCI workers
- Node certificates issued by main cluster
- TLS for all API communication
- Automatic certificate rotation

Access Control

- SSH keys managed via secrets
- Ignition configuration secured
- RBAC for HCI cluster resources
- Audit logging enabled

3.2.8 Performance Considerations

Network Latency

- **Gateway VM overhead**: ~0.5-1ms additional latency
- **Cross-cluster communication**: Depends on physical network

- **Mitigation**: Place gateway VM on low-latency nodes

Nested Virtualization

- **HCI workers are VMs**: Some overhead vs bare metal
- **CPU performance**: ~5-10% overhead
- **Memory**: Direct mapping, minimal overhead
- **Storage**: Depends on HCI storage backend

Resource Allocation

Gateway VM: - 2 vCPU, 4GB RAM (minimal overhead) - Can be increased for high-throughput scenarios

Worker VMs: - Default: 4 vCPU, 8GB RAM, 50GB disk per worker (`consumer.workers.resources`) - Adjust based on workload requirements - Consider HCI node capacity

3.2.9 Scalability

Horizontal Scaling

- **Add workers**: Increase `WorkerPool.spec.consumer.workers.count`
- **Multiple namespaces**: Support multiple integration namespaces
- **Gateway HA**: Deploy multiple gateway VMs for redundancy

Vertical Scaling

- **Worker resources**: Adjust vCPU/memory per worker
- **Gateway resources**: Increase for high-throughput scenarios
- **Storage**: Expand worker VM disks as needed

Limitations

- **Network bandwidth**: Gateway VM is potential bottleneck
- **HCI capacity**: Limited by physical HCI resources
- **IP address space**: Internal network size (default /24 = 254 hosts)

3.2.10 High Availability

Gateway VM HA

Options for gateway VM redundancy:

1. **Active-Passive**: Deploy backup gateway VM
2. **Active-Active**: Multiple gateways with load balancing
3. **VM Live Migration**: Leverage OpenShift Virtualization HA

Worker VM HA

- **Pod disruption budgets**: Protect workloads during maintenance
- **Node affinity**: Spread workers across HCI nodes
- **VM live migration**: Move VMs without downtime

Network HA

- **Multiple gateway VMs**: Eliminate single point of failure
- **Redundant routing**: Multiple paths to main cluster
- **Monitoring**: Detect and alert on network issues

HCI Cluster-Level HA

The HA mechanisms above (live migration, node affinity, gateway redundancy) all operate **within** a single Fusion HCI cluster. They protect against individual HCI host failures but cannot protect against a full HCI cluster outage.

For deployments where loss of all x86 workers is unacceptable, the recommended approach is **dual-HCI**: two independent Fusion HCI units, each with its own `WorkerPool` CR on the main cluster. Worker VMs are spread across both units; if one HCI cluster goes down, the other continues to supply x86 capacity. This requires the main cluster workloads to tolerate reduced x86 capacity during a single-unit outage (e.g. via pod disruption budgets and appropriate replica counts).

3.2.11 Monitoring and Observability

Key Metrics

Gateway VM: - Network throughput (in/out) - Packet loss rate - CPU and memory utilization - DHCP lease activity

Worker VMs: - Node status and readiness - Resource utilization - Network connectivity - Pod scheduling success rate

Integration Health: - CSR approval rate - Node join time - Cross-architecture pod communication latency

Logging

- Gateway VM logs (dnsmasq, firewalld, NetworkManager)
- Worker VM console logs
- Main cluster node logs
- HCI cluster virtualization logs

Alerting

- Gateway VM down
- Worker VM failed to join
- Network connectivity issues
- Resource exhaustion

3.2.12 Operational Procedures

Adding Workers

1. Increase `WorkerPool.spec.consumer.workers.count`
2. The `hci-consumer` operator creates the additional VMs automatically
3. Monitor CSR approval and node join via `oc get workerpool -w`
4. Verify new nodes are Ready

Removing Workers

1. Decrease `WorkerPool.spec.consumer.workers.count`
2. The operator cordons and drains surplus nodes, then deletes the VMs
3. `WorkerPool.status.phase` shows `ScalingDown` / `ScalingDownBlocked` until complete

Upgrading

Main Cluster Upgrade: - HCI workers upgrade with main cluster - MachineConfigPool manages rolling updates - Workers reboot to apply new configuration

HCI Cluster Upgrade: - Gateway VM may need recreation - Worker VMs unaffected (run RHCOS from main cluster) - Plan maintenance window for gateway VM updates

Troubleshooting

See Troubleshooting Guide for detailed troubleshooting procedures.

3.2.13 Best Practices

Deployment

1. **Start small**: Deploy 1-2 workers initially
2. **Test thoroughly**: Verify functionality before scaling
3. **CSR approval is automatic**: the `csr-approver` sidecar in the operator handles it
4. **Document configuration**: Keep `WorkerPool` and `WorkerPoolProvisioning` CR manifests in version control

Operations

1. **Monitor continuously**: Set up alerts for critical metrics
2. **Regular backups**: Back up CR manifests and configurations
3. **Plan maintenance**: Schedule updates during low-traffic periods
4. **Test disaster recovery**: Verify recovery procedures work

Security

1. **Least privilege**: Use minimal RBAC permissions
2. **Network policies**: Restrict traffic where possible
3. **Regular updates**: Keep gateway VM OS updated
4. **Audit logs**: Review logs regularly for anomalies

3.2.14 Limitations and Known Issues

Current Limitations

1. **Single gateway VM**: Potential single point of failure (HA possible but not default)
2. **Network bandwidth**: Gateway VM throughput limited by VM resources
3. **IP address space**: Internal network /24 limits to ~250 workers
4. **CSR approval and node labeling**: Handled automatically by the `csr-approver` and `node-labeler` sidecars in the `hci-consumer-operator` Deployment
5. **HCI availability is a hard dependency for the main cluster**: All x86 worker nodes run as VMs on the Fusion HCI box. If the HCI cluster becomes unavailable (hardware failure, maintenance, power loss), the main cluster loses those worker nodes and has no path to replace them until HCI recovers. Intra-HCI redundancy (VM live migration, node affinity across HCI hosts) mitigates individual host failures within the HCI box, but does **not** protect against a full HCI cluster outage. **HA implication**: deployments with strict availability requirements should operate two independent Fusion HCI units, each backing a separate `WorkerPool` CR, so that the main cluster retains x86 capacity if one HCI unit is lost.

Known Issues

1. **OVN IPAM with localnet**: OVN IPAM is disabled; the gateway VM's IPs are set via cloud-init and the workers' IPs via DHCP reservations on the gateway
2. **Dual network conflict**: Worker VMs must disable HCI pod network interface
3. **MachineConfig inheritance**: Requires platform labeling to prevent conflicts

Workarounds

See Troubleshooting Guide for detailed workarounds.

3.2.15 Future Enhancements

Short-term

- Gateway VM HA implementation
- Enhanced monitoring dashboards
- Automated testing framework
- Performance optimization guide

Long-term

- Multi-gateway load balancing
- Integration with cluster autoscaler

3.2.16 References

Documentation

- Getting Started Guide
- Prerequisites
- Deployment Variants
- Troubleshooting Guide
- Developer Notes

OpenShift Documentation

- Multi-Architecture Compute
- OpenShift Virtualization
- OVN-Kubernetes
- MachineConfig

3.2.17 Conclusion

The Fusion HCI multi-architecture integration provides a robust, scalable solution for extending OpenShift clusters with x86 worker nodes. By leveraging OpenShift Virtualization and OVN-Kubernetes networking, the integration maintains cloud-native principles while enabling multi-architecture workload deployment.

The architecture balances complexity, performance, and operational requirements, providing a production-ready solution for organizations needing multi-architecture capabilities.

3.3 Network Design

This document provides detailed information about the network architecture used in the Fusion HCI multi-architecture integration.

3.3.1 Overview

The network design uses a dual-network approach with OVN-Kubernetes to provide:

- **External connectivity** to the main cluster network
- **Internal isolation** for HCI worker VMs
- **Gateway routing** between networks
- **DHCP and network boot services** for worker provisioning

> **Nothing on this page is configured by hand**
>
> The networks, gateway VM configuration (addressing, routing, NAT, firewall, dnsmasq) and worker NICs described here are created by the operators from the values in `integration-setup.yaml`. The snippets show *what the operators set up* so that you can understand and troubleshoot it; they are not steps to execute. The only manual network step is the one-time external network on the HCI cluster (see Localnet).

3.3.2 Network Topologies

OVN-Kubernetes Network Types

The integration uses two OVN-Kubernetes network topologies:

1. LOCALNET (EXTERNAL NETWORK)

Purpose: Connect gateway VM to physical network

Characteristics: - Maps directly to physical network interface - Provides external connectivity - No OVN overlay encapsulation - Direct Layer 2 access to physical network

Configuration:

```yaml
apiVersion: k8s.ovn.org/v1
kind: ClusterUserDefinedNetwork
metadata:
  name: router-external-network
spec:
  namespaceSelector:
    matchLabels:
      hci-gateway-enabled: "true"
  network:
    topology: Localnet
    localnet:
      physicalNetworkName: localnet1
      role: Secondary
      mtu: 1500
      ipam:
        mode: Disabled  # no OVN IPAM; see IP Assignment Methods
```

Key Points: - Cluster-scoped resource (one per HCI cluster), created once by the HCI cluster admin with `hci-cluster/cluster-network-setup.yaml` (see Getting Started) - Each integration namespace on the HCI cluster must carry the label `hci-gateway-enabled=true` to attach to it - Requires physical network mapping on HCI nodes - IPAM disabled - IPs are set statically on the gateway VM and via DHCP reservations for workers - Multiple namespaces can use same network

2. LAYER2 (INTERNAL NETWORK)

Purpose: Isolated network for HCI worker VMs

Characteristics: - Pure OVN logical switch - No physical network mapping - Isolated from HCI pod network - Gateway VM provides routing

Configuration:

```yaml
apiVersion: k8s.ovn.org/v1
kind: UserDefinedNetwork
metadata:
  name: router-internal-network
  namespace: main-cluster-workers
spec:
  topology: Layer2
  layer2:
    role: Secondary
    ipam:
      mode: Disabled  # no OVN IPAM; see IP Assignment Methods
```

Key Points: - Namespace-scoped resource, created per integration by `hci-provisioner` in `provider.namespace` (UDN step) - Completely isolated from other networks - No MTU is set; the OVN-Kubernetes default applies (see MTU Considerations) - Gateway VM bridges to external network

3.3.3 Network Addressing

Address Allocation

All addresses, names and sizes on this page are the example values from `templates/integration-setup.example.yaml`. The real values are set once in `integration-setup.yaml`; see the field mapping in Architecture Overview.

```
External Network (varies by deployment):
├── Mediated Topology: main cluster 10.0.0.0/24 (consumer.networking.internalNetwork)
├── Direct Topology: 172.16.0.0/16 (derived from provider.gateway.ip / prefixLength)
├── Bastion / DNS server: 172.16.0.1 (consumer.networking.dnsServer)
└── Gateway VM: 172.16.1.100 (provider.gateway.ip)

Internal Network (provider.networking.internalSubnet):
├── Network: 192.168.100.0/24
├── Gateway: 192.168.100.1 (network address + .1)
├── Workers: 192.168.100.10-19 (network address + .10, one per slot up to provider.maxPoolSize = 10)
└── Remainder: unused
```

IP Assignment Methods

OVN IPAM is disabled on both networks. Addresses are assigned in two different ways:

GATEWAY VM: STATIC CONFIGURATION VIA CLOUD-INIT

The `hci-provisioner` renders the gateway's NetworkManager connections into its cloud-init data. The external NIC gets the address from `provider.gateway.ip`:

```
# external connection (eth1), MAC 52:54:01:00:00:01
[ipv4]
method=manual
addresses=172.16.1.100/16        # provider.gateway.ip / prefixLength
gateway=172.16.0.1               # provider.gateway.defaultRouter
dns=172.16.0.1                   # consumer.networking.dnsServer
# mediated topology only:
route1=10.0.0.0/24,172.16.0.1    # consumer.networking.internalNetwork, internalGateway
```

The internal NIC (eth2) gets the first address of `provider.networking.internalSubnet` (`192.168.100.1/24`).

WORKER VMS: STATIC DHCP RESERVATIONS ON THE GATEWAY

Worker VMs have no cloud-init. The gateway's `dnsmasq` runs DHCP in static-only mode, with one reservation per possible worker slot (up to `provider.maxPoolSize`):

```
# /etc/dnsmasq.d/pxe.conf (generated)
dhcp-range=192.168.100.0,static
dhcp-option=3,192.168.100.1               # default gateway = gateway VM
dhcp-option=6,172.16.0.1                  # DNS = consumer.networking.dnsServer

# one pair per slot: MAC -> IP, and the hostname for that slot
dhcp-host=52:54:01:00:00:10,192.168.100.10,set:host1
dhcp-option=tag:host1,12,worker-x86-1.cluster.example.com
dhcp-host=52:54:01:00:00:11,192.168.100.11,set:host2
```

```
dhcp-option=tag:host2,12,worker-x86-2.cluster.example.com
# ... up to slot 10 (192.168.100.19)
```

Advantages: - Predictable IP address and hostname per worker slot - No dynamic lease pool to manage; scaling within the pool limit needs no gateway change - No dependency on OVN IPAM - Supports network boot

3.3.4 Gateway VM Architecture

Network Interfaces

The gateway VM has two network interfaces:

```
┌──────────────────────────────────────┐
│         Gateway VM                   │
│                                      │
│  ┌──────────────────────────────┐    │
│  │ eth1 (external)              │    │
│  │ - Localnet topology          │    │
│  │ - Physical network access    │    │
│  │ - IP: From external network  │    │
│  │ - MAC: 52:54:01:00:00:01     │    │
│  └──────────────────────────────┘    │
│                                      │
│  ┌──────────────────────────────┐    │
│  │ eth2 (internal)              │    │
│  │ - Layer2 topology            │    │
│  │ - OVN logical switch         │    │
│  │ - IP: 192.168.100.1/24       │    │
│  │ - MAC: 52:54:01:00:00:02     │    │
│  └──────────────────────────────┘    │
└──────────────────────────────────────┘
```

Routing Configuration

The gateway VM performs IP routing between networks:

```
# Enable IP forwarding
sysctl -w net.ipv4.ip_forward=1

# Route to main cluster (example)
ip route add 10.0.0.0/24 via 172.16.0.1

# Default route via external gateway
ip route add default via 172.16.0.1
```

NAT Configuration

Selective NAT for different traffic types:

```
# Transparent routing (no NAT) for main cluster
iptables -t nat -A POSTROUTING -s 192.168.100.0/24 \
  -d 10.0.0.0/24 -j ACCEPT

# Transparent routing (no NAT) for external network
iptables -t nat -A POSTROUTING -s 192.168.100.0/24 \
  -d 172.16.0.0/16 -j ACCEPT

# Masquerading (NAT) for internet
iptables -t nat -A POSTROUTING -s 192.168.100.0/24 \
  -o eth1 -j MASQUERADE
```

Why Selective NAT? - Main cluster needs to see real worker IPs for node management - External network may need real IPs for monitoring/logging - Internet traffic can use NAT for simplicity

3.3.5 Routing Strategies

Mediated Topology (Private Network)

```
Main Cluster (10.0.0.0/24)
         ↓
    Bastion Host
    (internal side on 10.0.0.0/24)
    (172.16.0.1 external)
         ↓
   Route: 192.168.100.0/24 via 172.16.1.100
```

```
        ↓
    Gateway VM
    (172.16.1.100 external)
    (192.168.100.1 internal)
        ↓
    Worker VMs
    (192.168.100.10+)
```

Configuration: - Bastion routes to internal network - No changes to main cluster nodes - All traffic through bastion - Selected with `consumer.networking.topology: mediated`; the bastion route (`main-cluster/scripts/01-add-route.sh`) and firewall (`02-firewall-setup.sh`) are driven by the values generated from `integration-setup.yaml` (`consumer.networking.internalNetwork`, `internalGateway`, `consumer.bastion.*`)

Direct Topology (Public Network)

```
Main Cluster (172.16.0.0/16)
    ├── Masters
    │   └── Route: 192.168.100.0/24 via 172.16.1.100
    └── Workers (s390x)
        └── Route: 192.168.100.0/24 via 172.16.1.100
        ↓
    Gateway VM
    (172.16.1.100 external)
    (192.168.100.1 internal)
        ↓
    Worker VMs
    (192.168.100.10+)
```

Configuration: - Direct connectivity to gateway VM - Static routes on master nodes - Static routes on s390x worker nodes - No bastion routing needed - Selected with `consumer.networking.topology: direct` (the default); the route MachineConfigs are generated by `generate-setup` from `provider.gateway.ip` and `provider.networking.internalSubnet`, and are created by the `hci-consumer` operator if absent

3.3.6 DNS Configuration

DNS Resolution Flow

Workers receive the main cluster DNS server directly through DHCP (option 6). Their queries are routed through the gateway VM like any other traffic to the main cluster network:

```
Worker VM
    ↓ Query: api-int.cluster.example.com  (DNS server from DHCP)
Gateway VM (routes the packet)
    ↓
Main Cluster DNS (consumer.networking.dnsServer)
    ↓ Response, same path back
Worker VM
```

Gateway VM DNS Configuration

The gateway's own `dnsmasq` also forwards to the main cluster DNS:

```
# /etc/dnsmasq.d/pxe.conf (generated)
no-resolv
server=172.16.0.1          # consumer.networking.dnsServer
domain=cluster.example.com  # consumer.domain
```

Worker VM DNS Configuration

Worker VMs have no cloud-init. DNS server, search domain and hostname come from the gateway's DHCP reservation:

```
dhcp-option=6,172.16.0.1                  # DNS server
domain=cluster.example.com                # DHCP option 15, search domain
dhcp-option=tag:host1,12,worker-x86-1.cluster.example.com   # hostname
```

3.3.7 DHCP and Network Boot

DHCP Configuration

The gateway VM provides DHCP for worker VMs in static-only mode: there is no dynamic address pool, only one reservation per worker slot. See IP Assignment Methods for the generated configuration.

Network Boot Configuration

Worker VMs boot via iPXE. `dnsmasq` serves the first-stage loader over TFTP and points iPXE to an HTTP boot script, `nginx` serves everything else:

```
# /etc/dnsmasq.d/pxe.conf (generated, excerpt)
enable-tftp
tftp-root=/var/lib/tftpboot
tftp-secure

# first request (native PXE firmware): iPXE loader over TFTP
# second request (iPXE announces itself with DHCP option 175): HTTP boot script
dhcp-match=set:ipxe,175
dhcp-boot=tag:!ipxe,undionly.kpxe
dhcp-boot=tag:ipxe,http://192.168.100.1/boot.ipxe
```

```
/var/lib/tftpboot/undionly.kpxe          iPXE loader (TFTP)
/var/www/html/boot.ipxe                  boot script (HTTP, port 80)
/var/www/html/rhcos/rhcos-live-*         RHCOS kernel, initramfs, rootfs (HTTP)
/var/www/html/rhcos/worker-hci.ign       ignition stub (HTTP)
```

The RHCOS files are downloaded by the gateway from the OpenShift mirror for the stream requested by the operator.

Boot Process

1. Worker VM powers on and requests DHCP
2. Gateway VM answers with the reserved IP, hostname and `undionly.kpxe`
3. The firmware loads `undionly.kpxe` via TFTP and starts iPXE
4. iPXE requests DHCP again and is pointed to `http://192.168.100.1/boot.ipxe`
5. `boot.ipxe` shows a 10-second menu; with an empty disk it installs RHCOS
6. iPXE loads the RHCOS live kernel and initramfs over HTTP
7. The installer fetches `worker-hci.ign` and writes RHCOS to disk
8. The worker reboots from disk and joins the cluster

The detailed sequence is in VM Provisioning.

3.3.8 Network Security

Firewall Rules

Gateway VM firewall configuration (set by the cloud-init data, excerpt):

```
# Interfaces and zones
firewall-cmd --permanent --zone=external --add-interface=eth1
firewall-cmd --permanent --zone=internal --add-interface=eth2

# Forwarding between the internal and external side
firewall-cmd --permanent --direct --add-rule ipv4 filter FORWARD 0 -i eth2 -o eth1 -j ACCEPT
firewall-cmd --permanent --direct --add-rule ipv4 filter FORWARD 0 -i eth1 -o eth2 -j ACCEPT

# Services on the internal side: network boot and ignition
firewall-cmd --permanent --zone=internal --add-service=dhcp
firewall-cmd --permanent --zone=internal --add-service=tftp
firewall-cmd --permanent --zone=internal --add-service=http

# SSH (management)
firewall-cmd --permanent --zone=external --add-service=ssh
firewall-cmd --permanent --zone=internal --add-service=ssh
```

The NAT rules shown under NAT Configuration are added the same way.

Network Policies

> **Optional hardening, not created by the operators**
>
> The operators do not create any `NetworkPolicy`. The example below only shows what an administrator could add on top; adapt the labels (the gateway VM is named `hci-gateway`) before using it.

Standard Kubernetes network policies (enforced by OVN-Kubernetes) can restrict traffic:

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-gateway-only
spec:
  podSelector:
    matchLabels:
      app: worker-vm
  policyTypes:
  - Ingress
  - Egress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          app: gateway-vm
  egress:
  - to:
    - podSelector:
        matchLabels:
          app: gateway-vm
```

3.3.9 MTU Considerations

MTU Sizing

```
Physical Network: 1500 (standard Ethernet)
    ↓
Localnet (external): 1500 (set in cluster-network-setup.yaml, no encapsulation)
    ↓
Layer2 (internal): OVN-Kubernetes default (no MTU set by the provisioner)
```

Why the Internal Network May Need a Lower MTU

- OVN adds encapsulation headers (Geneve) on the internal Layer2 network
- Typical overhead: 50-100 bytes
- The operators neither set an MTU on the internal network nor send an MTU to the workers via DHCP

MTU Configuration

The external network MTU is set once in `hci-cluster/cluster-network-setup.yaml`:

```yaml
localnet:
  mtu: 1500
```

If your environment requires a specific MTU on the internal network, verify the effective value on the `UserDefinedNetwork` and inside a worker VM after the first worker is up.

3.3.10 Performance Optimization

Network Throughput

Gateway VM as Bottleneck: - VM network throughput: ~10 Gbps typical - Physical network: 25-50 Gbps available - Gateway VM is limiting factor

Mitigation: - Increase gateway VM vCPU allocation - Use multiple gateway VMs for HA/load balancing - Enable virtio network drivers - Consider SR-IOV for high-performance scenarios

Latency Optimization

Typical Latencies: - Physical network: <0.1ms - OVN overlay: +0.2-0.5ms - Gateway VM routing: +0.5-1ms - Total: ~1-2ms additional latency

Optimization: - Place gateway VM on low-latency nodes - Use CPU pinning for gateway VM - Enable NUMA awareness - Minimize network hops

3.3.11 High Availability

> ⚠ **Not implemented**
>
> The operators create exactly one gateway VM per integration (`hci-gateway`). The multi-gateway layout below is a design option; Gateway VM HA is listed under Future Enhancements in the Architecture Overview.

Multiple Gateway VMs

A redundant design would use multiple gateway VMs:

```
# Gateway VM 1
external: 172.16.1.100
internal: 192.168.100.1

# Gateway VM 2
external: 172.16.1.101
internal: 192.168.100.2
```

Load Balancing Options: 1. Round-robin DNS 2. VRRP (keepalived) 3. BGP routing 4. External load balancer

Failover Scenarios

Gateway VM Failure: 1. Detect failure (health check) 2. Promote backup gateway 3. Update routing tables 4. Workers reconnect automatically

Network Failure: 1. OVN detects link down 2. Reroutes traffic if alternate path exists 3. Alerts monitoring system

3.3.12 Monitoring

Key Metrics

Gateway VM: - Network throughput (in/out) - Packet loss rate - NAT connection table size - CPU utilization - Memory utilization

Network Health: - Latency between networks - Packet loss percentage - MTU path discovery results - DNS resolution time

Monitoring Tools

```
# Network throughput
iftop -i eth1

# Packet loss
ping -c 100 192.168.100.10 | grep loss

# NAT connections
conntrack -L | wc -l

# DNS performance
dig @192.168.100.1 api.cluster.example.com +stats
```

3.3.13 Troubleshooting

Common Issues

Workers can't reach main cluster: - Check gateway VM routing table - Verify NAT rules (should NOT NAT cluster traffic) - Test connectivity from gateway VM - Check firewall rules

DNS resolution fails: - Verify gateway VM DNS forwarder configuration - Check main cluster DNS server accessibility - Test DNS from gateway VM - Verify worker VM DNS configuration

Network boot fails: - Check `dnsmasq` (DHCP and TFTP) and `nginx` (HTTP) on the gateway VM - Check that the worker's MAC address has a `dhcp-host` reservation - Check `undionly.kpxe` in the TFTP root and `boot.ipxe` in the HTTP root - Check that the RHCOS boot files and `current-version.txt` exist under `/var/www/html/rhcos/` - Verify network connectivity during boot

See Troubleshooting Guide for detailed solutions.

3.3.14 Best Practices

Network Design

1. **Use static IPs for gateway VM** - Ensures consistent routing
2. **Document IP allocations** - Maintain IP address inventory
3. **Plan for growth** - Reserve IP ranges for future workers
4. **Monitor network performance** - Establish baselines

Security

1. **Minimize exposed services** - Only open required ports
2. **Use network policies** - Restrict traffic where possible
3. **Regular security audits** - Review firewall rules
4. **Encrypt sensitive traffic** - Use TLS for management

Operations

1. **Automate configuration** - Let the operators generate the gateway configuration from `integration-setup.yaml`
2. **Test failover scenarios** - Verify HA works
3. **Monitor continuously** - Set up alerts
4. **Document changes** - Keep network diagrams updated

3.3.15 References

- Architecture Overview
- Deployment Variants
- Troubleshooting Guide
- OVN-Kubernetes Documentation

3.4 VM Provisioning Architecture

This document describes how worker VMs are provisioned, configured, and integrated into the main OpenShift cluster.

All names, addresses and sizes are the example values from `templates/integration-setup.example.yaml`. The real values are set once in `integration-setup.yaml`; see the field mapping in Architecture Overview.

The resources and commands shown below describe what the operators create and what you can inspect when troubleshooting. They are not manual deployment steps: the `hci-provisioner` bootstraps the gateway VM and the ignition file, and the `hci-consumer` creates the worker VMs, once `integration-setup.yaml` has been turned into a `WorkerPool` by `generate-setup`.

3.4.1 Overview

The VM provisioning process involves:

1. **VM Creation** - OpenShift Virtualization creates VMs on HCI cluster
2. **Network Boot** - VMs boot via iPXE from the gateway VM
3. **OS Installation** - RHCOS installs from ignition configuration
4. **Cluster Join** - Workers register with main cluster
5. **Configuration** - MachineConfig applies worker-specific settings

3.4.2 Provisioning Components

1. OpenShift Virtualization (KubeVirt)

Purpose: Provides VM infrastructure on HCI cluster

Key Resources: - `VirtualMachine` - VM definition and lifecycle - `VirtualMachineInstance` - Running VM instance - `DataVolume` - VM disk storage - `UserDefinedNetwork` - Network connectivity (`router-internal-network`)

VM definition created by the `hci-consumer` (abridged):

```yaml
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: worker-x86-1
  namespace: main-cluster-workers
spec:
  running: true
  template:
    metadata:
      annotations:
        k8s.v1.cni.cncf.io/networks: '[{"name":"router-internal-network","interface":"internal"}]'
    spec:
      domain:
        cpu:
          cores: 4
        resources:
          requests:
            memory: 8Gi
        devices:
          disks:
          - name: rootdisk
            disk:
              bus: virtio
            bootOrder: 2
          interfaces:
          - name: default            # pod network, not used for cluster traffic
            masquerade: {}
            bootOrder: 3
          - name: internal           # worker network, boots first
            bridge: {}
            bootOrder: 1
            macAddress: "52:54:01:00:00:10"
      networks:
      - name: default
        pod: {}
      - name: internal
        multus:
          networkName: router-internal-network
      volumes:
```

```
      - name: rootdisk
        dataVolume:
          name: worker-x86-1-rootdisk
```

2. Gateway VM

Purpose: Provides provisioning services

Services: - **DHCP** - IP address assignment - **TFTP** - Boot file serving - **HTTP** - Ignition config serving - **DNS** - Name resolution

Configuration: created by the `hci-provisioner` (GatewayVM step) from a cloud-init template filled with the `provider.gateway.*`, `provider.networking.*` and `consumer.networking.*` values of `integration-setup.yaml`. `dnsmasq` provides DHCP, TFTP and DNS; `nginx` serves the ignition file. Nothing is installed or enabled by hand.

3. Ignition Configuration

Purpose: Configures RHCOS during first boot

Source: Generated by the `hci-provisioner` (Ignition step) from the `worker-user-data-managed` Secret in `openshift-machine-api` on the main cluster. The result is a small stub that points the worker at the Machine Config Server of the configured MachineConfigPool (`consumer.machineConfigPool`, example `hci-vm`), and is copied to the gateway VM's HTTP server. It does not depend on the OpenShift version and is created once at bootstrap.

The worker hostname is not part of the ignition file. It is handed out by the gateway's DHCP server per worker slot (`<namePrefix>-<index>.<domain>`).

3.4.3 Provisioning Flow

Phase 1: VM Creation

```
WorkerPool CR (hci-consumer operator)
        ↓
hci-consumer: observe/diff/act reconcile
        ↓
Create VirtualMachine Resource on HCI
        ↓
Allocate Storage (DataVolume)
        ↓
Attach Networks (NADs from WorkerPoolProvisioning)
        ↓
Start VM (VirtualMachineInstance)
```

WorkerPool CR spec (excerpt):

```
spec:
  consumer:
    workers:
      count: 3
      namePrefix: worker-x86
  provider:
    namespace: main-cluster-workers
    maxPoolSize: 10
```

Phase 2: Network Boot (iPXE)

The gateway VM runs `dnsmasq` (DHCP and TFTP) and `nginx` (HTTP). The worker VM boots from its internal NIC first (`bootOrder: 1`), so the sequence below starts as soon as it is powered on.

```
VM Powers On
        ↓
PXE firmware requests DHCP on the internal network
        ↓
Gateway VM answers from its static reservation for this MAC
  - IP:        192.168.100.10   (fixed per worker slot)
  - Hostname:  worker-x86-1.cluster.example.com
  - Gateway:   192.168.100.1
  - DNS:       172.16.0.1       (consumer.networking.dnsServer)
  - Boot file: undionly.kpxe    (served via TFTP)
        ↓
Firmware downloads undionly.kpxe via TFTP and starts iPXE
        ↓
```

```
iPXE requests DHCP again (it announces itself with DHCP option 175)
        ↓
Gateway VM answers with an HTTP boot URL instead
  - http://192.168.100.1/boot.ipxe
        ↓
iPXE runs boot.ipxe: 10-second menu, default "Boot from local disk"
  - disk empty or not bootable → falls through to "Install RHCOS"
        ↓
iPXE loads the RHCOS live kernel and initramfs over HTTP
```

DHCP: the gateway does not hand out addresses from a dynamic range. It runs `dnsmasq` in static-only mode (`dhcp-range=192.168.100.0,static`) with one `dhcp-host` entry for every possible worker slot, up to `provider.maxPoolSize`. Each entry fixes MAC address, IP address and hostname (see MAC Address Assignment). Scaling workers up or down within the pool limit therefore never requires a gateway change.

Boot parameters (from `boot.ipxe`):

```
kernel http://192.168.100.1/rhcos/rhcos-live-kernel \
  ip=enp2s0:dhcp ip=enp1s0:off rd.neednet=1 \
  coreos.inst.install_dev=/dev/vda \
  coreos.live.rootfs_url=http://192.168.100.1/rhcos/rhcos-live-rootfs.img \
  coreos.inst.ignition_url=http://192.168.100.1/rhcos/worker-hci.ign \
  ignition.platform.id=metal \
  coreos.inst.node-role.kubernetes.io/hci-vm= \
  coreos.inst.karg=rd.multipath=0 ...
initrd http://192.168.100.1/rhcos/rhcos-live-initramfs.img
```

`enp2s0` is the internal NIC (DHCP), `enp1s0` the pod-network NIC (left unconfigured).

Boot files: the kernel, initramfs and rootfs under `/rhcos/` are downloaded by the gateway itself from the OpenShift mirror (`rhcos-sync.timer`, every 5 minutes), for the RHCOS stream requested by the operator. Scale-up is held back until the download is complete.

Phase 3: OS Installation

```
RHCOS live image boots (kernel + initramfs + rootfs over HTTP)
        ↓
Fetch Ignition Config
  URL: http://192.168.100.1/rhcos/worker-hci.ign
        ↓
coreos-installer writes RHCOS to /dev/vda
        ↓
Reboot from disk
        ↓
Ignition stub on first boot contacts the Machine Config Server
  - https://api-int.cluster.example.com:22623/config/hci-vm
  - outbound traffic is routed (and NATed where needed) by the gateway VM
```

The ignition file is the stub described in Ignition Configuration; the actual machine configuration is pulled from the main cluster's Machine Config Server for the `hci-vm` pool.

Installation Progress:

```
# Monitor from VM console
journalctl -u coreos-installer -f
```

Phase 4: First Boot

```
VM Boots from Disk
        ↓
Ignition stub runs (first boot)
  - contacts the Machine Config Server of the main cluster
  - https://api-int.cluster.example.com:22623/config/hci-vm
  - merges the rendered configuration of the hci-vm pool
        ↓
Start kubelet (with the bootstrap credentials from that configuration)
        ↓
Kubelet submits a bootstrap CSR (client certificate) to the main cluster
```

Boot Process:

```
# Key services
systemctl status kubelet
systemctl status crio
```

Phase 5: Cluster Join

Two CSRs are involved. Both are approved automatically by the `csr-approver` sidecar of the `hci-consumer-operator` pod, and only for node names that match the worker name prefix (`consumer.workers.namePrefix`).

```
Bootstrap CSR (client certificate)
  - approved by csr-approver while the node object does not exist yet
        ↓
Node object registers in the main cluster
        ↓
Kubelet submits a serving CSR (server certificate)
  - approved by csr-approver once the node object exists
        ↓
Node Labeled by node-labeler
  - node-role.kubernetes.io/hci-vm: ""
  - only if: arch amd64, InternalIP in the internal subnet, name matches prefix
        ↓
Node matches the MachineConfigPool nodeSelector (pool: hci-vm)
        ↓
machine-config-daemon applies the pool's MachineConfigs
        ↓
Node Ready
```

Manual fallback if a CSR stays pending: `oc adm certificate approve <csr>`.

Verification:

```
# Check CSRs
oc get csr | grep Pending

# Check nodes
oc get nodes -l node-role.kubernetes.io/hci-vm

# Check MachineConfigPool
oc get mcp hci-vm
```

3.4.4 Storage Provisioning

DataVolume Creation

Purpose: Provides persistent storage for VM

Configuration:

```yaml
apiVersion: cdi.kubevirt.io/v1beta1
kind: DataVolume
metadata:
  name: worker-x86-1-rootdisk
spec:
  pvc:
    accessModes:
    - ReadWriteOnce
    resources:
      requests:
        storage: 50Gi            # consumer.workers.resources.storage
    storageClassName: ocs-storagecluster-ceph-rbd-virtualization
    volumeMode: Block
  source:
    blank: {}
```

The disk is created blank; RHCOS is written to it by the installer during Phase 3.

Storage class: `ocs-storagecluster-ceph-rbd-virtualization` (Ceph RBD block storage optimised for virtualization) is used for all worker disks.

Disk Layout

The installer writes the standard RHCOS layout to the virtual disk (`/dev/vda`, 50Gi by default). The operators do not define a custom partitioning.

Storage Performance

Considerations: - **IOPS** - Ceph RBD provides good random I/O - **Throughput** - Network-attached storage limits - **Latency** - Typically 1-5ms for Ceph - **Capacity** - Plan for OS + container images + logs

3.4.5 Network Provisioning

Network Attachment

Worker VMs attach to the `router-internal-network` `UserDefinedNetwork` that the `hci-provisioner` creates in `provider.namespace` (see Network Design). OVN-Kubernetes generates the matching `NetworkAttachmentDefinition` from it; none is written by hand. Each VM references it in a Multus annotation and a `multus` network.

IP Configuration: - **DHCP** - From the gateway VM, using a static per-slot reservation (MAC → IP → hostname)

MAC Address Assignment

Fixed MAC Addresses:

MACs are derived as `<macPrefix>:00:00:<hex>` with `hex = 0x10 + (index - 1)`. `macPrefix` is `provider.networking.macPrefix` from `integration-setup.yaml` (example: `52:54:01`), so `worker-x86-1` gets:

```
interfaces:
- name: internal
  macAddress: "52:54:01:00:00:10"
  bridge: {}
```

Why Fixed MACs? - The gateway's static DHCP reservations are keyed on the MAC - Predictable IP address and hostname per worker slot - Easier troubleshooting

3.4.6 Configuration Management

MachineConfig Application

Timeline (starts when the node has been labeled, see Platform Labeling):

```
Node Labeled (t=0)
        ↓
Node matches the MachineConfigPool nodeSelector (t=0)
        ↓
machine-config-daemon takes over the node (t=0)
        ↓
Check rendered MachineConfig of the pool (t=0-30s)
        ↓
Apply Configuration (t=30s-2m)
  - Kernel arguments
  - System files
  - Systemd units
        ↓
Reboot if Required (t=2m)
        ↓
Verify Configuration (t=3m)
        ↓
Node Ready (t=3-5m)
```

MachineConfigPool

Purpose: Group workers with common configuration

The pool name comes from `consumer.machineConfigPool` (default and example: `hci-vm`). If the pool and its kernel-argument MachineConfig (`99-hci-vm-kargs`) do not exist, the `hci-consumer` creates them; existing ones are never modified.

```
apiVersion: machineconfiguration.openshift.io/v1
kind: MachineConfigPool
metadata:
  name: hci-vm
spec:
  machineConfigSelector:
    matchExpressions:
    - key: machineconfiguration.openshift.io/role
      operator: In
      values:
      - worker
      - hci-vm
    - key: node-platform
      operator: NotIn
      values:
      - s390x
  nodeSelector:
```

```
    matchLabels:
      node-role.kubernetes.io/hci-vm: ""
  paused: false
  maxUnavailable: 1
```

Key Features: - Separate from main worker pool - Can pause updates independently - Allows HCI-specific configuration

Platform Labeling

Automatic Labeling: the `node-labeler` sidecar of the `hci-consumer` operator adds `node-role.kubernetes.io/hci-vm: ""` to a new node only if all of these hold:

- the node's architecture is `amd64`
- its InternalIP is within the internal worker subnet (`provider.networking.internalSubnet`)
- its name matches the worker name pattern (`consumer.workers.namePrefix`)

The sidecar reads these values from the `hci-node-manager-config` ConfigMap, which the operator writes from the `WorkerPool`. Nodes of other architectures are never labeled, so the s390x nodes cannot be moved into the `hci-vm` pool by mistake.

Why Automatic? - Ensures correct pool assignment - Prevents configuration conflicts - Simplifies operations

3.4.7 Automation

HCI Node Manager (sidecars)

Purpose: Automate node lifecycle tasks

Functions: 1. **CSR Auto-Approval** (`csr-approver`) — approves worker CSRs automatically via a Kubernetes watch stream 2. **Node Labeling** (`node-labeler`) — applies `node-role.kubernetes.io/hci-vm: ""` so the node joins the correct MachineConfigPool

Deployment: The node manager runs as two sidecar containers inside the `hci-consumer-operator` Deployment. No standalone Deployment is required. Both sidecars read their config from the `hci-node-manager-config` ConfigMap, which the operator writes on every `WorkerPool` reconcile.

```
# Check sidecars
oc get pods -n hci-node-manager -l app.kubernetes.io/name=hci-consumer-operator

# Logs
oc logs -n hci-node-manager deploy/hci-consumer-operator -c csr-approver | tail -20
oc logs -n hci-node-manager deploy/hci-consumer-operator -c node-labeler | tail -20
```

Operator Automation

Purpose: Declarative, self-healing VM lifecycle management

Features: - Observe/diff/act reconcile loop — continuously converges to desired state - Safe scale-down: cordon + drain gate before VM deletion - Status conditions and phase sub-states on the `WorkerPool` CR - Bootstrap (gateway VM, UDN, ignition) managed by `hci-provisioner`

Usage: `generate-setup` creates the initial `WorkerPool` with `workers.count: 0`. Scale by editing the live resource in the operator namespace (`consumer.operatorNamespace`, example `hci-node-manager`):

```
# Scale up
oc -n hci-node-manager patch workerpool hci-workers \
  --type merge -p '{"spec":{"consumer":{"workers":{"count":4}}}}'

# Scale down
oc -n hci-node-manager patch workerpool hci-workers \
  --type merge -p '{"spec":{"consumer":{"workers":{"count":2}}}}'

# Watch status
oc -n hci-node-manager get workerpool hci-workers -w
```

Re-running `generate-setup` rewrites the `WorkerPool` file with `workers.count: 0`; use `oc get workerpool hci-workers -o yaml` to keep the live state.

3.4.8 Monitoring Provisioning

Key Metrics

VM Creation: - Time to create VM: Target <2 minutes - Storage allocation time: Target <1 minute - Network attachment time: Target <30 seconds

Network Boot: - DHCP response time: Target <1 second - TFTP download of `undionly.kpxe`: Target <30 seconds - iPXE boot menu: 10 seconds, then installs automatically if the disk is empty

OS Installation: - Ignition fetch time: Target <5 seconds - RHCOS download time: Target 5-10 minutes - Installation time: Target 10-15 minutes - Total time: Target 15-20 minutes

Cluster Join: - CSR generation: Target <1 minute - CSR approval: Target <1 minute (auto) or manual - Node registration: Target <2 minutes - Node ready: Target <5 minutes

Monitoring Commands

```
# Watch VM creation
watch oc get vm -n main-cluster-workers

# Monitor network boot (from VM console)
virtctl console -n main-cluster-workers worker-x86-1

# Check installation progress
oc logs -n main-cluster-workers virt-launcher-worker-x86-1-xxxxx

# Monitor cluster join
watch oc get csr
watch oc get nodes -l node-role.kubernetes.io/hci-vm

# Check MachineConfigPool
watch oc get mcp hci-vm
```

3.4.9 Troubleshooting

VM Won't Start

Symptoms: - VM stuck in "Starting" state - VirtualMachineInstance not created

Checks:

```
# Check VM events
oc describe vm worker-x86-1 -n main-cluster-workers

# Check DataVolume
oc get dv -n main-cluster-workers

# Check storage capacity
oc get pvc -n main-cluster-workers
```

Network Boot Fails

Symptoms: - VM shows "No bootable device" - DHCP request times out - iPXE starts but cannot load `boot.ipxe` or the RHCOS kernel

Checks:

```
# Check gateway VM (named hci-gateway)
oc get vmi hci-gateway -n main-cluster-workers

# Log in to the gateway (external IP = provider.gateway.ip)
ssh core@172.16.1.100

# On the gateway: DHCP/TFTP and HTTP services
sudo systemctl status dnsmasq nginx

# On the gateway: is the worker's MAC being offered an address?
sudo journalctl -u dnsmasq | grep -i dhcp | tail -20

# On the gateway: iPXE loader, boot script and RHCOS boot files
ls -la /var/lib/tftpboot/
```

```
ls -la /var/www/html/ /var/www/html/rhcos/
cat /var/www/html/rhcos/current-version.txt
```

If `current-version.txt` is missing, the gateway has not finished downloading the RHCOS boot files yet (`systemctl status rhcos-sync.timer` , `journalctl -u rhcos-sync`).

Installation Fails

Symptoms: - Installation hangs - Cannot fetch ignition - Disk errors

Checks:

```
# Check ignition availability (from the gateway or any host on the internal network)
curl -I http://192.168.100.1/rhcos/worker-hci.ign

# Check disk
virtctl console -n main-cluster-workers worker-x86-1
# In console: lsblk, fdisk -l

# Check logs
journalctl -u coreos-installer
```

Node Won't Join

Symptoms: - CSR not generated - CSR pending forever - Node not appearing

Checks:

```
# Check CSRs
oc get csr | grep worker-x86

# Check kubelet
oc debug node/worker-x86-1
chroot /host
systemctl status kubelet
journalctl -u kubelet -n 100

# Check connectivity
ping api.cluster.example.com
curl -k https://api.cluster.example.com:6443/version
```

See Troubleshooting Guide for detailed solutions.

3.4.10 Best Practices

VM Configuration

1. **Use fixed MAC addresses** - Ensures consistent networking
2. **Allocate sufficient resources** - defaults are 4 vCPU, 8GB RAM, 50GB disk (`consumer.workers.resources`); size to your workload
3. **Use block storage** - Better performance than file storage
4. **Enable virtio drivers** - Optimal performance

Provisioning

1. **Test with one worker first** - Verify process works
2. **CSR approval is automatic** — the `csr-approver` sidecar in the operator handles it
3. **Monitor provisioning** - Watch for failures
4. **Document custom configurations** - Keep ignition changes tracked

Operations

1. **Use operator CRs for management** - Declarative and version-controlled
2. **Keep gateway VM updated** - Security patches

3. **Backup configurations** - CR manifests, ignition configs
4. **Plan for failures** - Have recovery procedures ready

3.4.11 References

- Architecture Overview
- Network Design
- Getting Started Guide
- Troubleshooting Guide
- OpenShift Virtualization Documentation
- Ignition Documentation

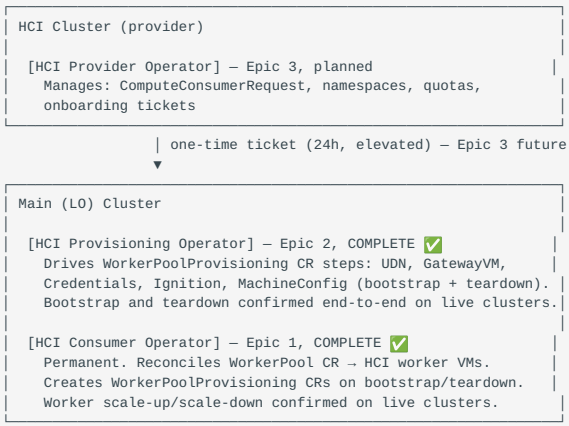
3.5 Provider/Consumer Model for HCI Integration

3.5.1 Overview

This document describes the provider/consumer architecture for the Fusion HCI integration, inspired by the ODF (OpenShift Data Foundation) Storage Provider/Client model. The goal is secure, operator-driven lifecycle management for multi-architecture worker nodes: x86 VMs running on a Fusion HCI cluster join an s390x (LinuxONE) OpenShift cluster as worker nodes.

3.5.2 Three-Actor Model

The architecture separates concerns across three distinct actors, each with a strictly limited privilege scope:



Key privilege-separation principle: the steady-state operator (which runs forever) never holds the elevated bootstrap credentials. The ticket exists only for the lifetime of the one-shot Bootstrap Job.

3.5.3 Implemented: LO Steady-State Operator

WorkerPool CRD

The `WorkerPool` custom resource is the single control surface for the steady-state operator. It is installed on the main (LO) cluster.

API group: `hci-integration.openshift.io/v1alpha1` **Short name**: `wp` **Namespace-scoped**

```yaml
apiVersion: hci-integration.openshift.io/v1alpha1
kind: WorkerPool
metadata:
  name: hci-workers
  namespace: hci-node-manager
spec:
  provider:
    apiEndpoint: "https://api.hci.example.com:6443"
    namespace: "main-cluster-workers"
    credentialsSecret: hci-provisioner-credentials
    maxPoolSize: 10
    gateway:
      ip: "172.16.1.100"
      prefixLength: 16
      defaultRouter: "172.16.0.1"
    networking:
      macPrefix: "52:54:01"
      internalSubnet: "192.168.100.0/24"

  consumer:
    networking:
      topology: direct    # or: mediated
      dnsServer: "172.16.0.1"
    workers:
      namePrefix: worker-x86
      count: 3          # desired number of HCI worker VMs / LO nodes
    machineConfigPool: hci-vm

status:
  phase: Ready          # Provisioning | Ready | Deprovisioning | Failed
  conditions:
```

```
    - type: ProviderCredentialsReady
      status: "True"
      reason: SecretFound
      message: "Provider credentials Secret found."
    - type: ScaleDownBlocked
      status: "False"
      reason: NoPendingDrain
      message: "No scale-down blocked."
    - type: CountExceedsMaxPoolSize
      status: "False"
      reason: CountWithinBounds
      message: "workers.count is within maxPoolSize."
```

Reconciliation Logic

The operator follows a strict **observe → diff → act** pattern:

1. **Observe** (`observe_world.yml`): reads the credentials Secret, lists all operator-owned VMs on HCI (by label), lists all matching nodes on the LO cluster, determines which nodes are Ready (have kubelet running). Zero decisions.
2. **Diff** (`diff_desired_actual.yml`): pure set arithmetic.
3. Computes `vms_to_create` = indices in `1..count` not covered by an existing VM
4. Computes `vms_to_delete` = existing VMs whose node is NotReady, capped at `surplus`
5. Applies `maxPoolSize` clamping with a `CountExceedsMaxPoolSize` condition
6. **Act** (`scale_up.yml`, `scale_down.yml`):
7. Scale-up: create DataVolume + VirtualMachine on HCI for each missing name
8. Scale-down: delete node object (LO) → delete VM (HCI) → delete DataVolume (HCI)

Scale-down safety gate: a VM is only eligible for deletion when its corresponding node's Ready condition is not `True` (kubelet has stopped). LOOps stops the VM it wants to remove; the operator observes the NotReady state and acts. No operator-side drain. The `ScaleDownBlocked` condition is set when `workers.count` is reduced but no NotReady node is yet available.

Complementary Component: hci-node-manager

The node manager runs as sidecar containers (`csr-approver`, `node-labeler`) inside the `hci-consumer-operator` Deployment. Responsibilities are strictly separated:

Component	Responsibility
hci-consumer operator	VM lifecycle: create, delete DataVolume + VM on HCI
hci-node-manager sidecars	Node join: CSR approval + node labeling (stream-watch, ms reaction time)

`hci-node-manager` owns the time-critical node-join path. The `hci-vm` role label must arrive before MCO renders ignition for the node — a polling-based operator cannot meet this window. The two components have non-overlapping responsibilities.

RBAC

The operator ServiceAccount (`config/rbac/role.yaml`) has minimal permissions on the main cluster only:

- **Nodes**: `get`, `list`, `watch`, `patch`, `update` (for labeling and deletion)
- **CertificateSigningRequests**: `get`, `list`, `watch`, `update` (for CSR approval, delegated to hci-node-manager)
- **MachineConfigPools**: `get`, `list`, `watch`
- **Pods**: `get`, `list` (for eviction check, future LO-09/LO-10)

HCI cluster access is via the narrow kubeconfig Secret (namespace-scoped to the consumer namespace on HCI, VirtualMachine/DataVolume/PVC write + Secret/ConfigMap read only).

3.5.4 Implemented: HCI Provisioning Operator (Epic 2)

A dedicated operator managing the full infrastructure lifecycle via the `WorkerPoolProvisioning` CRD. Triggered by the `WorkerPool` operator when bootstrap or teardown is needed. See `provisioning-model.md` for the complete design.

Key design principle: the credential scope is the discriminant between the two operators. The steady-state operator holds only a narrow SA token scoped to one HCI namespace. The provisioning operator holds broader credentials: `MachineConfig` write on LO, SSH key access, topology awareness. These never cross the operator boundary.

`WorkerPool` **phase state machine**: - `Provisioning` — `WorkerPoolProvisioning` (mode=bootstrap) in progress; worker reconcile skipped - `Ready` — all bootstrap steps complete; worker reconcile runs normally - `Deprovisioning` — `WorkerPoolProvisioning` (mode=teardown) in progress; worker reconcile skipped

Step execution model: each bootstrap/teardown step is a Kubernetes Job owned by the `WorkerPoolProvisioning` CR (via `ownerReference`). The provisioning operator watches Job termination status and owns all `status.steps` transitions. Jobs are garbage-collected automatically when the `WorkerPoolProvisioning` CR is deleted.

Credentials handover: BS-05 (automated SA/Secret creation) was descoped; credentials are created with a script. The Secret is referenced by name in `WorkerPool.spec.provider.credentialsSecret`.

3.5.5 Planned: HCI Provider Operator (Epic 3)

Runs on the HCI cluster. Automates what the HCI admin currently does manually.

Responsibilities (not yet implemented): 1. Watch `ComputeConsumerRequest` CRDs — process consumer onboarding requests 2. Provision consumer namespace, CUDN label, ResourceQuota on HCI 3. Generate onboarding ticket: privileged bootstrap ServiceAccount + time-limited token 4. Surface ticket in a Secret for the HCI admin to hand to the LO team 5. Monitor quota usage per consumer

---

3.5.6 Deferred: Negotiation Flow (Epic 4)

LO-initiated cross-cluster capacity requests. Design open — chicken-and-egg discovery problem (how does LO know the HCI API endpoint before any credentials exist?). Deferred to iteration 2. See the project operator backlog for NEG-01/02/03 story descriptions.

3.5.7 Credentials Deployment

The one-time cross-domain credentials handover is done by script, run from the repo root with `setup_env.sh` sourced. See Getting Started (steps B.2 and C.8) for the authoritative procedure.

```
# On the HCI cluster (cluster-admin required once)
./hci-cluster/setup-consumer-rbac.sh
# Script prints the oc create secret command to run on the LO bastion

# Or create the operator Secrets directly (both clusters logged in)
./operator/setup-provider-credentials.sh
```

These create, in the consumer namespace on HCI: - ServiceAccount(s), a namespace-scoped Role (VM, DataVolume, PVC write; Secret/ConfigMap read) and RoleBinding - A 1-year ServiceAccount token, written as a kubeconfig Secret on the LO cluster

The Secret name is referenced by `WorkerPool.spec.provider.credentialsSecret` (CRD default: `hci-provisioner-credentials`).

> ⚠ **Open inconsistency**
>
> The two scripts and the operator defaults do not agree on the Secret layout: `setup-consumer-rbac.sh` prints `hci-provider-credentials`; `setup-provider-credentials.sh` writes two Secrets (`hci-provider-credentials` and `hci-provisioner-credentials`); the CRD and role defaults use `hci-provisioner-credentials` only. To be resolved as part of the setup rework.

3.5.8 Security Model

Credential Lifecycle

Credential	Lifetime	Scope	Holder
Onboarding ticket	24h (planned)	Create ComputeConsumerBinding on HCI	Bootstrap Job only
Narrow SA token	1 year (manual) / 10 years (planned)	Consumer namespace on HCI: VM/ DV/PVC write	Steady-state operator
Cluster admin	Permanent	HCI admin workstation	HCI admin only — never in-cluster

What the Steady-State Operator Can and Cannot Do on HCI

The operator's kubeconfig is scoped to one namespace (e.g. `main-cluster-workers`) and can: - Create / delete `VirtualMachine` and `VirtualMachineInstance` - Create / delete `DataVolume` and `PersistentVolumeClaim` - Read `Secret` and `ConfigMap`

It **cannot**: - Access any other namespace on HCI - Create namespaces, RBAC, or network resources - Approve CSRs or label nodes on HCI

3.5.9 Related Documents

- Architecture Overview

Document Version: 2.2 **Last Updated**: 2026-10-12 **Status**: Updated to current CRD names (`WorkerPool`, `WorkerPoolProvisioning`). Epics 1 and 2 complete. Epics 3–4 described as planned.

3.6 HCI Provisioning Model — Lifecycle Operator Design

3.6.1 Overview

This document describes the architecture of the **HCI Provisioning Operator** (Epic 2), which manages the full lifecycle of a consumer relationship: bootstrap (create infrastructure) and teardown (remove infrastructure) triggered by the `WorkerPool` CR.

The steady-state operator (Epic 1) handles only the continuous worker VM reconcile loop. It deliberately knows nothing about how the infrastructure was set up. The provisioning operator is responsible for everything that must exist *before* the steady-state operator can function, and for removing it when the consumer relationship ends.

3.6.2 Separation of Concerns

The credential scope is the clean discriminant between the two operators:

Operator	Credential	Scope	Runs
HCI Consumer Operator (Epic 1)	Narrow SA token on HCI — permanent	HCI: `VirtualMachine` + `DataVolume` CRUD in one namespace. LO (in-cluster RBAC): `Node` patch/delete, CSR approve, `MachineConfigPool` + `MachineConfig` get/list/watch/**create** (create-if-absent, BS-02b); SSH key Secret read (for `desired-version.txt` management)	Continuously
HCI Provisioning Operator (Epic 2)	Broader SA on HCI — permanent for lifecycle ops	HCI: gateway VM + DataVolume, `UserDefinedNetwork`. LO (in-cluster RBAC): `MachineConfig` **read-only** (to read the `99-worker-ssh` key); SSH key Secret create/read	On CR create / delete

The steady-state operator **never** holds: - Topology-aware gateway configuration (direct vs mediated branching in cloud-init) - Knowledge of the gateway VM spec - `UserDefinedNetwork` or gateway VM permissions on the provider

These belong exclusively to the provisioning operator. `MachineConfigPool` / `MachineConfig` creation was moved to the consumer operator in BS-02b, because those are consumer-cluster resources the provisioner has no business owning. Source of truth for permissions: `operator/deploy/base/rbac.yaml`.

Design decision — bootfiles version management (BS-06b): The `hci_consumer` role owns `desired-version.txt` on the gateway and the scale-up gate, **not** the provisioning operator. Rationale: - The consumer runs every reconcile and is the actor that suffers if bootfiles are wrong — it is the natural owner of the "are workers bootable?" question. - The provisioner's responsibility ends when infrastructure is set up. Tracking OCP versions post-bootstrap is outside its scope. - The consumer reads `WorkerPoolProvisioning.status.sshKeySecret` to obtain the SSH key name, then uses the provisioner keypair to write `desired-version.txt` via SSH. The gateway's `rhcos-sync.timer` handles the download autonomously. - Scale-up gate: `GET /rhcos/current-version.txt == ClusterVersion.status.desired.version`. Non-200 or mismatch → defer scale-up to next reconcile (safe default). - There is **no** `Bootfiles` **step in** `WorkerPoolProvisioning` — the provisioner step list covers only one-time infrastructure setup.

3.6.3 The `WorkerPoolProvisioning` CRD

The provisioning operator introduces a new CRD: `WorkerPoolProvisioning`. This resource is created by the `WorkerPool` operator when it detects that provisioning is needed (gateway VM absent, phase `Pending`). It is the coordination board for all bootstrap and teardown steps.

```
apiVersion: hci-integration.openshift.io/v1alpha1
kind: WorkerPoolProvisioning
metadata:
  name: hci-workers-bootstrap
  namespace: hci-node-manager
```

```
  ownerReferences:
    - apiVersion: hci-integration.openshift.io/v1alpha1
      kind: WorkerPool
      name: hci-workers
      # ownerReference ensures GC when WorkerPool is deleted
spec:
  workerPoolRef: hci-workers
  mode: bootstrap         # bootstrap | teardown
  # providerAccess and infrastructure.{gateway,network,capacity} are copied
  # from WorkerPool.spec.provider / spec.consumer at creation time
status:
  phase: InProgress       # Pending | InProgress | Complete | Failed
  steps:
    - name: GatewayVM
      status: InProgress  # Pending | InProgress | Complete | Failed
      message: "Creating gateway VM and DataVolume on HCI."
    - name: UDN
      status: Pending
    - name: Ignition
      status: Pending
    - name: Credentials
      status: Pending
```

The `ownerReference` to `WorkerPool` means Kubernetes garbage-collects the `WorkerPoolProvisioning` CR automatically when the `WorkerPool` is deleted.

3.6.4 Step Execution Model

All steps currently run **directly in the Ansible reconcile loop** (`advance_steps.yml` dispatches to `step_<name>.yml` task files). Each step is idempotent and re-checked on every reconcile until it reaches a terminal state. The provisioning operator is the **sole owner of step status**.

> **Job-based steps: designed, not used**
>
> The original design ran each step as a Kubernetes `Job` owned by the `WorkerPoolProvisioning` CR (with ownerReference garbage collection and exit-code-driven status). `advance_steps.yml` still contains the Job launcher, but its input list is an empty placeholder — no step uses it today. Steps are cheap synchronous API calls (or SSH/TCP probes), so the extra Job indirection was not needed.

Ownership Chain

```
WorkerPool  →owns→  WorkerPoolProvisioning
```

Deleting `WorkerPool` removes `WorkerPoolProvisioning` via Kubernetes garbage collection.

Status Protocol

Actor	Transition	Trigger
Provisioning operator	`Pending → InProgress`	When the step's resources are created (or step re-check starts)
Provisioning operator	`InProgress → Complete`	Step's completion condition confirmed (e.g. TCP reachability for GatewayVM, resource absent for teardown)
Provisioning operator	`InProgress → Failed`	Step error, or timeout exceeded (GatewayVM)
Provisioning operator	`Failed → propagate`	Patches `WorkerPool` phase to `Failed` with message
Provisioning operator	all `Complete → phase Complete`	Patches `WorkerPool` phase to `Ready` (bootstrap) / `Deprovisioned` (teardown)

Step results are collected during the reconcile and written to `status.steps` **once** (consolidated write), because the operator-sdk proxy caches reads for the duration of a reconcile and per-step writes would clobber each other.

GatewayVM Step Design

The GatewayVM step is handled directly by the Ansible reconcile loop, **not a Job**. Rationale: creating three Kubernetes resources on HCI (Secret, DataVolume, VirtualMachine) is fast and synchronous. The slow part — waiting for the VM to boot and become reachable — must not block a single reconcile; it is re-observed on every subsequent reconcile until the VM is up or the timeout expires.

State machine:

State	Condition	Action
`Pending`	No gateway VM exists on HCI	Apply cloud-init Secret + DataVolume + VirtualMachine; set step `InProgress` + record `lastTransitionTime`
`InProgress`	Resources exist; VM not yet reachable	Check TCP `gateway.external_ip:22`; if open → set `Complete`; if `lastTransitionTime` elapsed > `gateway_vm_timeout_seconds` → set `Failed`
`Complete`	SSH port reachable	No-op (terminal)
`Failed`	Timeout reached	No-op (terminal); `WorkerPoolProvisioning` phase → `Failed`; `WorkerPool` phase → `Failed`

Topology-dependent cloud-init (direct vs mediated):

Element	Direct	Mediated
`route1=` in external nmconnection	absent	`networking.linuxone_network`, `networking.linuxone_gateway`
Masquerade exception rules	none	ACCEPT for `linuxone_network` and `gateway.external_subnet` before the catch-all MASQUERADE rule
Catch-all MASQUERADE rule	present (all outbound eth1)	present (catches everything not matched by exceptions)

In direct topology the gateway simply masquerades all outbound traffic from `internal_subnet` via eth1. In mediated topology worker VM traffic to the s390x cluster network must appear with its original source IP (transparent routing) — masquerading is suppressed only for those specific destinations. Both cases use firewalld direct rules with priority 0 (exceptions) and priority 1 (catch-all MASQUERADE).

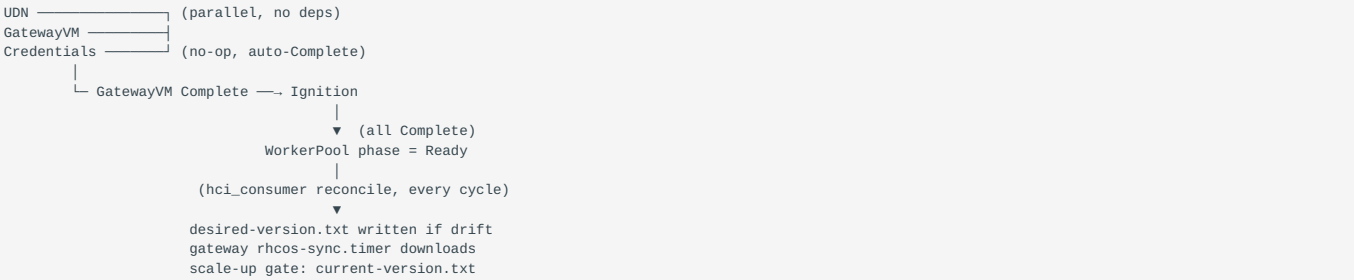
Teardown: delete VirtualMachine, wait for VMI to disappear (async), delete DataVolume, delete cloud-init Secret. Set step `Complete` once all three resources are absent.

Idempotency: all `kubernetes.core.k8s state: present/absent` calls are idempotent. If the operator restarts mid-step the resources already exist; re-applying is a no-op. TCP check resumes on the next reconcile.

Timeout default: `gateway_vm_timeout_seconds: 600` (10 minutes). Configurable in `roles/hci_provisioner/defaults/main.yml`.

Dependency Graph (bootstrap)

The active bootstrap steps and their dependencies are defined in `roles/hci_provisioner/defaults/main.yml` (`bootstrap_steps`):



`UDN`, `GatewayVM` and `Credentials` have no dependencies and run in the same reconcile. `Ignition` waits only on `GatewayVM` (SSH port must be open to distribute the file). `Credentials` is a placeholder that completes immediately — credential creation is done by script (BS-05 was descoped). There is no `Bootfiles` step in the provisioner — bootfiles version management is owned by `hci_consumer` (see design decision above).

> **MachineConfig is a consumer concern**
>
> The `MachineConfigPool` and `MachineConfig`s are created (if absent) by the **`hci_consumer`** role (`ensure_machineconfig.yml`, BS-02b) and the consumer gates creation of the `WorkerPoolProvisioning` on the pool being present and not `Degraded` (`MachineConfigPoolReady` condition on the `WorkerPool`). The provisioner has no visibility into consumer-cluster resources and has no `MachineConfig` step. The CRD enum still lists `MachineConfig` and `Bootfiles`, but neither is used.

GatewayVM does not depend on UDN: KubeVirt accepts the VirtualMachine object immediately; the NAD is resolved at VMI scheduling time, not at VM object creation time.

Key insight: `worker-hci.ign` is generated by rewriting the MCS endpoint in the base `worker-user-data-managed` Secret — that Secret always exists. The `hci-vm` MachineConfig is **not** consulted at ignition generation time; the node only contacts MCS at second boot. The target MCS path `/config/<machineConfigPool>` is taken from `infrastructure.network.machineConfigPool`.

Dependency Graph (teardown)

```
Ignition, Credentials   →  no deps (no-op, auto-Complete)
GatewayVM               →  no deps  (parallel with Workers)
Workers                 →  no deps  (parallel with GatewayVM)
UDN                     →  GatewayVM + Workers
```

`Workers` (BS-08) lists and deletes all consumer-labelled worker VMs and DataVolumes on the provider, and confirms them absent before `Complete`. `UDN` must wait for both `GatewayVM` and `Workers` because worker VMs hold UDN attachments and the UDN controller finalizer will not complete deletion while any VMI is still attached. `GatewayVM` and `Workers` teardown each use a re-check loop (`InProgress` → confirmed absent → `Complete`).

Consumer cleanup after teardown: `hci_consumer_cleanup` runs once the `WorkerPoolProvisioning` teardown is `Complete`. By then all VMs are gone, so it deletes the corresponding node objects directly (ownership fingerprint: name prefix + MCP role label) instead of requiring cordon+drain first. This avoids the deadlock that would otherwise occur because the gateway VM is gone and the nodes go `NotReady`.

Reconcile Loop (provisioning operator)

```
observe  (observe_world.yml):
  - WorkerPoolProvisioning spec (mode, step list)
  - current step statuses on WorkerPoolProvisioning.status.steps

diff  (diff_steps.yml):
  step Pending + all depends_on Complete          → launch (run the step task file)
  step Pending + dependencies not met             → no-op, wait
  step InProgress                                 → re-check (readiness / absence probe)

act  (advance_steps.yml):
  step results merged into steps[]                → single consolidated status write
  step probe succeeded                            → set step Complete
  step error / timeout                            → set step Failed, set phase Failed
  all steps Complete                              → set phase Complete
                                                    patch WorkerPool phase Ready
  phase Failed                                    → patch WorkerPool phase Failed
                                                    surface failure message
```

This is a fully stateless, re-entrant reconcile. Every decision is derivable from observed Kubernetes state — the operator can restart at any point and resume correctly.

3.6.5 WorkerPool Phase State Machine

The `WorkerPool` operator has a top-level phase gate. The worker reconcile loop only runs when `phase == Ready`.

```
    CR created
    credentials Secret present
    gateway VM absent
           |
           ▼
     [Provisioning]
operator creates WorkerPoolProvisioning
(mode=bootstrap)
worker reconcile: SKIP
           |
 WorkerPoolProvisioning phase=Complete
           |
           ▼
       [Ready]  ◄─── normal worker reconcile loop
    (workers.count reconciled continuously)
           |
    CR deletion requested
           |
           ▼
    [Deprovisioning]
operator creates WorkerPoolProvisioning
(mode=teardown)
worker reconcile: SKIP
           |
 WorkerPoolProvisioning phase=Complete
           |
           ▼
 finalizer released → CR deleted
```

`Provisioning` and `Deprovisioning` are observable via `oc get workerpool` (or `oc get wp`) — LOOps and administrators can see exactly where the consumer is in its lifecycle.

3.6.6 Step Implementations

Step	Mode	Task file	Responsibility	Key assets used
UDN	bootstrap / teardown	`step_udn.yml`, `step_udn_teardown.yml`	Pre-flight check that `ClusterUserDefinedNetwork router-external-network` exists; create / delete `UserDefinedNetwork router-internal-network`	`providerAccess.credential` kubeconfig
GatewayVM	bootstrap / teardown	`step_gateway_vm.yml`, `step_gateway_vm_readiness.yml`, `step_gateway_vm_teardown.yml`	Create cloud-init Secret + DataVolume + VirtualMachine; probe SSH; on teardown delete all three	Provider kubeconfig, rend cloud-init, provisioner SS Secret
Ignition	bootstrap only	`step_ignition.yml`	Rewrite MCS URL in `worker-user-data-managed`, SCP to gateway nginx root	Provisioner SSH key, gate internal IP
Workers	teardown only	`step_workers_teardown.yml`	Delete all consumer-labelled worker VMs + DataVolumes on the provider	Provider kubeconfig
Credentials	both	—	No-op placeholder (auto-Complete)	—

Note: There is no `Bootfiles` provisioning step. The consumer operator owns `desired-version.txt` management and the scale-up gate. See the separation of concerns decision above.

3.6.7 Testing Strategy

The per-step design makes isolated testing natural:

1. **Unit test a single step**: create a `WorkerPoolProvisioning` CR with only one step in `Pending` and all others `Complete`. Verify the step transitions correctly.
2. **Full bootstrap cycle**: apply `WorkerPool` CR with credentials absent (or gateway absent) → observe `WorkerPoolProvisioning` created → observe each step advance → observe `WorkerPool` phase reach `Ready` → verify gateway VM running on HCI.
3. **Full teardown cycle**: delete `WorkerPool` CR → observe teardown `WorkerPoolProvisioning` created → observe all assets removed → finalizer released.
4. **Create → delete → create**: the canonical idempotency test. Validates that teardown leaves the cluster clean enough for a fresh bootstrap.

3.6.8 Implementation Language

The provisioning operator controller is implemented as an **Ansible operator** — consistent with the steady-state operator and the rest of the project. The reconcile loop observes the step statuses, runs every step whose dependencies are satisfied, and writes the merged result once. Steps with no dependency between them run in the same reconcile pass; long waits (VM boot, resource deletion) are handled by re-checking on subsequent reconciles rather than blocking.

A migration to native Go operators (which would replace this loop with controller-runtime reconcilers) is tracked in Epic 6 — see Operator Optimization and Go Migration.

3.6.9 Related Documents

- Provider/Consumer Model
- Architecture Overview

Document Version: 2.0 **Last Updated**: 2026-10-12 **Status**: Aligned with the implemented operator (verified against `roles/hci_provisioner/defaults/main.yml`, `advance_steps.yml` and the CRDs). Steps run inline in the reconcile loop; MachineConfig handling lives in `hci_consumer`; `Bootfiles` and `Credentials` are not active provisioner steps.

3.7 Setup Concept — Direction C: `integration-setup.yaml`-Driven Day-0 Setup

3.7.1 Status

Implemented — the `generate-setup` script and `templates/integration-setup.example.yaml` are the production realisation of this concept. The description below remains accurate as design rationale; refer to Prerequisites — Environment Configuration and the Getting Started guide for the operational how-to.

3.7.2 The Problem

The hci-consumer and hci-provisioner operators are happy once a `WorkerPool` CR exists and the prerequisites are in place. The CR is the right abstraction for runtime — it is the single source of truth the operators act on.

But getting to that first `oc apply -f workerpool.yaml` requires a non-trivial amount of day-0 setup that is currently manual:

- **Main cluster (s390x/LinuxONE)**: MachineConfigPool + MachineConfig manifests applied by the main cluster admin; hci-node-manager built and deployed; operator images available
- **HCI cluster**: `router-external-network` CUDN exists; consumer namespace + quota created; `hci-provisioner-credentials` Secret produced by `setup-consumer-rbac.sh`
- **Joint**: registry model chosen and configured; WorkerPool CR fields agreed between HCI admin and main cluster admin

These setup steps involve decisions that must be negotiated between two admins who own different clusters. Today that negotiation is implicit and the results are scattered across `setup_env.sh`, hardcoded values in kustomize overlays, and README instructions.

3.7.3 The Concept: `integration-setup.yaml`

A single YAML file — the **cluster-pair configuration** — captures all the decisions that precede operator deployment. It is filled in jointly by the HCI admin (provider) and the main cluster admin (consumer) during the setup negotiation, and is the input to the generator script.

One file per HCI ↔ main cluster pair. Individual WorkerPools are instances within that relationship; their pool-specific parameters live in the WorkerPool CR, not here.

```yaml
# integration-setup.yaml
# Cluster-pair configuration — negotiated once between HCI admin and main cluster admin.
# Captures the decisions that precede any operator deployment.
# Input to generate-setup.

provider:
  context: hci-cluster              # oc context for the HCI cluster
  apiEndpoint: https://api.hci.example.com:6443
  namespace: main-cluster-workers
  gateway:
    ip: 172.16.1.100
    prefixLength: 16
    defaultRouter: 172.16.0.1
  networking:
    macPrefix: "52:54:01"
    internalSubnet: 192.168.100.0/24

consumer:
  context: main-cluster             # oc context for the main cluster
  domain: cluster.example.com
  operatorNamespace: hci-node-manager
  bastion:
    host: bastion.cluster.example.com
    user: root
  networking:
    topology: mediated              # direct | mediated
    dnsServer: 172.16.0.1

registry:
  model: cross-cluster              # local-s2i | cross-cluster | external
  proxyPort: 5443
```

```
  saName: registry-pusher
  secretName: hci-registry-push
```

What belongs here vs in the WorkerPool CR:

Decision	Where
Which two clusters	`integration-setup.yaml`
Network topology (direct/mediated)	`integration-setup.yaml`
MachineConfigPool name	`integration-setup.yaml`
Image registry model	`integration-setup.yaml`
HCI namespace for this pool	`WorkerPool.spec.provider.namespace`
Internal subnet for this pool	`WorkerPool.spec.provider.networking.internalSubnet`
Gateway IP	`WorkerPool.spec.provider.gateway.ip`
Max pool size	`WorkerPool.spec.provider.maxPoolSize`
Worker name prefix	`WorkerPool.spec.consumer.workers.namePrefix`
Desired worker count	`WorkerPool.spec.consumer.workers.count`

The rule: if it varies per WorkerPool, it belongs in the CR. If it is a cluster-pair-level decision made once, it belongs in `integration-setup.yaml`.

3.7.4 Generator Script

`generate-setup workspace/integration-setup.yaml`

Produces all **cluster-pair-level** day-0 artifacts. Run once when establishing a new HCI ↔ main cluster relationship.

Outputs:

Artifact	Description
`setup_env.sh`	Shell environment variables — sourced by all scripts and Makefiles.
`workspace/workerpool-hci-workers.yaml`	Initial `WorkerPool` CR (`workers.count: 0`). Main cluster admin applies, then edits to scale workers.
`operator/build/config.env`	Makefile variables for `make build` / `make deploy`.
`main-cluster/configuration/overlays/direct/`	Route MachineConfig YAMLs (direct topology only). Main cluster admin applies with `oc apply -k`.
`workspace/<domain>.zone`	Forward DNS zone section. Main cluster admin appends to bastion DNS.
`workspace/<subnet>.in-addr.arpa.zone`	Reverse DNS zonefile. Main cluster admin appends to bastion DNS.

All outputs are created with mode `0600` (readable by the owner only), and `generate-setup` refuses to write a file whose content looks like a credential (tokens, private keys). No secrets belong in these files; credentials are created on the cluster. If you hand a file to someone else, such as the zone files for the DNS admin, adjust its permissions or copy it with `scp` or `oc`.

The WorkerPool skeleton pre-fills: - `spec.consumer.machineConfigPool` ← from `consumer.machineConfigPool` - `spec.consumer.networking.topology` ← from `consumer.networking.topology` - `spec.consumer.networking.dnsServer` ← from `consumer.networking.dnsServer` - `metadata.namespace` ← from `consumer.operatorNamespace` - `spec.provider.credentialsSecret` ← default name

`generate-pool-setup` (not yet implemented)

A second generator for **per-pool** day-0 artifacts is planned but not yet implemented. When implemented it will consume an agreed WorkerPool CR and produce the HCI namespace + ResourceQuota manifest, CUDN manifest, and a pre-filled `setup-consumer-rbac.sh` invocation.

3.7.5 Relationship to the Operator

The operators are unaffected by this concept. The hci-consumer operator is entirely happy with a `WorkerPool` CR — it does not know or care how the prerequisites were set up. The configuration file and generator are a **developer experience** layer that sits entirely outside the operator runtime.

```
workspace/integration-setup.yaml
        │
        └─ generate-setup ──────────────► setup_env.sh
                                          workspace/workerpool-hci-workers.yaml
                                          operator/build/config.env
                                          main-cluster/configuration/overlays/<topology>/
                                          workspace/<domain>.zone
                                          workspace/<subnet>.in-addr.arpa.zone
                        │
                        ▼
             oc apply -f workspace/workerpool-hci-workers.yaml
                        │
                        ▼
             hci-consumer + hci-provisioner operators
             (runtime — no knowledge of setup phase)
```

3.7.6 Image Registry Models

Three models are supported, selected via `registry.model`:

Model	Build	Push	Pull	Day-0 setup
`local-s2i`	Main cluster S2I	Main cluster ImageStream	Main cluster	`buildconfig.yaml` + S2I build script
`cross-cluster`	Main cluster S2I	HCI registry via bastion proxy	Main cluster pulling from HCI	Registry proxy on bastion + cross-cluster push credentials
`external`	CI/CD or developer machine	quay.io / Artifactory / …	Both clusters	Registry credentials, image mirror policy

All three models apply equally to: hci-node-manager, hci-consumer operator image, hci-provisioner operator image. `generate-setup` resolves the correct `config.env` variables and registry URLs from the chosen model.

3.7.7 What This Is Not

- **Not a runtime component** — no new CRD, no new operator, no controller loop
- **Not a replacement for the WorkerPool CR** — the CR remains the operator's single source of truth

---

3.7.8 Remaining Work

1. Implement `generate-pool-setup` — per-pool HCI + main cluster artifacts (namespace+quota, CUDN, rbac script invocation)
2. Formalise `integration-setup.yaml` schema with JSON Schema for validation

Document Version: 1.1 **Last Updated**: 2026-07-22 **Status**: Implemented (`generate-setup`). `generate-pool-setup` not yet implemented.

4. Configuration

4.1 HCI Cluster Configuration

This document describes, step by step, what is configured on the HCI cluster to host the x86 worker VMs that join the main OpenShift cluster.

> **This page describes the manual steps behind the scripted setup**
>
> The recommended way to set up the HCI side is the script-supported flow of the Setup Guide. Every step on this page is either a command from that guide or something its scripts do for you. Use this page to understand what each step does, to repeat a single step by hand, or to troubleshoot.
>
> All values (cluster contexts, namespaces, addresses) come from `workspace/integration-setup.yaml` via `generate-setup`. The commands below use the variables from the generated `setup_env.sh`, for example `${HCI_CLUSTER_CONTEXT}` and `${HCI_INTEGRATION_NAMESPACE}`; source it first (see Workspace Setup). Example values in this page are those of `templates/integration-setup.example.yaml`.

> **Operator-managed**
>
> Gateway VM, internal `UserDefinedNetwork`, worker VM lifecycle and ignition distribution are managed by the **`hci-provisioner`** and **`hci-consumer`** operators. See `operator/hci-consumer/` and `operator/hci-provisioner/` for usage.

4.1.1 Overview

Manual step on this page	Setup Guide step	Scripted by	Cadence
Physical network mapping	prerequisite, not scripted	—	once per HCI cluster
Cluster-scoped external network	A.2	`oc apply -f hci-cluster/cluster-network-setup.yaml`	once per HCI cluster
Integration namespace and label	B.1	`oc create namespace` / `oc label`	once per main cluster
Consumer RBAC on the HCI cluster	B.2	`hci-cluster/setup-consumer-rbac.sh`	once per main cluster
Operator credentials	C.7	`operator/setup-provider-credentials.sh`	once per main cluster
WorkerPool CR	C.9	`generate-setup`, then `oc apply`	once per main cluster
Verification	Verify Integration	—	after each deployment

The HCI image registry (Setup Guide A.1) is not covered here.

4.1.2 Prerequisites

Before configuring the HCI cluster, ensure:

- ✅ HCI cluster is operational (OpenShift 4.20 or later)
- ✅ OpenShift Virtualization operator installed
- ✅ Sufficient resources available (CPU, memory, storage)
- ✅ Network connectivity to main cluster established
- ✅ `workspace/integration-setup.yaml` completed and `generate-setup` run

See Prerequisites for detailed requirements.

4.1.3 Cluster-Admin Setup

Done once per HCI cluster, shared by all integrations. Setup Guide: Tier A.

Step 1: Apply the Cluster-Scoped Network Resource

Purpose: Create the `ClusterUserDefinedNetwork` that gives every gateway VM its external connectivity.

Who: Cluster administrator (one-time setup)

Setup Guide: A.2

Command:

```
oc --context ${HCI_CLUSTER_CONTEXT} apply -f hci-cluster/cluster-network-setup.yaml
```

Resource created:

```
apiVersion: k8s.ovn.org/v1
kind: ClusterUserDefinedNetwork
metadata:
  name: router-external-network
spec:
  namespaceSelector:
    matchLabels:
      hci-gateway-enabled: "true"
  network:
    topology: Localnet
    localnet:
      physicalNetworkName: localnet1
      role: Secondary
      mtu: 1500
      ipam:
        mode: Disabled
```

Verification:

```
oc --context ${HCI_CLUSTER_CONTEXT} get clusteruserdefinednetwork router-external-network
# router-external-network    Active
```

Step 2: Physical Network Mapping

Purpose: Map the OVN network name `localnet1` (the `physicalNetworkName` above) to the bridge that is connected to the physical network of the HCI nodes.

Who: Cluster administrator. This depends on how the HCI nodes are wired and is a **prerequisite**, not part of the scripted setup. Apply it before the namespace is labeled in step B.1 of the Setup Guide.

Example (requires the Kubernetes NMState Operator; adapt the bridge to your nodes):

```
apiVersion: nmstate.io/v1
kind: NodeNetworkConfigurationPolicy
metadata:
  name: localnet1-bridge-mapping
spec:
```

```
  nodeSelector:
    node-role.kubernetes.io/worker: ""
  desiredState:
    ovn:
      bridge-mappings:
      - localnet: localnet1   # must equal physicalNetworkName in the ClusterUserDefinedNetwork
        bridge: br-ex         # bridge connected to the physical network
        state: present
```

Verification:

```
# Policy applied
oc --context ${HCI_CLUSTER_CONTEXT} get nncp localnet1-bridge-mapping

# Mapping present on a node
oc --context ${HCI_CLUSTER_CONTEXT} debug node/<hci-node> -- chroot /host \
  ovs-vsctl get open . external_ids:ovn-bridge-mappings
# "localnet1:br-ex,..."
```

4.1.4 Namespace Setup

Done once per main cluster you connect. Setup Guide: Tier B. The namespace name is `provider.namespace` in `integration-setup.yaml` (example: `main-cluster-workers`), available as `${HCI_INTEGRATION_NAMESPACE}`.

Step 1: Create Integration Namespace

Setup Guide: B.1

```
oc --context ${HCI_CLUSTER_CONTEXT} create namespace ${HCI_INTEGRATION_NAMESPACE}
```

Step 2: Label Namespace for Network Access

Purpose: Let the `ClusterUserDefinedNetwork` create the `router-external-network` attachment in this namespace.

```
oc --context ${HCI_CLUSTER_CONTEXT} label namespace ${HCI_INTEGRATION_NAMESPACE} \
  hci-gateway-enabled=true
```

Verification:

```
# Namespace label
oc --context ${HCI_CLUSTER_CONTEXT} get namespace ${HCI_INTEGRATION_NAMESPACE} --show-labels

# NetworkAttachmentDefinition created by the controller
oc --context ${HCI_CLUSTER_CONTEXT} get network-attachment-definition \
  -n ${HCI_INTEGRATION_NAMESPACE}
# router-external-network
```

Expected: the attachment should appear within 1-2 minutes after labeling. It must exist before the gateway VM is created.

Step 3: RBAC for the Operators

Setup Guide: B.2

```
./hci-cluster/setup-consumer-rbac.sh
```

The script requires the namespace from the previous steps and, in `${HCI_INTEGRATION_NAMESPACE}` on the HCI cluster, creates:

- the `hci-consumer-operator` ServiceAccount with a namespace-scoped Role and RoleBinding (VirtualMachines, DataVolumes, PVCs, Secrets, ConfigMaps, `UserDefinedNetwork`)
- a read-only ClusterRole for the `ClusterUserDefinedNetwork` existence check done by the provisioner
- the `registry-pusher` ServiceAccount and token, used only by the cross-cluster registry model

Pass `--delete` to remove everything again before re-running it.

Step 4: Secrets

No other Secrets need to be created by hand on the HCI cluster. The gateway VM cloud-init Secret and the provisioner SSH keypair are generated by the `hci-provisioner` operator. The only credential handover is the provider kubeconfig Secret on the main cluster — see Operator Deployment below.

4.1.5 Operator Deployment

The gateway VM, internal `UserDefinedNetwork`, ignition distribution and worker VM lifecycle are fully managed by the two operators. You apply **one CR** — the `WorkerPool`. Build and deployment of the operators themselves is described in Building and Deploying the Operators (Setup Guide C.8).

How the two operators interact

1. You apply the `WorkerPool` CR.
2. The `hci-consumer` operator creates the `MachineConfigPool` + `MachineConfig`s on the main cluster if absent, then gates on the pool being healthy.
3. The `hci-consumer` creates the `WorkerPoolProvisioning` CR automatically and sets `WorkerPool.status.phase = Provisioning`.
4. The `hci-provisioner` picks up the `WorkerPoolProvisioning` CR and runs the bootstrap steps (UDN → GatewayVM → Ignition) to `Complete`.
5. The provisioner patches `WorkerPool.status.phase = Ready`.
6. The `hci-consumer` immediately begins scaling up to `workers.count`.

Step 1: Set Up Credentials

Setup Guide: C.7

Run the credential setup script once. It uses both cluster contexts from `setup_env.sh`, creates the ServiceAccounts and Roles on the HCI cluster, and writes the `hci-provisioner-credentials` and `hci-provider-credentials` kubeconfig Secrets to the operator namespace (`${MAIN_INTEGRATION_NAMESPACE}`) on the main cluster:

```
./operator/setup-provider-credentials.sh
```

See `operator/setup-provider-credentials.sh` for the `--consumer` / `--provisioner` / `--delete` flags and environment overrides.

Step 2: Apply the WorkerPool CR

Setup Guide: C.9

The `WorkerPool` CR is not written by hand. `generate-setup` renders it from `integration-setup.yaml` into `workspace/workerpool-hci-workers.yaml` with `workers.count: 0`, so applying it provisions the gateway only. The provider values come from the HCI admin (`provider.*`), the consumer values from the main cluster admin (`consumer.*`). With the example file the generated CR contains:

```yaml
apiVersion: hci-integration.openshift.io/v1alpha1
kind: WorkerPool
metadata:
  name: hci-workers
  namespace: hci-node-manager
spec:
  provider:
    apiEndpoint: "https://api.hci.example.com:6443"
    namespace: "main-cluster-workers"
    credentialsSecret: "hci-provisioner-credentials"
    maxPoolSize: 10
    gateway:
      ip: "172.16.1.100"
      prefixLength: 16
      defaultRouter: "172.16.0.1"
    networking:
      macPrefix: "52:54:01"
      internalSubnet: "192.168.100.0/24"
  consumer:
    machineConfigPool: "hci-vm"
    networking:
```

```
      topology: "mediated"          # or: direct
      dnsServer: "172.16.0.1"
      internalNetwork: "10.0.0.0/24"   # mediated topology only
      internalGateway: "172.16.0.1"    # mediated topology only
    workers:
      count: 0
      namePrefix: "worker-x86"
```

The field-by-field mapping is in Architecture Overview. The operator repository also contains an annotated sample: `workerpool-sample.yaml`.

Apply and watch:

```
oc --context ${MAIN_CLUSTER_CONTEXT} apply -f workspace/workerpool-hci-workers.yaml

# WorkerPool: Provisioning → Ready
oc --context ${MAIN_CLUSTER_CONTEXT} -n ${MAIN_INTEGRATION_NAMESPACE} get workerpool hci-workers -w

# WorkerPoolProvisioning: created automatically; shows per-step progress
oc --context ${MAIN_CLUSTER_CONTEXT} -n ${MAIN_INTEGRATION_NAMESPACE} get workerpoolprovisioning -w
```

Once the pool is `Ready`, add workers as described under Scaling Operations. The deployment is complete when `PHASE=Ready` and `WORKERS`, `DEPLOYED` and `READY` are equal with `PENDING=0`.

4.1.6 Post-Deployment Verification

Setup Guide: Verify Integration. The checks below look at the HCI side. The gateway VM is named `hci-gateway`; it is reached over SSH at its external address (`provider.gateway.ip`), the VMs are not pods.

Step 1: Verify Gateway VM

Check VM status:

```
oc --context ${HCI_CLUSTER_CONTEXT} get vm hci-gateway -n ${HCI_INTEGRATION_NAMESPACE}
oc --context ${HCI_CLUSTER_CONTEXT} get vmi hci-gateway -n ${HCI_INTEGRATION_NAMESPACE}
# PHASE=Running
```

Check inside the gateway VM:

```
ssh core@172.16.1.100          # provider.gateway.ip

# 1. Network interfaces
ip addr show
# - eth1: 172.16.1.100/16   (external)
# - eth2: 192.168.100.1/24  (internal)

# 2. Routing
ip route show
# - default via 172.16.0.1 dev eth1
# - 10.0.0.0/24 via 172.16.0.1 dev eth1   (mediated topology only)
# - 192.168.100.0/24 dev eth2

# 3. Services (both active)
systemctl status dnsmasq nginx

# 4. DHCP log
sudo journalctl -u dnsmasq -n 50
```

Step 2: Verify Worker VMs

Check VM status:

```
oc --context ${HCI_CLUSTER_CONTEXT} get vm  -n ${HCI_INTEGRATION_NAMESPACE} -l app=worker-vm
oc --context ${HCI_CLUSTER_CONTEXT} get vmi -n ${HCI_INTEGRATION_NAMESPACE} -l app=worker-vm
# PHASE=Running for all
```

Check network boot:

```
virtctl console -n ${HCI_INTEGRATION_NAMESPACE} worker-x86-1

# Should show:
# 1. iPXE boot menu ("Worker VM Boot Menu")
# 2. RHCOS installation progress
# 3. Eventually: login prompt after installation
```

Monitor events:

```
oc --context ${HCI_CLUSTER_CONTEXT} get events -n ${HCI_INTEGRATION_NAMESPACE} --watch
```

Step 3: Verify Networks

```
# External network (from the ClusterUserDefinedNetwork) and internal network (UDN)
oc --context ${HCI_CLUSTER_CONTEXT} get network-attachment-definition \
  -n ${HCI_INTEGRATION_NAMESPACE}
# router-external-network
# router-internal-network

# Details of one attachment
oc --context ${HCI_CLUSTER_CONTEXT} get network-attachment-definition router-external-network \
  -n ${HCI_INTEGRATION_NAMESPACE} -o yaml
```

Connectivity test from the gateway VM:

```
ssh core@172.16.1.100

ping -c 3 172.16.0.1               # bastion / DNS (consumer.networking.dnsServer)
ping -c 3 api.cluster.example.com  # main cluster API
ping -c 3 192.168.100.10           # first worker VM (once it has booted)
```

4.1.7 Scaling Operations

Scale by editing the live `WorkerPool` on the main cluster, in the operator namespace (`${MAIN_INTEGRATION_NAMESPACE}`, example `hci-node-manager`). Workers can be added up to `provider.maxPoolSize`; DNS records and DHCP reservations exist for that many slots. See also Scaling the Worker Pool.

Adding Workers

```
oc --context ${MAIN_CLUSTER_CONTEXT} -n ${MAIN_INTEGRATION_NAMESPACE} patch workerpool hci-workers \
  --type merge -p '{"spec":{"consumer":{"workers":{"count":5}}}}'
```

The `hci-consumer` operator creates the additional VMs and waits for them to join the cluster. `WorkerPool.status.phase` moves through `ScalingUp` → `Ready`.

Removing Workers

```
oc --context ${MAIN_CLUSTER_CONTEXT} -n ${MAIN_INTEGRATION_NAMESPACE} patch workerpool hci-workers \
  --type merge -p '{"spec":{"consumer":{"workers":{"count":2}}}}'
```

The operator cordons and drains surplus nodes (highest-index first) before deleting the VMs. `WorkerPool.status.phase` reflects `ScalingDown` or `ScalingDownBlocked` (if live workloads are preventing drain) until the pool reaches the target size.

> **Regenerating the setup**
>
> `generate-setup` rewrites `workspace/workerpool-hci-workers.yaml` with `workers.count: 0`. To keep the current size of a live pool, save it with `oc get workerpool hci-workers -o yaml`.

4.1.8 Maintenance

Updating Gateway VM Configuration

Gateway VM resources (CPU, memory, storage) are set in `provider.gateway.resources` of `integration-setup.yaml` and end up in the `WorkerPoolProvisioning` CR. The gateway VM is a one-shot provisioned resource, not continuously reconciled after `Complete`, so a change requires a teardown and re-bootstrap.

For in-place configuration changes on a running gateway VM (for example dnsmasq tuning), SSH directly:

```
ssh core@172.16.1.100          # provider.gateway.ip
sudo -i
```

```
# make changes, e.g.:
systemctl restart dnsmasq
```

Updating Worker VMs

Worker VMs run RHCOS from the main cluster — OS-level updates happen via the main cluster's `MachineConfigPool`. To change VM hardware resources (CPU/memory), scale the pool to 0 and back up; the `hci-consumer` operator will recreate the VMs with the updated `WorkerPool` spec values.

4.1.9 Troubleshooting

Gateway VM Won't Start

Check:

```
# VM events
oc --context ${HCI_CLUSTER_CONTEXT} describe vm hci-gateway -n ${HCI_INTEGRATION_NAMESPACE}

# DataVolume status
oc --context ${HCI_CLUSTER_CONTEXT} get dv -n ${HCI_INTEGRATION_NAMESPACE}

# Storage capacity
oc --context ${HCI_CLUSTER_CONTEXT} get pvc -n ${HCI_INTEGRATION_NAMESPACE}

# Provisioner progress and errors
oc --context ${MAIN_CLUSTER_CONTEXT} -n ${MAIN_INTEGRATION_NAMESPACE} get workerpoolprovisioning -o yaml
```

Worker VM Won't Network-Boot

Check:

```
ssh core@172.16.1.100          # provider.gateway.ip

# DHCP/TFTP and HTTP services
sudo systemctl status dnsmasq nginx

# iPXE loader, boot script, RHCOS boot files
ls -la /var/lib/tftpboot/
ls -la /var/www/html/ /var/www/html/rhcos/

# Is the worker's MAC being offered an address?
sudo journalctl -u dnsmasq | grep -i dhcp | tail -20
```

See VM Provisioning for the full list of checks.

Network Attachment Fails

Check:

```
# Attachments exist
oc --context ${HCI_CLUSTER_CONTEXT} get network-attachment-definition -n ${HCI_INTEGRATION_NAMESPACE}

# Namespace carries the label
oc --context ${HCI_CLUSTER_CONTEXT} get namespace ${HCI_INTEGRATION_NAMESPACE} --show-labels

# ClusterUserDefinedNetwork status
oc --context ${HCI_CLUSTER_CONTEXT} get clusteruserdefinednetwork router-external-network -o yaml

# OVN controller logs
oc --context ${HCI_CLUSTER_CONTEXT} logs -n openshift-ovn-kubernetes -l app=ovnkube-node --tail=100
```

See Troubleshooting Guide for detailed solutions.

4.1.10 Best Practices

Resource Planning

1. **Start small** - Deploy 1-2 workers initially
2. **Monitor usage** - Check CPU, memory, storage utilization

3. **Plan capacity** - Ensure HCI cluster has sufficient resources
4. **Reserve resources** - Don't overcommit HCI cluster

Configuration Management

1. **Version control** - Keep `integration-setup.yaml` (without credentials) and an export of the live `WorkerPool` in Git
2. **Use separate CRs per consumer** - One `WorkerPool` CR per LinuxONE tenant
3. **Document changes** - Maintain changelog
4. **Test changes** - Verify in dev before production

Security

1. **Protect secrets** - Use sealed secrets or external secret management
2. **Limit access** - Use RBAC to control namespace access
3. **Network policies** - Restrict traffic where possible
4. **Regular updates** - Keep gateway VM OS updated

Operations

1. **Monitor continuously** - Watch `WorkerPool.status` phase and condition columns
2. **Backup CR manifests** - Keep the exported `WorkerPool` YAML in Git
3. **Test recovery** - Verify teardown and re-bootstrap procedures
4. **Document procedures** - Maintain runbooks

4.1.11 References

- Setup Guide
- Architecture Overview
- Network Design
- VM Provisioning
- Troubleshooting Guide
- hci-consumer Operator README
- hci-provisioner Operator

4.2 Main Cluster Configuration

This document describes the main-cluster side of the integration: what has to exist on the main OpenShift cluster (and its bastion) so that x86 worker VMs hosted on the HCI cluster can join it.

Manual steps behind the scripted setup

You normally do **not** perform these steps by hand. The Setup Guide – Tier C runs them for you with scripts, using the values from your `integration-setup.yaml` (rendered into `setup_env.sh` and `workspace/` by `generate-setup`). This page documents what each step does, so you can understand, verify, or repeat a single step manually.

Manual step on this page	Setup Guide step	Done by
Bastion routing and firewall	C.2	`main-cluster/scripts/01-add-route.sh`, `02-firewall-setup.sh`
MachineConfigPool and MachineConfigs	C.4	`oc apply -k` on the overlay (or the `hci-consumer` operator)
Registry proxy	C.5, C.6	`main-cluster/bastion-registry/setup-registry-proxy.sh`, `setup-registry-access.sh`
HCI Node Manager	C.8	Deployed with the `hci-consumer` operator
Operator credentials, WorkerPool CR	C.7, C.9	See HCI Cluster Configuration

DNS (C.1) and multi-architecture support (C.3) are covered only in the Setup Guide.

4.2.1 Overview

The main-cluster configuration consists of:

1. **MachineConfigPool and MachineConfigs** — created automatically by the `hci-consumer` operator from the `WorkerPool` CR; kustomize overlays exist as an optional manual fallback
2. **Deployment variant** — the *mediated* or *direct* overlay (the direct overlay adds static routes)
3. **Bastion routing** — mediated topology only
4. **Registry configuration** — only when `registry.model: cross-cluster`

4.2.2 Prerequisites

Before configuring the main cluster, ensure:

- ✅ Main cluster is operational (OpenShift 4.20+)
- ✅ Cluster admin access available (`${MAIN_CLUSTER_CONTEXT}`)
- ✅ Network connectivity to the HCI cluster established
- ✅ `integration-setup.yaml` written and `generate-setup` run (see Workspace Setup)
- ✅ Topology (`mediated` or `direct`) and registry model chosen in `integration-setup.yaml`

See Prerequisites for detailed requirements.

4.2.3 Configuration Components

Directory Structure

```
main-cluster/
├── configuration/
│   ├── base/                    # Common configuration
│   │   ├── kustomization.yaml
│   │   ├── machineconfig-pool-hci-vm.yaml
│   │   └── machineconfig-hci-vm-kargs.yaml
│   └── overlays/
│       ├── mediated/            # Mediated topology (no extra resources)
│       │   ├── kustomization.yaml
│       │   └── README.md
│       └── direct/              # Direct topology (adds route MachineConfigs)
│           ├── kustomization.yaml
│           ├── 99-master-s390x-hci-network-route.yaml   # generated
│           ├── 99-worker-s390x-hci-network-route.yaml   # generated
│           └── README.md
├── scripts/                     # Bastion helper scripts
└── bastion-registry/            # Registry proxy (cross-cluster model)
```

MachineConfigs are operator-managed

The `hci-consumer` operator creates the `MachineConfigPool`, the kernel-argument `MachineConfig` and — for the direct topology — the two route `MachineConfigs` from the `WorkerPool` CR if they do not exist. The kustomize overlays are the equivalent manual path: apply them if you want the pool to exist before the operator first reconciles (Setup Guide step C.4 does exactly that).

Generated route MachineConfigs

The two `99-*-s390x-hci-network-route.yaml` files in `overlays/direct/` are **generated by** `generate-setup` from `provider.networking.internalSubnet` and `provider.gateway.ip`. They are only written for `topology: direct`. Do not edit them by hand — change `integration-setup.yaml` and re-run `generate-setup`.

4.2.4 Bastion Routing and Firewall (Mediated Topology only)

Setup Guide: C.2

In the mediated topology the main-cluster nodes reach the HCI worker subnet through the bastion. The bastion needs a route to the internal subnet via the gateway VM's external IP, and a firewall rule that admits the internal subnet:

```
# Route: 192.168.100.0/24 via 172.16.1.100 (gateway external IP)
sudo ./main-cluster/scripts/01-add-route.sh
sudo ./main-cluster/scripts/02-firewall-setup.sh

ip route show | grep 192.168.100.0/24
# 192.168.100.0/24 via 172.16.1.100
```

The scripts source the generated `setup_env.sh` from the repository root (`INTERNAL_NETWORK`, `ROUTER_VM_EXTERNAL`).

4.2.5 MachineConfigPool and MachineConfigs

Setup Guide: C.4

Step 1: Review the Base MachineConfigs

MACHINECONFIGPOOL

File: `base/machineconfig-pool-hci-vm.yaml`

Purpose: Create a separate pool for the HCI workers

```yaml
apiVersion: machineconfiguration.openshift.io/v1
kind: MachineConfigPool
metadata:
  name: hci-vm
  labels:
```

```
    machineconfiguration.openshift.io/role: hci-vm
spec:
  machineConfigSelector:
    matchExpressions:
    - key: machineconfiguration.openshift.io/role
      operator: In
      values:
      - worker
      - hci-vm
    - key: node-platform
      operator: NotIn
      values:
      - s390x
  nodeSelector:
    matchLabels:
      node-role.kubernetes.io/hci-vm: ""
  paused: false
  maxUnavailable: 1
```

Key Features:

- Separate from the main worker pool; the name is `consumer.machineConfigPool` (default `hci-vm`)
- Selects nodes with the `node-role.kubernetes.io/hci-vm: ""` label
- Excludes MachineConfigs labelled `node-platform: s390x`, so the s390x route MachineConfig never lands on x86 VMs
- Can be paused independently for maintenance

KERNEL ARGUMENTS

File: `base/machineconfig-hci-vm-kargs.yaml` (`99-hci-vm-kargs`; also created by the operator on first reconcile if absent)

Purpose: Configure HCI worker-specific boot settings

```
kernelArguments:
  - "root=/dev/disk/by-label/root"
  - "rd.multipath=0"    # Disable multipath — not needed for VMs
  - "rd.neednet=1"
  - "ip=enp1s0:off"     # Disable the HCI pod-network interface (prevents OVN conflicts)
  - "ip=enp2s0:dhcp"    # Cluster network interface via (static-lease) DHCP
```

Node role labeling is done automatically by the `node-labeler` sidecar in the `hci-consumer-operator` pod — no MachineConfig labeling is required.

Step 2: Choose the Deployment Variant

The variant follows `consumer.networking.topology` in `integration-setup.yaml`.

MEDIATED TOPOLOGY (PRIVATE NETWORK)

When to use:

- Main-cluster nodes sit on a private network (example: `10.0.0.0/24`)
- The bastion (`172.16.0.1` in the example) routes all traffic to the HCI gateway
- Common in z/VM and LPAR environments

What's included: the base MachineConfigPool and kernel arguments. No additional routing on the cluster nodes — the bastion handles it.

Apply:

```
oc --context ${MAIN_CLUSTER_CONTEXT} apply -k main-cluster/configuration/overlays/mediated/
```

DIRECT TOPOLOGY (PUBLIC NETWORK)

When to use:

- Main-cluster nodes are on the same routable network as the HCI gateway's external IP (`172.16.1.100` in the example)
- Common in KVM and bare-metal environments

What's included: the base resources plus one static-route MachineConfig for the master nodes and one for the s390x worker nodes, each routing the internal subnet (`192.168.100.0/24`) via the gateway's external IP.

Apply:

```
oc --context ${MAIN_CLUSTER_CONTEXT} apply -k main-cluster/configuration/overlays/direct/
```

See Deployment Variants for a detailed comparison.

4.2.6 Applying Configuration

Step 1: Apply Base + Overlay

For Mediated Topology:

```
cd /path/to/fusion-hci-integration

oc --context ${MAIN_CLUSTER_CONTEXT} apply -k main-cluster/configuration/overlays/mediated/

# Verify the MachineConfigPool
oc --context ${MAIN_CLUSTER_CONTEXT} get mcp hci-vm

# Expected output:
# NAME     CONFIG    UPDATED   UPDATING   DEGRADED   MACHINECOUNT   READYMACHINECOUNT
# hci-vm   rendered  True      False      False      0              0
```

For Direct Topology:

```
oc --context ${MAIN_CLUSTER_CONTEXT} apply -k main-cluster/configuration/overlays/direct/

oc --context ${MAIN_CLUSTER_CONTEXT} get mcp hci-vm
oc --context ${MAIN_CLUSTER_CONTEXT} get mc 99-master-s390x-hci-network-route
oc --context ${MAIN_CLUSTER_CONTEXT} get mc 99-worker-s390x-hci-network-route

# The master and worker pools roll out the new route
oc --context ${MAIN_CLUSTER_CONTEXT} get mcp master worker
```

Step 2: Wait for MachineConfig Application

Mediated Topology: no node updates are required; the pool is ready immediately.

Direct Topology: master nodes and s390x worker nodes restart one by one. This can take 10–30 minutes.

```
watch oc --context ${MAIN_CLUSTER_CONTEXT} get mcp master worker

# Wait for UPDATED=True, UPDATING=False, DEGRADED=False
```

Step 3: Verify Configuration

```
# Pool details
oc --context ${MAIN_CLUSTER_CONTEXT} get mcp hci-vm -o yaml
# - machineConfigSelector includes "hci-vm"
# - nodeSelector matches node-role.kubernetes.io/hci-vm: ""
# - paused: false

# MachineConfigs for the HCI pool
oc --context ${MAIN_CLUSTER_CONTEXT} get mc | grep -E "hci"
# 99-hci-vm-kargs
# 99-master-s390x-hci-network-route   (direct topology only)
# 99-worker-s390x-hci-network-route   (direct topology only)
```

Direct topology — verify the routes on a node:

```
oc --context ${MAIN_CLUSTER_CONTEXT} debug node/<master-node>
chroot /host
ip route | grep 192.168.100
# 192.168.100.0/24 via 172.16.1.100 dev br-ex

# Same check on an s390x worker (no "dev br-ex")
# 192.168.100.0/24 via 172.16.1.100
```

4.2.7 HCI Node Manager

Setup Guide: C.8 (deployed with the operator)

Purpose

Automates the HCI worker node lifecycle:

- **CSR approval** — approves the bootstrap and serving CSRs of the worker VMs
- **Node labeling** — applies `node-role.kubernetes.io/hci-vm: ""`

Deployment

The node manager runs as two sidecar containers (`csr-approver` and `node-labeler`) inside the `hci-consumer-operator` Deployment. It is deployed automatically as part of the operator — no separate build or deployment step is required.

The sidecars are configured via the `hci-node-manager-config` ConfigMap, which the operator writes on every reconcile from the `WorkerPool` CR:

ConfigMap key	Source
`NODE_NAME_PATTERN`	`spec.consumer.workers.namePrefix` (example: `worker-x86`)
`TARGET_NETWORK`	`spec.provider.networking.internalSubnet` (example: `192.168.100.0/24`)

The cluster domain is derived automatically from `config.openshift.io/v1 DNS/cluster` — no manual configuration is needed.

Both sidecars re-read the ConfigMap every ~30 s, so a pod that started before the first reconcile picks up the config by itself. The `node-labeler` labels a node only if its name matches `<NODE_NAME_PATTERN>-<N>`, its `kubernetes.io/arch` is `amd64`, **and** its InternalIP is in `TARGET_NETWORK`. This keeps s390x control-plane and compute nodes out of the `hci-vm` MachineConfigPool even if the networks overlap.

Verification

```
# The consumer operator pod (the sidecars are containers within it)
oc --context ${MAIN_CLUSTER_CONTEXT} get pods -n hci-node-manager \
  -l app.kubernetes.io/name=hci-consumer-operator

# csr-approver logs
oc --context ${MAIN_CLUSTER_CONTEXT} logs -n hci-node-manager \
  deploy/hci-consumer-operator -c csr-approver | tail -20

# node-labeler logs
oc --context ${MAIN_CLUSTER_CONTEXT} logs -n hci-node-manager \
  deploy/hci-consumer-operator -c node-labeler | tail -20
```

4.2.8 Registry Configuration (Cross-Cluster Model only)

Setup Guide: C.5 and C.6

When Needed

Only when `registry.model: cross-cluster` is set in `integration-setup.yaml`. With `local-s2i` or `external`, skip this section.

Setup

Location: `main-cluster/bastion-registry/`

1. **Bastion HAProxy and cluster insecure-registry entry** — run as your normal user, **not** with `sudo` (the script uses `sudo` internally for HAProxy, firewall and certificates, and runs `oc` with your own login against `${MAIN_CLUSTER_CONTEXT}`):

```
./main-cluster/bastion-registry/setup-registry-proxy.sh
```

The script generates a self-signed certificate, configures HAProxy to forward `bastion.cluster.example.com:5443` to the HCI registry, and adds that address to `image.config.openshift.io/cluster` `spec.registrySources.insecureRegistries` on the main cluster. That change triggers a MachineConfigPool rollout (10–30 minutes).

2. **Registry push secret** — reads the token of the `registry-pusher` ServiceAccount from the HCI cluster (created by `setup-consumer-rbac.sh`, see HCI Cluster Configuration) and stores it as a docker-registry Secret on the main cluster:

```
./main-cluster/bastion-registry/setup-registry-access.sh
# creates Secret hci-registry-push in namespace hci-node-manager
```

Verify:

```
oc --context ${MAIN_CLUSTER_CONTEXT} get image.config.openshift.io cluster \
  -o jsonpath='{.spec.registrySources.insecureRegistries}'
curl -sk https://localhost:5443/v2/ && echo ok     # on the bastion
oc --context ${MAIN_CLUSTER_CONTEXT} get secret hci-registry-push -n hci-node-manager
```

See the Bastion Registry README for details.

4.2.9 Helper Scripts

The scripts in `main-cluster/scripts/` run on the bastion and source the generated `setup_env.sh`:

Script	Purpose	Setup Guide
`01-add-route.sh`	Route to the internal subnet via the gateway external IP (mediated)	C.2
`02-firewall-setup.sh`	Verify/complete the bastion firewall configuration (mediated)	C.2

The worker ignition file (`worker-hci.ign`) is produced by the `hci-provisioner` operator; no script is needed for it.

4.2.10 Post-Configuration Tasks

Step 1: Verify the MachineConfigPool

```
oc --context ${MAIN_CLUSTER_CONTEXT} get mcp hci-vm

# MACHINECOUNT: 0 (no workers yet)
# READYMACHINECOUNT: 0
# UPDATED: True   UPDATING: False   DEGRADED: False
```

Step 2: Prepare for Worker Join

CSRs are approved automatically by the `csr-approver` sidecar in the `hci-consumer-operator` pod. No manual action is needed once the operator is deployed.

```
# Monitor csr-approver activity
oc --context ${MAIN_CLUSTER_CONTEXT} logs -n hci-node-manager \
  deploy/hci-consumer-operator -c csr-approver -f

# Manual approval (e.g. operator not yet deployed)
oc --context ${MAIN_CLUSTER_CONTEXT} get csr | grep Pending
oc --context ${MAIN_CLUSTER_CONTEXT} get csr | awk '/Pending/ {print $1}' \
  | xargs -r oc --context ${MAIN_CLUSTER_CONTEXT} adm certificate approve
```

Step 3: Monitor Worker Join

```
watch oc --context ${MAIN_CLUSTER_CONTEXT} get nodes -l node-role.kubernetes.io/hci-vm
watch oc --context ${MAIN_CLUSTER_CONTEXT} get mcp hci-vm

# As workers join, MACHINECOUNT and READYMACHINECOUNT increase
```

4.2.11 Updating Configuration

Updating MachineConfigs

```
# Edit the MachineConfig
vi main-cluster/configuration/base/machineconfig-hci-vm-kargs.yaml

# Apply the change
oc --context ${MAIN_CLUSTER_CONTEXT} apply -k main-cluster/configuration/overlays/<variant>/

# Workers restart to apply it
watch oc --context ${MAIN_CLUSTER_CONTEXT} get mcp hci-vm
```

The `hci-consumer` operator only creates the MachineConfigs when they are absent; it never overwrites an existing one, so manual changes are preserved.

Pausing updates:

```
oc --context ${MAIN_CLUSTER_CONTEXT} patch mcp hci-vm --type=merge -p '{"spec":{"paused":true}}'
# Make changes ...
oc --context ${MAIN_CLUSTER_CONTEXT} patch mcp hci-vm --type=merge -p '{"spec":{"paused":false}}'
```

Switching Variants

Not recommended: switching the topology after deployment requires redeploying the workers.

If necessary:

```
# 1. Scale the pool to 0 workers (edit spec.consumer.workers.count) and wait
oc --context ${MAIN_CLUSTER_CONTEXT} -n hci-node-manager edit workerpool hci-workers

# 2. Delete the old route MachineConfigs (and the pool, if it is to be recreated)
oc --context ${MAIN_CLUSTER_CONTEXT} delete mc 99-master-s390x-hci-network-route 99-worker-s390x-hci-network-route --ignore-not-found

# 3. Change consumer.networking.topology in integration-setup.yaml,
#    re-run generate-setup, then apply the matching overlay
oc --context ${MAIN_CLUSTER_CONTEXT} apply -k main-cluster/configuration/overlays/<new-variant>/

# 4. Scale the pool back up
```

4.2.12 Troubleshooting

MachineConfigPool Stuck Updating

Symptoms: `UPDATING: True` for an extended period, or `DEGRADED: True`.

```
oc --context ${MAIN_CLUSTER_CONTEXT} get mcp hci-vm -o yaml
oc --context ${MAIN_CLUSTER_CONTEXT} get mcp hci-vm -o jsonpath='{.status.conditions[*].message}'

# machine-config-daemon logs on affected nodes
oc --context ${MAIN_CLUSTER_CONTEXT} logs -n openshift-machine-config-operator -l k8s-app=machine-config-daemon
```

Workers Not Joining the Pool

Symptoms: workers appear in the cluster but not in the `hci-vm` pool.

```
oc --context ${MAIN_CLUSTER_CONTEXT} get node worker-x86-1 --show-labels
# Should have: node-role.kubernetes.io/hci-vm

# The node-labeler requires: name matches worker-x86-<N>, arch amd64,
# InternalIP in 192.168.100.0/24.  Check its log first:
oc --context ${MAIN_CLUSTER_CONTEXT} logs -n hci-node-manager \
  deploy/hci-consumer-operator -c node-labeler | tail -30
```

```
# If you must label manually:
oc --context ${MAIN_CLUSTER_CONTEXT} label node worker-x86-1 node-role.kubernetes.io/hci-vm=
```

CSRs Not Being Approved

```
oc --context ${MAIN_CLUSTER_CONTEXT} get csr | grep Pending

oc --context ${MAIN_CLUSTER_CONTEXT} logs -n hci-node-manager \
  deploy/hci-consumer-operator -c csr-approver | tail -30
```

Routes Not Applied (Direct Topology)

```
# MachineConfigs exist
oc --context ${MAIN_CLUSTER_CONTEXT} get mc 99-master-s390x-hci-network-route 99-worker-s390x-hci-network-route

# Applied to the pools
oc --context ${MAIN_CLUSTER_CONTEXT} get mcp master -o yaml | grep 99-master-s390x-hci-network-route
oc --context ${MAIN_CLUSTER_CONTEXT} get mcp worker -o yaml | grep 99-worker-s390x-hci-network-route

# Route on a node
oc --context ${MAIN_CLUSTER_CONTEXT} debug node/<node>
chroot /host
ip route | grep 192.168.100
```

See the Troubleshooting Guide for detailed solutions.

4.2.13 Best Practices

Configuration Management

1. **Keep `integration-setup.yaml` as the single source of truth** — regenerate rather than hand-edit generated files
2. **Version control** — keep `integration-setup.yaml` and any customized MachineConfigs in Git (never the generated `setup_env.sh` / credentials)
3. **Test changes** — verify in a dev environment before production
4. **Document customizations** — maintain a changelog

MachineConfigPool Management

1. **Separate pools** — keep HCI workers isolated
2. **Pause for maintenance** — prevent automatic updates during changes
3. **Monitor status** — watch for degraded state
4. **Plan updates** — schedule during maintenance windows

Security

1. **Limit access** — use RBAC for MachineConfig management
2. **Audit changes** — review MachineConfig modifications
3. **Protect credentials** — secure ignition files, SSH keys and operator credential Secrets
4. **Regular updates** — keep RHCOS updated

Operations

1. **CSR approval is automatic** — handled by the `csr-approver` sidecar in the consumer operator
2. **Monitor continuously** — set up alerts for pool status
3. **Test recovery** — verify disaster recovery procedures
4. **Document procedures** — maintain runbooks

4.2.14 References

- Setup Guide – Tier C
- Architecture Overview
- Deployment Variants
- HCI Cluster Configuration
- Troubleshooting Guide
- Kustomize Overlays

4.3 Kustomize Overlays Guide

This document explains the Kustomize structure in `main-cluster/configuration/`, which holds the main-cluster `MachineConfigPool` and `MachineConfig` resources for the two deployment variants.

> **Manual path behind the scripted setup**
>
> The overlays are what Setup Guide step C.4 – Apply MachineConfigs applies. The values inside the **direct** overlay are not written by hand: `generate-setup` renders the two route MachineConfigs from your `integration-setup.yaml`. The same resources are also created by the `hci-consumer` operator if they are absent (see Main Cluster Configuration).

4.3.1 Overview

The integration uses Kustomize to manage configuration variants:

- **Base** — common configuration for all deployments
- **Overlays** — variant-specific additions
- `mediated` — mediated topology (private network, bastion routes traffic)
- `direct` — direct topology (main-cluster nodes route to the HCI gateway themselves)

The overlay you apply follows `consumer.networking.topology` in `integration-setup.yaml`.

4.3.2 Why Kustomize?

- **DRY** — common configuration is not repeated
- **Variant management** — one overlay per deployment type
- **Native to OpenShift** — `oc apply -k`, no extra tooling
- **Version-control friendly** — clear separation of base and variant

4.3.3 Directory Structure

```
main-cluster/configuration/
├── base/
│   ├── kustomization.yaml
│   ├── machineconfig-pool-hci-vm.yaml
│   └── machineconfig-hci-vm-kargs.yaml
└── overlays/
    ├── mediated/
    │   ├── kustomization.yaml
    │   └── README.md
    └── direct/
        ├── kustomization.yaml
        ├── 99-master-s390x-hci-network-route.yaml   # generated by generate-setup
        ├── 99-worker-s390x-hci-network-route.yaml   # generated by generate-setup
        └── README.md
```

4.3.4 What `generate-setup` Writes

File	Written when	Source values in `integration-setup.yaml`
`overlays/direct/99-master-s390x-hci-network-route.yaml`	`consumer.networking.topology: direct`	`provider.networking.internalSubnet`, `provider.gateway.ip`
`overlays/direct/99-worker-s390x-hci-network-route.yaml`	`consumer.networking.topology: direct`	`provider.networking.internalSubnet`, `provider.gateway.ip`

With `topology: mediated` (the value in the example file) nothing is written — the mediated overlay needs no per-installation values. The base files and both `kustomization.yaml` files are static and are never touched by `generate-setup`.

> ⚠ **Do not edit the generated route files**
>
> They are overwritten on every `generate-setup` run. To change the route, change `provider.gateway.ip` or `provider.networking.internalSubnet` and re-run `generate-setup`.

4.3.5 Base Configuration

kustomization.yaml

Location: `main-cluster/configuration/base/kustomization.yaml`

```yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

resources:
  - machineconfig-pool-hci-vm.yaml
  - machineconfig-hci-vm-kargs.yaml

commonLabels:
  hci-integration: "true"
```

- **resources** — the base manifests
- **commonLabels** — `hci-integration: "true"` is added to every base resource

Base Resources

1. MACHINECONFIGPOOL

File: `machineconfig-pool-hci-vm.yaml`

Purpose: a separate pool for the HCI worker VMs

```yaml
apiVersion: machineconfiguration.openshift.io/v1
kind: MachineConfigPool
metadata:
  name: hci-vm
  labels:
    machineconfiguration.openshift.io/role: hci-vm
spec:
  machineConfigSelector:
    matchExpressions:
      - key: machineconfiguration.openshift.io/role
        operator: In
        values: [worker, hci-vm]
      # Exclude s390x platform-specific MachineConfigs (multipath storage)
      - key: node-platform
        operator: NotIn
        values: [s390x]
  nodeSelector:
    matchLabels:
      node-role.kubernetes.io/hci-vm: ""
  paused: false
  maxUnavailable: 1
```

Why in the base: required for every variant and has no variant-specific values. The pool name matches `consumer.machineConfigPool` (default `hci-vm`). The `node-platform NotIn s390x` rule keeps the s390x route MachineConfig (below) from being rendered into the x86 pool.

2. KERNEL ARGUMENTS

File: `machineconfig-hci-vm-kargs.yaml`

Purpose: boot settings for the x86 worker VMs

```yaml
apiVersion: machineconfiguration.openshift.io/v1
kind: MachineConfig
metadata:
  name: 99-hci-vm-kargs
  labels:
```

```
    machineconfiguration.openshift.io/role: hci-vm
    node-platform: hci-vm
spec:
  kernelArguments:
    - "root=/dev/disk/by-label/root"
    - "rd.multipath=0"       # no s390x multipath storage on VMs
    - "rd.neednet=1"
    - "ip=enp1s0:off"        # disable the HCI pod-network interface
    - "ip=enp2s0:dhcp"       # cluster network interface (static DHCP lease)
  config:
    ignition:
      version: 3.5.0
```

Why in the base: every worker VM needs these arguments, independent of the topology. Node role labeling is **not** done here — the `node-labeler` sidecar in the `hci-consumer-operator` pod labels the nodes.

4.3.6 Mediated Overlay

kustomization.yaml

Location: `main-cluster/configuration/overlays/mediated/kustomization.yaml`

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

namespace: openshift-machine-config-operator

bases:
  - ../../base
```

Characteristics: minimal — only references the base. The bastion routes the traffic to the HCI worker subnet (see Setup Guide C.2).

When to Use

Use the mediated overlay when:

- ✅ Main-cluster nodes are on a private network (example: `10.0.0.0/24`)
- ✅ The bastion (example: `172.16.0.1`) routes all traffic to the HCI gateway
- ✅ Common in z/VM and LPAR environments

What It Provides

- The `hci-vm` MachineConfigPool
- Kernel arguments for the worker VMs (including disabling `enp1s0`)
- No additional routing configuration

Application

```
oc --context ${MAIN_CLUSTER_CONTEXT} apply -k main-cluster/configuration/overlays/mediated/

# Verify
oc --context ${MAIN_CLUSTER_CONTEXT} get mcp hci-vm
oc --context ${MAIN_CLUSTER_CONTEXT} get mc 99-hci-vm-kargs
```

4.3.7 Direct Overlay

kustomization.yaml

Location: `main-cluster/configuration/overlays/direct/kustomization.yaml`

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

namespace: openshift-machine-config-operator

bases:
  - ../../base
```

```
resources:
  - 99-master-s390x-hci-network-route.yaml
  - 99-worker-s390x-hci-network-route.yaml
```

Characteristics: extends the base with two routing MachineConfigs — one for the masters, one for the s390x workers.

When to Use

Use the direct overlay when:

- ✅ Main-cluster nodes can reach the gateway's external IP (example: `172.16.1.100`) directly
- ✅ No bastion routing is wanted
- ✅ Common in KVM and bare-metal environments

Additional Resources

Both files are rendered by `generate-setup`. The examples below show the output for `internalSubnet: 192.168.100.0/24` and `gateway.ip: 172.16.1.100`.

1. MASTER NODE ROUTE

File: `99-master-s390x-hci-network-route.yaml`

Purpose: let the masters reach the HCI worker VMs (CSR handling, node status, scheduling, health checks)

```
apiVersion: machineconfiguration.openshift.io/v1
kind: MachineConfig
metadata:
  labels:
    machineconfiguration.openshift.io/role: master
  name: 99-master-s390x-hci-network-route
spec:
  config:
    ignition:
      version: 3.5.0
    systemd:
      units:
        - name: hci-network-route.service
          enabled: true
          contents: |
            [Unit]
            Description=Add static route to HCI worker network
            After=network-online.target ovs-configuration.service
            Wants=network-online.target
            Requires=ovs-configuration.service

            [Service]
            Type=oneshot
            RemainAfterExit=yes
            ExecStartPre=/bin/bash -c 'until ip link show br-ex; do sleep 1; done'
            ExecStart=/bin/bash -c '/usr/sbin/ip route add 192.168.100.0/24 via 172.16.1.100 dev br-ex || true'

            [Install]
            WantedBy=multi-user.target
```

The master unit waits for `ovs-configuration.service` and the `br-ex` bridge, because the route is bound to `br-ex`.

2. S390X WORKER ROUTE

File: `99-worker-s390x-hci-network-route.yaml`

Purpose: let the dedicated s390x workers reach the HCI worker VMs (pod-to-pod traffic, service endpoints, multi-architecture applications)

```
apiVersion: machineconfiguration.openshift.io/v1
kind: MachineConfig
metadata:
  labels:
    machineconfiguration.openshift.io/role: worker
    node-platform: s390x
  name: 99-worker-s390x-hci-network-route
spec:
  config:
    ignition:
      version: 3.5.0
    systemd:
```

```
      units:
        - name: hci-network-route.service
          enabled: true
          contents: |
            [Unit]
            Description=Add static route to HCI worker network
            After=network-online.target
            Wants=network-online.target

            [Service]
            Type=oneshot
            RemainAfterExit=yes
            ExecStart=/bin/bash -c '/usr/sbin/ip route add 192.168.100.0/24 via 172.16.1.100 || true'

            [Install]
            WantedBy=multi-user.target
```

The worker route has no `dev br-ex`. The `node-platform: s390x` label is what keeps this MachineConfig out of the x86 `hci-vm` pool (see the pool's `NotIn` rule above).

Note: the worker route only applies to **dedicated** s390x workers. Compact clusters (master+worker roles) only need the master route.

Application

```
oc --context ${MAIN_CLUSTER_CONTEXT} apply -k main-cluster/configuration/overlays/direct/

# Base resources
oc --context ${MAIN_CLUSTER_CONTEXT} get mcp hci-vm
oc --context ${MAIN_CLUSTER_CONTEXT} get mc 99-hci-vm-kargs

# Routing MachineConfigs
oc --context ${MAIN_CLUSTER_CONTEXT} get mc 99-master-s390x-hci-network-route
oc --context ${MAIN_CLUSTER_CONTEXT} get mc 99-worker-s390x-hci-network-route

# Master and worker pools roll out the route (10–30 minutes)
watch oc --context ${MAIN_CLUSTER_CONTEXT} get mcp master worker
# Wait for UPDATED=True, UPDATING=False
```

4.3.8 Creating Custom Overlays

Custom overlays are not part of the supported flow, but the structure allows them. Keep them **outside** the generated files so `generate-setup` does not overwrite your work.

Use Case: Additional MachineConfig

1. **Create the overlay directory:**

```
mkdir -p main-cluster/configuration/overlays/custom
```

2. **Create `kustomization.yaml`:**

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

resources:
  - ../mediated          # or ../direct
  - custom-config.yaml

patches:
  - path: patch-kargs.yaml
```

3. **Add the custom resource:**

```
# custom-config.yaml
apiVersion: machineconfiguration.openshift.io/v1
kind: MachineConfig
metadata:
  labels:
    machineconfiguration.openshift.io/role: hci-vm
  name: 99-hci-vm-custom
spec:
  config:
    ignition:
      version: 3.5.0
    storage:
      files:
        - path: /etc/custom.conf
```

```
          contents:
            inline: |
              # Custom configuration
```

4. **Apply:**

```
oc --context ${MAIN_CLUSTER_CONTEXT} apply -k main-cluster/configuration/overlays/custom/
```

Use Case: Patching the Base

Example: add a kernel argument without changing the base files

```
# patch-kargs.yaml
apiVersion: machineconfiguration.openshift.io/v1
kind: MachineConfig
metadata:
  name: 99-hci-vm-kargs
spec:
  kernelArguments:
    - "root=/dev/disk/by-label/root"
    - "rd.multipath=0"
    - "rd.neednet=1"
    - "ip=enp1s0:off"
    - "ip=enp2s0:dhcp"
    - "custom.arg=value"     # added
```

A strategic-merge patch replaces the `kernelArguments` list, so repeat the existing entries and append yours.

4.3.9 Best Practices

Overlay Design

1. **Keep the base minimal** — only truly common configuration
2. **Use overlays for variants** — do not modify the base for specific cases
3. **Do not hand-edit generated files** — change `integration-setup.yaml` instead
4. **Test each overlay independently**

Configuration Management

1. **Version control** — keep custom overlays in Git
2. **Tag releases** — tag stable overlay versions
3. **Document changes** — maintain a changelog

Naming Conventions

1. **Descriptive names** — `mediated`, `direct`, not `variant1`
2. `99-` **prefix** for high-priority MachineConfigs
3. **Clear labels** — use labels to identify resources

Testing

1. **Dry run first** — `oc apply -k <dir> --dry-run=client`
2. **Test in dev** — verify the overlay in a development cluster
3. **Validate output** — `oc kustomize <dir>`
4. **Monitor** — watch MachineConfigPool status

4.3.10 Troubleshooting

Kustomize Build Fails

Symptoms:

```
Error: accumulating resources: accumulation err='accumulating resources from '../../base': ...
```

Check:

```
# Render the overlay
oc kustomize main-cluster/configuration/overlays/direct/

# Verify the base path and referenced files exist
ls -la main-cluster/configuration/base/
ls -la main-cluster/configuration/overlays/direct/
```

For the direct overlay, `no such file ... 99-master-s390x-hci-network-route.yaml` means the route files were not generated — check that `topology: direct` is set in `integration-setup.yaml` and re-run `generate-setup`.

Resources Not Applied

Symptoms: `oc apply -k` succeeds but resources are missing.

```
# Preview what would be applied
oc apply -k main-cluster/configuration/overlays/direct/ --dry-run=client -o yaml

# Look for the resources
oc --context ${MAIN_CLUSTER_CONTEXT} get mc | grep hci
```

Patches Not Working

Symptoms: a patch is defined but not applied.

```
# Build and inspect the output
oc kustomize main-cluster/configuration/overlays/custom/ | grep -A 20 "kind: MachineConfig"

# Verify the patch target (name and kind) matches the base resource
```

Wrong Overlay Applied

Symptoms: the wrong variant was applied.

```
# Delete the resources of the wrong overlay
oc --context ${MAIN_CLUSTER_CONTEXT} delete -k main-cluster/configuration/overlays/<wrong-variant>/

# Apply the correct overlay
oc --context ${MAIN_CLUSTER_CONTEXT} apply -k main-cluster/configuration/overlays/<correct-variant>/

# Verify
oc --context ${MAIN_CLUSTER_CONTEXT} get mcp hci-vm
oc --context ${MAIN_CLUSTER_CONTEXT} get mc -l hci-integration=true
```

Deleting an overlay also deletes the base `hci-vm` pool and kernel-argument MachineConfig (they are part of both overlays). Re-applying the correct overlay recreates them; no workers should be running while you do this.

4.3.11 References

- Kustomize Documentation
- Setup Guide – C.4 Apply MachineConfigs
- Main Cluster Configuration
- Deployment Variants
- Architecture Overview

5. Operations

5.1 Troubleshooting Guide

This guide covers common issues encountered when deploying and operating the Fusion HCI multi-architecture integration, along with their solutions.

Troubleshooting and the Setup Guide

Most initial setup issues trace back to specific steps in the Setup Guide. When debugging, verify the corresponding stage and automated resources:

Issue area	Setup Guide step / Component	Key resources & logs
Gateway VM & CUDN / NAD	Tier A, Tier B	`ClusterUserDefinedNetwork`, `UserDefinedNetwork`, `vmi/hci-gateway`
DNS & Bastion routing	C.1, C.2	Bastion `named`, `ip route`, `iptables`
Direct topology routes	C.4	`99-master-s390x-hci-network-route`, `99-worker-s390x-hci-network-route`
Worker boot & PXE	C.8, C.9	Gateway `dnsmasq`, `nginx`, `rhcos-sync.service`
Node joining & CSRs	C.8	`csr-approver`, `node-labeler`, `mcp/hci-vm`

5.1.1 Table of Contents

- Network Issues
- VM Deployment Issues
- Node Join Issues
- DNS and Routing Issues
- Multi-Namespace Deployment Issues
- Diagnostic Commands
- Gateway Verification

5.1.2 Network Issues

Worker VM Has Dual Network Conflict

Symptoms: - Worker VM has two active interfaces (enp1s0 and enp2s0) - Dual default routes causing routing conflicts - DNS resolution fails or returns wrong results - Node cannot reach main cluster API

Root Cause: The worker VM has two network interfaces: - **enp1s0**: Connected to HCI cluster's Pod network (should be disabled) - **enp2s0**: Connected to main cluster network (should be active)

Both interfaces receive DHCP configuration, creating conflicts.

Solution:

The `99-hci-vm-kargs` MachineConfig (and the iPXE boot command) disables `enp1s0` and configures `enp2s0` via kernel arguments:

```
# 99-hci-vm-kargs (created automatically by the hci-consumer operator)
apiVersion: machineconfiguration.openshift.io/v1
kind: MachineConfig
```

```
metadata:
  name: 99-hci-vm-kargs
  labels:
    machineconfiguration.openshift.io/role: hci-vm
    node-platform: hci-vm
spec:
  kernelArguments:
    - "root=/dev/disk/by-label/root"
    - "rd.multipath=0"
    - "rd.neednet=1"
    - "ip=enp1s0:off"
    - "ip=enp2s0:dhcp"
  config:
    ignition:
      version: 3.5.0
```

During installation, the iPXE boot command on the gateway also supplies `coreos.inst.karg=ip=enp1s0:off coreos.inst.karg=ip=enp2s0:dhcp` so that RHCOS installs and boots with the correct interface active.

Verification:

```
# Debug worker node
oc --context ${MAIN_CLUSTER_CONTEXT} debug node/worker-x86-1
chroot /host

# Check interface status
ip link show enp1s0  # Should be DOWN or unconfigured
ip link show enp2s0  # Should be UP

# Check IP addresses
ip addr show enp1s0  # Should have no IP
ip addr show enp2s0  # Should have 192.168.100.x

# Check routes
ip route show  # Should only show routes via enp2s0 (192.168.100.1 gateway)

# Check DNS
cat /etc/resolv.conf  # Should show main cluster DNS (172.16.0.1)
```

OVN-Kubernetes br-ex Bridge Conflict

Symptoms: - OVN-Kubernetes creates br-ex bridge on wrong interface (enp1s0) - Default route appears on br-ex instead of enp2s0 - Worker cannot reach main cluster API - DNS resolution uses wrong DNS server

Root Cause: OVN-Kubernetes automatically selects the first available network interface for the external bridge (br-ex). In HCI workers, it incorrectly chooses enp1s0 instead of enp2s0.

Solution:

Force OVN to use enp2s0 for br-ex:

```
# In MachineConfig
storage:
  files:
    - path: /etc/systemd/system/ovs-configuration.service.d/override.conf
      mode: 0644
      overwrite: true
      contents:
        inline: |
          [Service]
          Environment="OVN_KUBERNETES_EXTERNAL_INTERFACE=enp2s0"
```

Verification:

```
# Check OVN interface selection
systemctl cat ovs-configuration.service | grep OVN_KUBERNETES_EXTERNAL_INTERFACE

# Check br-ex is using enp2s0
ovs-vsctl show | grep -A 5 br-ex

# Should show enp2s0 as port, not enp1s0
```

Gateway VM Gets Wrong IP Address (IPAM Issue)

Symptoms: - Gateway VM receives a dynamic IP (e.g. `172.16.0.2`) instead of the requested static IP (`172.16.1.100`) - Network conflicts or routing mismatches with bastion - Workers cannot reach main cluster through the gateway

Root Cause: ClusterUserDefinedNetwork with Localnet topology does not honor static IP addresses specified in pod annotations. OVN's IPAM for localnet networks assigns IPs sequentially from the subnet.

Solution:

The gateway VM configures its static IP via cloud-init network configuration (handled automatically by the `hci-provisioner` operator using `gateway-vm-cloudinit.j2`):

```
# Inside gateway-vm-cloudinit Secret (rendered by operator)
apiVersion: v1
kind: Secret
metadata:
  name: gateway-vm-cloudinit
stringData:
  networkdata: |
    version: 2
    ethernets:
      external:
        match:
          macaddress: "52:54:01:00:00:01"
        addresses:
          - 172.16.1.100/16
        gateway4: 172.16.0.1
        nameservers:
          addresses:
            - 172.16.0.1
      internal:
        match:
          macaddress: "52:54:01:00:00:02"
        addresses:
          - 192.168.100.1/24
```

Why This Works: - Cloud-init configures the static IP and MAC matching directly inside the VM - No dependency on OVN IPAM for the external physical interface - Standard approach for physical network attachment - Gateway has full control over its external and internal routing

Verification:

```
# Check gateway VM status
oc --context ${HCI_CLUSTER_CONTEXT} get vmi -n ${HCI_INTEGRATION_NAMESPACE} hci-gateway

# SSH to gateway VM
ssh core@172.16.1.100

# Inside VM: check IP addresses
ip addr show
# Should show 172.16.1.100 on external interface (eth1)
# Should show 192.168.100.1 on internal interface (eth2)
```

5.1.3 VM Deployment Issues

VM Stuck in ContainerCreating

Symptoms: - virt-launcher pod stuck in ContainerCreating for hours - VM never starts - Pod events show network attachment errors

Possible Causes:

1. WRONG NETWORKATTACHMENTDEFINITION REFERENCE

Error Message:

```
failed to get pod annotation: timed out waiting for annotations: context deadline exceeded
```

Check:

```
# Get NAD configuration
oc --context ${HCI_CLUSTER_CONTEXT} get nad router-external-network -n ${HCI_INTEGRATION_NAMESPACE} -o jsonpath='{.spec.config}' | jq .

# Check netAttachDefName field
# Should be: "${HCI_INTEGRATION_NAMESPACE}/router-external-network"
# NOT: "other-namespace/router-external-network"
```

Solution: See Multi-Namespace Deployment Issues

2. MISSING CLUSTERUSERDEFINEDNETWORK

Check:

```
# Verify ClusterUserDefinedNetwork exists on HCI
oc --context ${HCI_CLUSTER_CONTEXT} get clusteruserdefinednetwork router-external-network

# Verify namespace is labeled
oc --context ${HCI_CLUSTER_CONTEXT} get namespace ${HCI_INTEGRATION_NAMESPACE} --show-labels | grep hci-gateway-enabled
```

Solution:

```
# Apply cluster-admin network setup (Setup Guide A.2)
oc --context ${HCI_CLUSTER_CONTEXT} apply -f hci-cluster/cluster-network-setup.yaml

# Label namespace (Setup Guide B.1)
oc --context ${HCI_CLUSTER_CONTEXT} label namespace ${HCI_INTEGRATION_NAMESPACE} hci-gateway-enabled=true
```

3. OVN CONTROLLER ISSUES

Check:

```
# Check OVN controller logs on HCI
oc --context ${HCI_CLUSTER_CONTEXT} logs -n openshift-ovn-kubernetes -l app=ovnkube-node --tail=100 | grep -i error

# Check for NAD creation
oc --context ${HCI_CLUSTER_CONTEXT} get nad -n ${HCI_INTEGRATION_NAMESPACE}
```

Solution: Wait for OVN controller to create NAD (usually 1-2 minutes after namespace labeling).

VM Fails to Boot from PXE

Symptoms: - VM starts but doesn't PXE boot - Console shows "No bootable device" or iPXE timeout - DHCP requests not answered

Possible Causes:

1. GATEWAY VM NOT RUNNING

Check:

```
oc --context ${HCI_CLUSTER_CONTEXT} get vmi -n ${HCI_INTEGRATION_NAMESPACE} hci-gateway
# Should show "Running"
```

Solution:

```
# Check virt-launcher pod logs
oc --context ${HCI_CLUSTER_CONTEXT} logs -n ${HCI_INTEGRATION_NAMESPACE} -l kubevirt.io/domain=hci-gateway

# Restart if needed
virtctl restart hci-gateway -n ${HCI_INTEGRATION_NAMESPACE}
```

2. DHCP/PXE SERVICE NOT RUNNING

Check:

```
# SSH to gateway VM
ssh core@<gateway-external-ip>

# Check dnsmasq and nginx
systemctl status dnsmasq nginx
journalctl -u dnsmasq -n 50
```

Solution:

```
# Restart services
sudo systemctl restart dnsmasq nginx

# Check static DHCP reservations
cat /etc/dnsmasq.d/pxe.conf
```

3. NETWORK CONNECTIVITY ISSUE

Check:

```
# From gateway VM, test internal network
ping 192.168.100.10  # Worker IP

# Check firewall rules
sudo iptables -L -n -v | grep 192.168.100
```

4. RHCOS BOOT FILES MISSING OR DOWNLOADING

Check:

```
# SSH to gateway VM
ssh core@<gateway-external-ip>

# Verify boot files exist in webroot
ls -lh /var/www/html/rhcos/

# Check sync timer status
systemctl status rhcos-sync.service
```

5.1.4 Node Join Issues

Worker Node Doesn't Appear in Cluster

Symptoms: - Worker VM boots successfully - RHCOS installed - Node doesn't appear in `oc get nodes`

Diagnostic Steps:

```
# 1. Check if node is trying to join
oc --context ${MAIN_CLUSTER_CONTEXT} get csr | grep Pending

# 2. Access worker node (via debug or SSH from bastion)
oc --context ${MAIN_CLUSTER_CONTEXT} debug node/worker-x86-1  # If visible
# Or: ssh core@192.168.100.10 (from bastion)
# Or: virtctl console worker-x86-1 -n ${HCI_INTEGRATION_NAMESPACE}

# 3. Check kubelet status
chroot /host
systemctl status kubelet
journalctl -u kubelet -n 100

# 4. Check machine-config-daemon
systemctl status machine-config-daemon
journalctl -u machine-config-daemon -n 100
```

Common Issues:

1. CSR NOT APPROVED

The `csr-approver` container in the `hci-consumer-operator` pod automatically approves both bootstrap and node-serving CSRs for nodes matching the configured prefix, internal subnet, and `amd64` architecture.

Manual Approval (if approver is not running):

```
# Approve pending CSRs
oc --context ${MAIN_CLUSTER_CONTEXT} get csr -o name | xargs oc --context ${MAIN_CLUSTER_CONTEXT} adm certificate approve
```

2. CANNOT REACH API SERVER

Check:

```
# From worker node
curl -k https://api.cluster.example.com:6443/version
```

Solution: - Verify DNS resolution - Check routing (see DNS and Routing Issues) - Verify gateway VM is routing correctly

3. STALE `kube-apiserver-lb-signer` CA BUNDLE (MCP CREATED LONG BEFORE FIRST WORKER)

Symptoms:

- Worker installs RHCOS and boots successfully
- kubelet cannot register with the API server
- SSH to the worker (`ssh core@worker-x86-<n>` from the bastion) shows:

```
journalctl -b | grep -i "22623\|mcs\|ignition\|tls" | tail -30
# x509: certificate signed by unknown authority (possibly because of
# "crypto/rsa: verification error" while trying to verify candidate
# authority certificate "kube-apiserver-lb-signer")
```

Root cause:

The `rendered-hci-vm` MachineConfig is generated by the MCO when the `hci-vm` MachineConfigPool is first created. It bakes in the current `kube-apiserver-lb-signer` CA bundle at that point in time. If a significant amount of time passes between MCP creation and the first worker actually joining the pool, the cluster's lb-signer certificate rotates — but the MCO does not re-render an empty pool (no nodes = no trigger). Workers that boot and receive the stale `rendered-hci-vm` apply the old CA bundle and kubelet can no longer verify the API server certificate.

Once the MCP is current, additional workers can join at any later time: the MCO keeps the rendered MC up to date for pools that have at least one node, so the bundle stays current going forward.

Fix:

```
# 1. Scale the WorkerPool down to zero (removes all worker VMs cleanly)
oc --context ${MAIN_CLUSTER_CONTEXT} patch workerpool hci-workers -n ${MAIN_INTEGRATION_NAMESPACE} \
  --type=merge -p '{"spec":{"consumer":{"workers":{"count":0}}}}'

# Wait for all VMs to be gone
oc --context ${HCI_CLUSTER_CONTEXT} get vm -n ${HCI_INTEGRATION_NAMESPACE} -w

# 2. Delete the MCP and its source MachineConfig
#    The hci-consumer operator recreates them on the next reconcile,
#    causing the MCO to render a fresh rendered-hci-vm with the current
#    kube-apiserver-lb-signer bundle.
oc --context ${MAIN_CLUSTER_CONTEXT} delete mcp hci-vm
oc --context ${MAIN_CLUSTER_CONTEXT} delete mc 99-hci-vm-kargs

# 3. Wait for the operator to recreate the MCP and for the MCO to render
oc --context ${MAIN_CLUSTER_CONTEXT} get mc -w | grep rendered-hci-vm          # wait for new rendered-hci-vm
oc --context ${MAIN_CLUSTER_CONTEXT} get mcp hci-vm -o jsonpath='{.status.configuration.name}'  # must be non-empty

# 4. Verify the MCS now serves the config without error
curl -sk https://api-int.${MAIN_CLUSTER_DOMAIN}:22623/config/hci-vm \
  | jq .ignition.version

# 5. Scale back up — start with one canary worker to confirm
oc --context ${MAIN_CLUSTER_CONTEXT} patch workerpool hci-workers -n ${MAIN_INTEGRATION_NAMESPACE} \
  --type=merge -p '{"spec":{"consumer":{"workers":{"count":1}}}}'

# Once worker-x86-1 is Ready, scale to the desired count
oc --context ${MAIN_CLUSTER_CONTEXT} patch workerpool hci-workers -n ${MAIN_INTEGRATION_NAMESPACE} \
  --type=merge -p '{"spec":{"consumer":{"workers":{"count":3}}}}'
```

4. MACHINE CONFIG DAEMON STUCK

Check:

```
journalctl -u machine-config-daemon-pull.service -f
```

Common causes: - Cannot pull images (check internet connectivity) - Cannot reach image registry - Disk space issues

Solution:

```
# Check disk space
df -h

# Test image pull
podman pull quay.io/openshift-release-dev/ocp-release:latest

# Check registry connectivity
curl -v https://quay.io
```

Node Stuck in NotReady State

Symptoms: - Node appears in cluster - Status shows NotReady - Pods cannot be scheduled

Check:

```
# Get node details
oc describe node worker-x86-1

# Look for conditions
oc get node worker-x86-1 -o jsonpath='{.status.conditions[*].message}'
```

Common Issues:

1. CNI NOT READY

Solution:

```
# Check OVN pods on node
oc get pods -n openshift-ovn-kubernetes -o wide | grep worker-x86-1

# Check logs
oc logs -n openshift-ovn-kubernetes <ovn-pod-name>
```

2. CONTAINER RUNTIME ISSUES

Solution:

```
# SSH to node
oc debug node/worker-x86-1
chroot /host

# Check CRI-O
systemctl status crio
journalctl -u crio -n 100
```

WorkerPool Stuck in ScalingUp Although All Workers Are Ready

Symptoms: - `oc get workerpool -n hci-node-manager` shows `phase: ScalingUp` indefinitely - `status.deployedWorkers` equals `spec.consumer.workers.count`, but `status.readyWorkers: 0` - All `worker-x86-*` nodes are `Ready` and report `kubernetes.io/arch=amd64` - The x86 nodes do **not** carry the `node-role.kubernetes.io/hci-vm` label - `oc get mcp hci-vm` shows `MACHINECOUNT 0`; the x86 nodes are counted in the `worker` pool instead - The WorkerPool may still show the stale condition `WorkerReconcileActive=False / BootstrapInProgress`, even though the `WorkerPoolProvisioning` CR is `Complete`. This condition is left over from the first reconcile and does not mean anything here.

Background:

The operator counts a node as ready only when the node name matches `<namePrefix>-*` **and** the node has the MCP role label `node-role.kubernetes.io/<machineConfigPool>`. The `node-labeler` sidecar in the `hci-consumer-operator` pod adds this label. It reads its config from the `hci-node-manager-config` ConfigMap, which the operator writes on its first WorkerPool reconcile.

Root cause in older images (fixed by OPS-08):

Older `node-labeler` versions read `TARGET_NETWORK` **only once, at startup**. If the operator pod started before a WorkerPool CR existed, which is the normal order on a fresh install, the ConfigMap was missing. The labeler then fell back to `192.168.100.0/24` and kept that value for the lifetime of the pod.

Current images re-read the ConfigMap every ~30 s and fix themselves. If the symptom still shows up with a current image, the most likely cause is a mismatch between `TARGET_NETWORK` / `NODE_NAME_PATTERN` and the actual node IPs/names. Check the labeler's debug output and compare the ConfigMap with the node list.

Check:

```
# Check node-labeler logs
oc --context ${MAIN_CLUSTER_CONTEXT} logs -n ${MAIN_INTEGRATION_NAMESPACE} deploy/hci-consumer-operator -c node-labeler | grep -E "fallback|Target network|
Labeling"
# Healthy output:
#   Labeling active: network=192.168.100.0/24, name pattern=worker-x86-.*, arch=amd64

# Network the labeler should be using
oc --context ${MAIN_CLUSTER_CONTEXT} get cm hci-node-manager-config -n ${MAIN_INTEGRATION_NAMESPACE} -o jsonpath='{.data.TARGET_NETWORK}{"\n"}'
```

```
# Nodes missing the role label
oc --context ${MAIN_CLUSTER_CONTEXT} get nodes -l 'kubernetes.io/arch=amd64,!node-role.kubernetes.io/hci-vm'
```

Solution:

```
# Option A (preferred): restart the operator pod to force a re-scan
oc --context ${MAIN_CLUSTER_CONTEXT} rollout restart deployment/hci-consumer-operator -n ${MAIN_INTEGRATION_NAMESPACE}
oc --context ${MAIN_CLUSTER_CONTEXT} rollout status deployment/hci-consumer-operator -n ${MAIN_INTEGRATION_NAMESPACE}

# Option B: label the nodes manually
oc --context ${MAIN_CLUSTER_CONTEXT} label node <worker-x86-N>.<cluster-domain> node-role.kubernetes.io/hci-vm= --overwrite
```

Note: Once labelled, the nodes move from the `worker` MCP to the `hci-vm` MCP. HCI workers boot from `/config/hci-vm` and already run the `rendered-hci-vm-*` config. Check this with the node annotation `machineconfiguration.openshift.io/currentConfig`. If it already shows `rendered-hci-vm-*`, labelling triggers **no** MCO update and **no** reboot. If it shows `rendered-worker-*`, MCO applies the `hci-vm` config and reboots the nodes one at a time (`maxUnavailable: 1`).

⚠️ Never put `hci-vm` on s390x nodes. The `hci-vm` config carries x86-VM kernel arguments and would make them unbootable. Current labeler images enforce this through a name + arch guard.

Verify:

```
oc --context ${MAIN_CLUSTER_CONTEXT} get nodes -l node-role.kubernetes.io/hci-vm
oc --context ${MAIN_CLUSTER_CONTEXT} get mcp hci-vm                      # MACHINECOUNT == worker count
oc --context ${MAIN_CLUSTER_CONTEXT} get workerpool -n ${MAIN_INTEGRATION_NAMESPACE}   # phase: Ready, readyWorkers == count
```

5.1.5 DNS and Routing Issues

DNS Resolution Fails

Symptoms: - Cannot resolve cluster hostnames - Cannot resolve internet hostnames - Intermittent resolution failures

Diagnostic Steps:

```
# From worker node
oc --context ${MAIN_CLUSTER_CONTEXT} debug node/worker-x86-1
chroot /host

# Check DNS configuration
cat /etc/resolv.conf

# Test DNS resolution
dig api.cluster.example.com
dig quay.io

# Test DNS server directly
dig @172.16.0.1 api.cluster.example.com
```

Common Issues:

1. WRONG DNS SERVER

Expected:

```
nameserver 172.16.0.1
search cluster.example.com
```

Solution: Verify the DHCP server (`dnsmasq`) on the gateway VM is advertising `dhcp-option=option:dns-server,172.16.0.1` and `dhcp-option=option:domain-search,cluster.example.com`.

2. DNS SERVER NOT ALLOWING RECURSION

Check:

```
# On bastion/DNS server (172.16.0.1)
# Check named configuration
grep -r "allow-recursion" /etc/named*

# Should include internal worker network and mediated network
allow-recursion { localhost; 10.0.0.0/24; 192.168.100.0/24; 172.16.0.0/16; };
```

Solution: Update DNS server configuration to allow recursion from worker network.

Routing Issues

Symptoms: - Cannot reach main cluster API - Cannot reach internet - Packets not being forwarded

Diagnostic Steps:

```
# From worker node
oc --context ${MAIN_CLUSTER_CONTEXT} debug node/worker-x86-1
chroot /host

# Check routing table
ip route show

# Test connectivity
ping -c 3 192.168.100.1  # Gateway
ping -c 3 api.cluster.example.com
ping -c 3 8.8.8.8  # Internet

# Trace route
traceroute api.cluster.example.com
```

Common Issues:

1. MISSING DEFAULT ROUTE

Solution:

```
# Check if default route exists
ip route show default

# If missing, check DHCP
journalctl -u NetworkManager | grep DHCP
```

2. GATEWAY NOT ROUTING (MEDIATED TOPOLOGY)

Check:

```
# On bastion
ip route | grep 192.168.100

# Should show route to HCI internal network via gateway external IP
```

Solution:

```
# Add route on bastion (Setup Guide C.2 / 01-add-route.sh)
sudo ip route add 192.168.100.0/24 via 172.16.1.100

# Make persistent
sudo nmcli connection modify <conn-name> +ipv4.routes "192.168.100.0/24 172.16.1.100"
```

3. MASTER NODES CANNOT REACH WORKERS (DIRECT TOPOLOGY)

Symptoms: - CSRs not being approved - Node status updates delayed - Pods cannot be scheduled

Check:

```
# From master node
oc --context ${MAIN_CLUSTER_CONTEXT} debug node/<master-node>
chroot /host

# Check route to HCI network
ip route | grep 192.168.100

# Test connectivity
ping -c 3 192.168.100.10
```

Solution: Verify MachineConfig for master route is applied (Setup Guide C.4):

```
# Check MachineConfig
oc --context ${MAIN_CLUSTER_CONTEXT} get mc 99-master-s390x-hci-network-route

# Check if applied to masters
oc --context ${MAIN_CLUSTER_CONTEXT} get mcp master

# If not applied, apply from workspace rendered by generate-setup
oc --context ${MAIN_CLUSTER_CONTEXT} apply -f workspace/main-cluster-machineconfigs/
```

4. S390X WORKERS CANNOT REACH HCI WORKERS (DIRECT TOPOLOGY)

Symptoms: - Cross-architecture pod communication fails - Services spanning both worker types don't work - Network policies fail

Check:

```
# From s390x worker
oc --context ${MAIN_CLUSTER_CONTEXT} debug node/<s390x-worker>
chroot /host

# Check route
ip route | grep 192.168.100

# Test connectivity
ping -c 3 192.168.100.10
```

Solution: Verify MachineConfig for s390x worker route is applied (Setup Guide C.4):

```
# Check MachineConfig
oc --context ${MAIN_CLUSTER_CONTEXT} get mc 99-worker-s390x-hci-network-route

# Check if applied to workers
oc --context ${MAIN_CLUSTER_CONTEXT} get mcp worker

# If not applied, apply from workspace rendered by generate-setup
oc --context ${MAIN_CLUSTER_CONTEXT} apply -f workspace/main-cluster-machineconfigs/
```

Gateway VM NAT Issues

Symptoms: - Workers can reach main cluster but not internet - Or vice versa

Check:

```
# SSH to gateway VM
ssh core@172.16.1.100

# Check NAT rules
sudo iptables -t nat -L -n -v | grep 192.168.100

# Check IP forwarding
sysctl net.ipv4.ip_forward  # Should be 1
```

Expected NAT Rules:

```
# Transparent routing (no NAT) for mediated private network
ACCEPT all -- 192.168.100.0/24 10.0.0.0/24

# Transparent routing (no NAT) for external network
ACCEPT all -- 192.168.100.0/24 172.16.0.0/16

# Masquerading (NAT) for internet
MASQUERADE all -- 192.168.100.0/24 0.0.0.0/0
```

Solution: Check gateway VM cloud-init configuration and redeploy if needed.

5.1.6 Multi-Namespace Deployment Issues

Second Namespace Deployment Fails

Symptoms: - First namespace deployment works fine - Second namespace VMs stuck in ContainerCreating - Network attachment errors in pod events

Root Cause: ClusterUserDefinedNetwork is cluster-scoped. Only the cluster-admin should create it; each integration namespace gets the resulting NAD automatically via the namespace label.

Error Message:

```
network cluster_udn_router-external-network NAD main-cluster-workers/router-external-network
failed to get pod annotation: timed out waiting for annotations
```

Solution:

1. **Cluster admin applies cluster-scoped resources once (Setup Guide A.2):**

```
oc --context ${HCI_CLUSTER_CONTEXT} apply -f hci-cluster/cluster-network-setup.yaml
```

2. **Label each namespace (Setup Guide B.1):**

```
oc --context ${HCI_CLUSTER_CONTEXT} label namespace <namespace> hci-gateway-enabled=true
```

3. **Apply `WorkerPoolProvisioning` CR** — the `hci-provisioner` operator creates the namespace-scoped `UserDefinedNetwork` and gateway VM automatically.

Quick fix for existing deployments:

```
# 1. Apply cluster-admin setup
oc --context ${HCI_CLUSTER_CONTEXT} apply -f hci-cluster/cluster-network-setup.yaml

# 2. Label all namespaces
oc --context ${HCI_CLUSTER_CONTEXT} label namespace main-cluster-workers hci-gateway-enabled=true
oc --context ${HCI_CLUSTER_CONTEXT} label namespace main-cluster-workers-2 hci-gateway-enabled=true

# 3. Stop gateway VMs
oc --context ${HCI_CLUSTER_CONTEXT} patch vm hci-gateway -n main-cluster-workers --type merge -p '{"spec":{"running":false}}'
oc --context ${HCI_CLUSTER_CONTEXT} patch vm hci-gateway -n main-cluster-workers-2 --type merge -p '{"spec":{"running":false}}'

# 4. Delete incorrect NADs
oc --context ${HCI_CLUSTER_CONTEXT} delete nad router-external-network -n main-cluster-workers
oc --context ${HCI_CLUSTER_CONTEXT} delete nad router-external-network -n main-cluster-workers-2

# 5. Wait for controller to recreate NADs (1-2 minutes)
watch oc --context ${HCI_CLUSTER_CONTEXT} get nad -A

# 6. Verify NAD configuration
oc --context ${HCI_CLUSTER_CONTEXT} get nad router-external-network -n main-cluster-workers -o jsonpath='{.spec.config}' | jq .netAttachDefName
# Should show: "main-cluster-workers/router-external-network"

# 7. Start VMs
oc --context ${HCI_CLUSTER_CONTEXT} patch vm hci-gateway -n main-cluster-workers --type merge -p '{"spec":{"running":true}}'
oc --context ${HCI_CLUSTER_CONTEXT} patch vm hci-gateway -n main-cluster-workers-2 --type merge -p '{"spec":{"running":true}}'
```

5.1.7 Diagnostic Commands

Quick Health Check

```
#!/bin/bash
# Quick health check for HCI integration

echo "=== Cluster Status (Main Cluster) ==="
oc --context ${MAIN_CLUSTER_CONTEXT} get nodes -l node-role.kubernetes.io/hci-vm

echo "=== MachineConfigPools (Main Cluster) ==="
oc --context ${MAIN_CLUSTER_CONTEXT} get mcp hci-vm

echo "=== Gateway VM (HCI Cluster) ==="
oc --context ${HCI_CLUSTER_CONTEXT} get vmi -n ${HCI_INTEGRATION_NAMESPACE} hci-gateway

echo "=== Worker VMs (HCI Cluster) ==="
oc --context ${HCI_CLUSTER_CONTEXT} get vmi -n ${HCI_INTEGRATION_NAMESPACE} -l app=worker-vm

echo "=== Pending CSRs (Main Cluster) ==="
oc --context ${MAIN_CLUSTER_CONTEXT} get csr | grep Pending

echo "=== Network Attachments (HCI Cluster) ==="
oc --context ${HCI_CLUSTER_CONTEXT} get nad -n ${HCI_INTEGRATION_NAMESPACE}

echo "=== ClusterUserDefinedNetwork (HCI Cluster) ==="
oc --context ${HCI_CLUSTER_CONTEXT} get clusteruserdefinednetwork router-external-network
```

Network Diagnostics

```
#!/bin/bash
# Network diagnostics from worker node

echo "=== Interfaces ==="
ip link show

echo "=== IP Addresses ==="
ip addr show
```

```
echo "=== Routes ==="
ip route show

echo "=== DNS Configuration ==="
cat /etc/resolv.conf

echo "=== Test DNS ==="
dig api.cluster.example.com
dig quay.io

echo "=== Test Connectivity ==="
ping -c 3 192.168.100.1  # Gateway
ping -c 3 api.cluster.example.com  # Main cluster
ping -c 3 8.8.8.8  # Internet
```

Gateway VM Diagnostics

```
#!/bin/bash
# Gateway VM diagnostics (run via ssh core@<gateway-ip>)

echo "=== Network Interfaces ==="
ip addr show

echo "=== Routing Table ==="
ip route show

echo "=== NAT Rules ==="
sudo iptables -t nat -L -n -v

echo "=== IP Forwarding ==="
sysctl net.ipv4.ip_forward

echo "=== DHCP/PXE Services ==="
systemctl status dnsmasq nginx

echo "=== Static DHCP Reservations ==="
cat /etc/dnsmasq.d/pxe.conf

echo "=== Test Connectivity ==="
ping -c 3 172.16.0.1  # Bastion/DNS
ping -c 3 api.cluster.example.com  # Main cluster
ping -c 3 8.8.8.8  # Internet
```

Log Collection

```
#!/bin/bash
# Collect logs for troubleshooting

LOGDIR="hci-integration-logs-$(date +%Y%m%d-%H%M%S)"
mkdir -p "$LOGDIR"

# Main Cluster resources
oc --context ${MAIN_CLUSTER_CONTEXT} get nodes -o yaml > "$LOGDIR/nodes.yaml"
oc --context ${MAIN_CLUSTER_CONTEXT} get mcp -o yaml > "$LOGDIR/machineconfigpools.yaml"
oc --context ${MAIN_CLUSTER_CONTEXT} get mc -o yaml > "$LOGDIR/machineconfigs.yaml"
oc --context ${MAIN_CLUSTER_CONTEXT} get csr -o yaml > "$LOGDIR/csrs.yaml"
oc --context ${MAIN_CLUSTER_CONTEXT} get workerpool -n ${MAIN_INTEGRATION_NAMESPACE} -o yaml > "$LOGDIR/workerpools.yaml"
oc --context ${MAIN_CLUSTER_CONTEXT} logs -n ${MAIN_INTEGRATION_NAMESPACE} deploy/hci-consumer-operator -c hci-consumer-operator > "$LOGDIR/hci-consumer-
operator.log"
oc --context ${MAIN_CLUSTER_CONTEXT} logs -n ${MAIN_INTEGRATION_NAMESPACE} deploy/hci-consumer-operator -c node-labeler > "$LOGDIR/node-labeler.log"
oc --context ${MAIN_CLUSTER_CONTEXT} logs -n ${MAIN_INTEGRATION_NAMESPACE} deploy/hci-consumer-operator -c csr-approver > "$LOGDIR/csr-approver.log"

# HCI Cluster resources
oc --context ${HCI_CLUSTER_CONTEXT} get all -n ${HCI_INTEGRATION_NAMESPACE} -o yaml > "$LOGDIR/hci-resources.yaml"
oc --context ${HCI_CLUSTER_CONTEXT} get vmi -n ${HCI_INTEGRATION_NAMESPACE} -o yaml > "$LOGDIR/vmis.yaml"
oc --context ${HCI_CLUSTER_CONTEXT} get nad -n ${HCI_INTEGRATION_NAMESPACE} -o yaml > "$LOGDIR/nads.yaml"
oc --context ${HCI_CLUSTER_CONTEXT} get clusteruserdefinednetwork -o yaml > "$LOGDIR/cudns.yaml"
oc --context ${HCI_CLUSTER_CONTEXT} get workerpoolprovisioning -n ${HCI_INTEGRATION_NAMESPACE} -o yaml > "$LOGDIR/workerpoolprovisioning.yaml"
oc --context ${HCI_CLUSTER_CONTEXT} logs -n ${HCI_INTEGRATION_NAMESPACE} deploy/hci-provisioner-operator > "$LOGDIR/hci-provisioner-operator.log"

# Gateway VM pod logs
oc --context ${HCI_CLUSTER_CONTEXT} logs -n ${HCI_INTEGRATION_NAMESPACE} -l kubevirt.io/domain=hci-gateway --tail=500 > "$LOGDIR/gateway-vm-launcher.log"
oc --context ${HCI_CLUSTER_CONTEXT} logs -n openshift-ovn-kubernetes -l app=ovnkube-node --tail=500 > "$LOGDIR/ovn-kubernetes.log"

# Worker node logs (if accessible)
for node in $(oc --context ${MAIN_CLUSTER_CONTEXT} get nodes -l node-role.kubernetes.io/hci-vm -o name); do
    nodename=$(basename $node)
    oc --context ${MAIN_CLUSTER_CONTEXT} adm node-logs $node > "$LOGDIR/${nodename}-node.log" 2>&1
done

tar czf "${LOGDIR}.tar.gz" "$LOGDIR"
echo "Logs collected in ${LOGDIR}.tar.gz"
```

5.1.8 Gateway Verification

Use these checks when `WorkerPoolProvisioning` is `Complete` but worker VMs are not booting or joining the cluster as expected.

Confirm All Provisioning Steps Completed

```
oc config use-context ${MAIN_CLUSTER_CONTEXT}

# All steps should show status: Complete
oc get workerpoolprovisioning -o yaml | grep -A3 'steps:'
```

Verify Gateway VM Services

```
# SSH to the gateway VM (provisioner SSH key is injected automatically)
ssh core@<gateway-external-ip>

# Ignition file — placed by the operator's Ignition step
curl -sI http://localhost/rhcos/worker-hci.ign
# Should return: HTTP/1.1 200 OK

# RHCOS boot files — downloaded by rhcos-sync timer on first boot
ls -lh /var/www/html/rhcos/
# Should show rhcos-live-kernel, rhcos-live-initramfs.img, rhcos-live-rootfs.img symlinks

# DHCP/PXE services
systemctl status dnsmasq nginx
# Both should show: active (running)

exit
```

> **RHCOS file download timing**
>
> The operator writes `desired-version.txt` to the gateway during bootstrap. The `rhcos-sync` systemd timer then downloads the matching RHCOS files in the background — typically a few minutes after first boot. The `hci-consumer` operator gates scale-up on `current-version.txt` being present; worker VMs will not be created until the files are confirmed downloaded.

Monitor Worker Boot Progress

Workers go through these stages automatically once the pool scales up:

1. **VM Creation** (1–2 min) — operator creates DataVolume + VirtualMachine on HCI
2. **PXE Boot** (2–3 min) — VM fetches iPXE → RHCOS kernel/initramfs/rootfs → ignition
3. **RHCOS Installation** (5–10 min) — installs to `/dev/vda`, applies ignition
4. **Cluster Join** (5–10 min) — boots from disk, contacts MCS, submits CSRs

```
# Watch VM status on HCI cluster
oc --context ${HCI_CLUSTER_CONTEXT} get vmi -n ${HCI_INTEGRATION_NAMESPACE} -w

# Watch CSR approval on main cluster (auto-approved if hci-node-manager is running)
oc get csr -w

# Watch nodes join
oc get nodes -w

# Access VM console to see boot messages
virtctl console <worker-vm-name> -n ${HCI_INTEGRATION_NAMESPACE}
```

5.1.9 Getting Help

If you encounter issues not covered in this guide:

1. **Check the logs** using the diagnostic commands above
2. **Review the architecture documentation** to understand expected behavior
3. **Search existing issues** in the GitHub repository
4. **Create a new issue** with:
5. Detailed problem description
6. Steps to reproduce
7. Log output from diagnostic commands
8. Your topology (mediated or direct) and platform (z/VM, KVM, LPAR)
9. OpenShift version (4.20+)

5.1.10 Related Documentation

- Getting Started Guide - Initial setup
- Deployment Variants - Understanding your deployment type
- Prerequisites - Requirements and dependencies
- Developer Notes - Technical deep dives

5.2 Monitoring Guide

This document describes how to monitor the Fusion HCI multi-architecture integration, including key metrics, dashboards, and alerting strategies.

Monitoring integration components

Monitoring covers both the HCI cluster infrastructure (gateway VM, worker VMs, OVN networks) and the main cluster integration (x86 worker nodes, CSR approval, Node Manager operator).

Monitored Area	Cluster / Namespace	Key Components
Gateway VM & services	HCI cluster (`${HCI_INTEGRATION_NAMESPACE}`)	`vmi/hci-gateway` , `dnsmasq` , `nginx` , `rhcos-sync`
Worker VMs	HCI cluster (`${HCI_INTEGRATION_NAMESPACE}`)	`vmi/worker-x86-*` , KubeVirt virt-launcher
Worker Nodes	Main cluster	`node/worker-x86-*` , kubelet, CRI-O
Integration Operator	Main cluster (`${MAIN_INTEGRATION_NAMESPACE}`)	`hci-consumer-operator` , `node-labeler` , `csr-approver`

5.2.1 Overview

Effective monitoring covers:

1. **Gateway VM** - Routing and services health (`dnsmasq` , `nginx` , `rhcos-sync`)
2. **Worker VMs** - VM and node status
3. **Network** - Connectivity and performance (internal UDN, gateway routing)
4. **Integration Health** - Overall system status, CSR approval, and WorkerPool reconciliation

5.2.2 Monitoring Stack

OpenShift Built-in Monitoring

Components: - Prometheus - Metrics collection and storage - Grafana - Visualization and dashboards - Alertmanager - Alert routing and notification

Access:

```
# OpenShift console
https://console-openshift-console.apps.cluster.example.com

# Navigate to: Observe → Dashboards/Metrics/Alerts
```

Custom Metrics

ServiceMonitor for Gateway VM:

```yaml
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: hci-gateway-monitor
  namespace: main-cluster-workers
spec:
  selector:
    matchLabels:
      app: worker-vm-router
  endpoints:
```

```
    - port: metrics
      interval: 30s
```

5.2.3 Key Metrics

Gateway VM Metrics

NETWORK THROUGHPUT

Metric: `node_network_receive_bytes_total`, `node_network_transmit_bytes_total`

Query:

```
# Receive rate (bytes/sec)
rate(node_network_receive_bytes_total{device="eth1"}[5m])

# Transmit rate (bytes/sec)
rate(node_network_transmit_bytes_total{device="eth1"}[5m])
```

Thresholds: - Normal: < 1 Gbps - Warning: > 5 Gbps - Critical: > 8 Gbps (approaching VM limit)

PACKET LOSS

Metric: `node_network_receive_drop_total`, `node_network_transmit_drop_total`

Query:

```
# Packet drop rate
rate(node_network_receive_drop_total{device="eth1"}[5m])
```

Thresholds: - Normal: 0 drops/sec - Warning: > 10 drops/sec - Critical: > 100 drops/sec

CPU UTILIZATION

Metric: `node_cpu_seconds_total`

Query:

```
# CPU usage percentage
100 - (avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)
```

Thresholds: - Normal: < 50% - Warning: > 70% - Critical: > 90%

MEMORY UTILIZATION

Metric: `node_memory_MemAvailable_bytes`, `node_memory_MemTotal_bytes`

Query:

```
# Memory usage percentage
100 - ((node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes) * 100)
```

Thresholds: - Normal: < 70% - Warning: > 80% - Critical: > 95%

DHCP SERVICE

Metric: Custom metric from dnsmasq

Query:

```
# DHCP leases active
dnsmasq_leases_active
```

Thresholds: - Normal: Matches expected worker count - Warning: Unexpected lease count - Critical: DHCP service down

Worker VM Metrics

VM STATUS

Metric: `kubevirt_vmi_phase_count`

Query:

```
# Running VMs on HCI cluster
kubevirt_vmi_phase_count{phase="Running", namespace="main-cluster-workers"}
```

Thresholds: - Normal: Matches expected count - Warning: < expected count - Critical: 0 VMs running

NODE STATUS

Metric: `kube_node_status_condition`

Query:

```
# Ready nodes
kube_node_status_condition{condition="Ready", status="true", node=~"worker-x86-.*"}
```

Thresholds: - Normal: All nodes Ready - Warning: Any node NotReady - Critical: Multiple nodes NotReady

RESOURCE UTILIZATION

CPU:

```
# Node CPU usage
100 - (avg by (node) (rate(node_cpu_seconds_total{mode="idle", node=~"worker-x86-.*"}[5m])) * 100)
```

Memory:

```
# Node memory usage
100 - ((node_memory_MemAvailable_bytes{node=~"worker-x86-.*"} / node_memory_MemTotal_bytes{node=~"worker-x86-.*"}) * 100)
```

Thresholds: - Normal: < 80% - Warning: > 85% - Critical: > 95%

Network Metrics

LATENCY

Metric: Custom metric from network probes

Query:

```
# Ping latency to gateway
probe_duration_seconds{target="192.168.100.1"}
```

Thresholds: - Normal: < 2ms - Warning: > 5ms - Critical: > 10ms

CONNECTIVITY

Metric: `probe_success`

Query:

```
# Gateway reachability
probe_success{target="192.168.100.1"}
```

Thresholds: - Normal: 1 (success) - Critical: 0 (failure)

Integration Health Metrics

CSR APPROVAL RATE

Metric: `csr_approval_duration_seconds`

Query:

```
# Average CSR approval time
avg(csr_approval_duration_seconds{node=~"worker-x86-.*"})
```

Thresholds: - Normal: < 60 seconds - Warning: > 120 seconds - Critical: > 300 seconds

NODE JOIN TIME

Metric: Custom metric tracking node join duration

Query:

```
# Time from VM start to node Ready
node_join_duration_seconds{node=~"worker-x86-.*"}
```

Thresholds: - Normal: < 600 seconds (10 minutes) - Warning: > 900 seconds (15 minutes) - Critical: > 1800 seconds (30 minutes)

5.2.4 Dashboards

Gateway VM Dashboard

Panels:

1. **Network Throughput**
2. Line graph: Receive/transmit rates
3. Time range: Last 6 hours
4. **Resource Utilization**
5. Gauge: CPU usage
6. Gauge: Memory usage
7. Gauge: Disk usage
8. **Service Status**
9. Stat: dnsmasq status
10. Stat: nginx status
11. Stat: Active DHCP leases
12. **Packet Statistics**
13. Line graph: Packet drops
14. Line graph: Packet errors

JSON Export:

```
{
  "dashboard": {
    "title": "Gateway VM Monitoring",
    "panels": [
      {
        "title": "Network Throughput",
        "targets": [
          {
            "expr": "rate(node_network_receive_bytes_total{device='eth1'}[5m])"
          }
        ]
      }
    ]
  }
}
```

Worker Nodes Dashboard

Panels:

1. **Node Status**
2. Table: Node name, status, age
3. Color coding: Green (Ready), Red (NotReady)
4. **Resource Utilization**
5. Heatmap: CPU usage per node
6. Heatmap: Memory usage per node
7. **Pod Count**
8. Bar graph: Pods per node
9. Stacked by namespace
10. **Network Traffic**
11. Line graph: Network I/O per node

Integration Health Dashboard

Panels:

1. **VM Status**
2. Stat: Total VMs
3. Stat: Running VMs
4. Stat: Failed VMs
5. **Node Status**
6. Stat: Total nodes
7. Stat: Ready nodes
8. Stat: NotReady nodes
9. **CSR Status**
10. Stat: Pending CSRs
11. Stat: Approved CSRs
12. Line graph: CSR approval rate
13. **Network Health**
14. Stat: Gateway reachability
15. Line graph: Latency to gateway
16. Line graph: Packet loss rate

5.2.5 Alerting

Alert Rules

GATEWAY VM DOWN

```yaml
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata:
  name: hci-gateway-alerts
  namespace: main-cluster-workers
spec:
  groups:
  - name: hci-gateway
    interval: 30s
    rules:
    - alert: GatewayVMDown
      expr: kubevirt_vmi_phase_count{name="hci-gateway", phase="Running"} == 0
```

```
      for: 2m
      labels:
        severity: critical
      annotations:
        summary: "Gateway VM is down"
        description: "Gateway VM {{ $labels.name }} is not running"
```

HIGH NETWORK UTILIZATION

```
- alert: GatewayHighNetworkUtilization
  expr: rate(node_network_transmit_bytes_total{device="eth1"}[5m]) > 8e9
  for: 5m
  labels:
    severity: warning
  annotations:
    summary: "Gateway VM network utilization high"
    description: "Network throughput > 8 Gbps for 5 minutes"
```

WORKER NODE NOTREADY

```
- alert: WorkerNodeNotReady
  expr: kube_node_status_condition{condition="Ready", status="true", node=~"worker-x86-.*"} == 0
  for: 5m
  labels:
    severity: warning
  annotations:
    summary: "Worker node not ready"
    description: "Node {{ $labels.node }} is NotReady for 5 minutes"
```

CSR APPROVAL DELAYED

```
- alert: CSRApprovalDelayed
  expr: count(kube_certificatesigningrequest_condition{condition="Approved", status="false"}) > 0
  for: 10m
  labels:
    severity: warning
  annotations:
    summary: "CSR approval delayed"
    description: "{{ $value }} CSRs pending approval for > 10 minutes"
```

NETWORK CONNECTIVITY LOST

```
- alert: GatewayUnreachable
  expr: probe_success{target="192.168.100.1"} == 0
  for: 2m
  labels:
    severity: critical
  annotations:
    summary: "Gateway VM unreachable"
    description: "Cannot reach gateway VM at 192.168.100.1"
```

Alert Routing

Alertmanager Configuration:

```
route:
  group_by: ['alertname', 'cluster', 'service']
  group_wait: 10s
  group_interval: 10s
  repeat_interval: 12h
  receiver: 'default'
  routes:
  - match:
      severity: critical
    receiver: 'pagerduty'
  - match:
      severity: warning
    receiver: 'slack'

receivers:
- name: 'default'
  email_configs:
  - to: 'team@example.com'

- name: 'pagerduty'
  pagerduty_configs:
  - service_key: '<pagerduty-key>'

- name: 'slack'
  slack_configs:
  - api_url: '<slack-webhook-url>'
    channel: '#hci-alerts'
```

5.2.6 Logging

Gateway VM Logs

Access (via SSH to the gateway):

```
# SSH into gateway VM
ssh core@172.16.1.100

# dnsmasq logs
sudo journalctl -u dnsmasq -f

# nginx logs (boot files & ignition requests)
sudo tail -f /var/log/nginx/access.log

# RHCOS sync timer logs
sudo journalctl -u rhcos-sync.service -f

# System logs
sudo journalctl -f
```

Key Log Patterns:

DHCP Lease:

```
DHCPACK(eth2) 192.168.100.10 52:54:01:00:00:10 worker-x86-1
```

PXE Boot & iPXE Chaining:

```
TFTP sent /var/lib/tftpboot/undionly.kpxe to 192.168.100.10
GET /boot.ipxe HTTP/1.1" 200 ...
GET /rhcos/rhcos-live-kernel-x86_64 HTTP/1.1" 200 ...
GET /rhcos/rhcos-live-initramfs.x86_64.img HTTP/1.1" 200 ...
GET /rhcos/worker-hci.ign HTTP/1.1" 200 ...
```

DNS Query:

```
query[A] api.cluster.example.com from 192.168.100.10
```

Worker Node Logs

Access:

```
# Kubelet logs
oc --context ${MAIN_CLUSTER_CONTEXT} adm node-logs worker-x86-1 -u kubelet

# Machine config daemon logs
oc --context ${MAIN_CLUSTER_CONTEXT} adm node-logs worker-x86-1 -u machine-config-daemon

# CRI-O logs
oc --context ${MAIN_CLUSTER_CONTEXT} adm node-logs worker-x86-1 -u crio
```

Centralized Logging

Using OpenShift Logging (Vector / Loki):

```
# Install logging operator
oc --context ${MAIN_CLUSTER_CONTEXT} create -f - <<EOF
apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: cluster-logging
  namespace: openshift-logging
spec:
  channel: stable-5.9
  name: cluster-logging
  source: redhat-operators
  sourceNamespace: openshift-marketplace
EOF

# Create ClusterLogForwarder instance
oc --context ${MAIN_CLUSTER_CONTEXT} create -f - <<EOF
apiVersion: observability.openshift.io/v1
kind: ClusterLogForwarder
metadata:
  name: instance
  namespace: openshift-logging
```

```
spec:
  managementState: Managed
  serviceAccount:
    name: logcollector
  pipelines:
    - name: default-pipeline
      inputRefs:
        - application
        - infrastructure
      outputRefs:
        - default-loki
  outputs:
    - name: default-loki
      type: lokiStack
      lokiStack:
        target:
          name: logging-loki
        authentication:
          token:
            from: serviceAccount
EOF
```

5.2.7 Performance Monitoring

Network Performance

Tools:

```
# iperf3 between gateway and worker
# On gateway VM:
iperf3 -s

# On worker:
iperf3 -c 192.168.100.1 -t 60

# Expected: > 1 Gbps
```

Latency Testing:

```
# Ping test
ping -c 100 192.168.100.1 | tail -1

# Expected: avg < 2ms
```

Resource Monitoring

Node Exporter Metrics:

```
# CPU usage
curl -s http://worker-x86-1:9100/metrics | grep node_cpu_seconds_total

# Memory usage
curl -s http://worker-x86-1:9100/metrics | grep node_memory

# Disk I/O
curl -s http://worker-x86-1:9100/metrics | grep node_disk
```

5.2.8 Troubleshooting with Monitoring

High Latency

Check: 1. Gateway VM CPU usage 2. Network throughput 3. Packet loss rate

Queries:

```
# CPU usage
100 - (avg(rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)

# Network throughput
rate(node_network_transmit_bytes_total[5m])

# Packet loss
rate(node_network_transmit_drop_total[5m])
```

Node NotReady

Check: 1. Node conditions 2. Kubelet status 3. Network connectivity

Commands:

```
# Node conditions
oc --context ${MAIN_CLUSTER_CONTEXT} describe node worker-x86-1 | grep Conditions -A 10

# Kubelet logs
oc --context ${MAIN_CLUSTER_CONTEXT} adm node-logs worker-x86-1 -u kubelet --tail=50

# Network test
oc --context ${MAIN_CLUSTER_CONTEXT} debug node/worker-x86-1 -- chroot /host ping -c 3 api.cluster.example.com
```

VM Performance Issues

Check: 1. VM resource allocation 2. Host node resources 3. Storage performance

Queries:

```
# VM CPU usage
kubevirt_vmi_vcpu_seconds

# VM memory usage
kubevirt_vmi_memory_resident_bytes

# Storage latency
kubevirt_vmi_storage_iops_read_total
```

5.2.9 Best Practices

Monitoring Strategy

1. **Proactive monitoring** - Set up alerts before issues occur
2. **Baseline metrics** - Establish normal operating ranges
3. **Regular review** - Review dashboards and alerts weekly
4. **Capacity planning** - Monitor trends for growth planning

Alert Management

1. **Meaningful alerts** - Only alert on actionable issues
2. **Appropriate severity** - Critical for immediate action, warning for investigation
3. **Clear descriptions** - Include context and remediation steps
4. **Avoid alert fatigue** - Tune thresholds to reduce noise

Dashboard Design

1. **Purpose-driven** - One dashboard per role/task
2. **Key metrics first** - Most important metrics at top
3. **Consistent layout** - Similar panels in similar positions
4. **Appropriate time ranges** - Match to use case

Log Management

1. **Structured logging** - Use consistent log formats
2. **Appropriate levels** - INFO for normal, ERROR for issues
3. **Retention policy** - Balance storage vs. history needs
4. **Regular review** - Check logs for patterns

5.2.10 References

- Troubleshooting Guide
- Architecture Overview
- OpenShift Monitoring Documentation
- Prometheus Documentation
- Grafana Documentation

6. Development

6.1 Developer Notes

This document captures key technical decisions, lessons learned, and best practices discovered during the development of this integration solution.

6.1.1 Core Design Principles

1. Separation of Concerns

Principle: Cluster-scoped resources should never be managed by namespace-scoped deployments.

Why: Multiple integration namespaces would conflict when trying to manage the same cluster-scoped resource (e.g., `ClusterUserDefinedNetwork`).

Implementation: - Cluster-admin applies `ClusterUserDefinedNetwork` once (Setup Guide Tier A) - Integration namespaces are labeled `hci-gateway-enabled=true` - Namespace-scoped `UserDefinedNetwork` and gateway/worker VMs are created automatically by the `hci-provisioner` operator - Clear separation between cluster-admin and integration-team responsibilities

Reference: See `hci-cluster/` for cluster-scoped setup.

2. Network Interface Management

Principle: Disable unused network interfaces completely, don't just deprioritize them.

Why: Worker VMs have two interfaces: `enp1s0` (HCI pod network) and `enp2s0` (main cluster network / internal UDN). Having both active causes: - Dual default routes - DNS conflicts - Identity confusion - NetworkManager and OVN-Kubernetes bridge attachment conflicts

Implementation: - Completely disable `enp1s0` via kernel arguments in the `99-hci-vm-kargs` MachineConfig - Pass `coreos.inst.karg=ip=enp1s0:off coreos.inst.karg=ip=enp2s0:dhcp` during iPXE first boot / installation - Use only `enp2s0` for all cluster communication and OVN `br-ex` mapping

Code:

```
# 99-hci-vm-kargs
apiVersion: machineconfiguration.openshift.io/v1
kind: MachineConfig
metadata:
  name: 99-hci-vm-kargs
  labels:
    machineconfiguration.openshift.io/role: hci-vm
    node-platform: hci-vm
spec:
  kernelArguments:
    - "root=/dev/disk/by-label/root"
    - "rd.multipath=0"
    - "rd.neednet=1"
    - "ip=enp1s0:off"
    - "ip=enp2s0:dhcp"
  config:
    ignition:
      version: 3.5.0
```

Reference: `main-cluster/configuration/base/machineconfig-hci-vm-kargs.yaml`

3. Platform-Based Configuration Inheritance

Principle: Use labels to prevent architecture-specific config inheritance.

Why: HCI x86 workers were inheriting s390x multipath storage configs from the base worker pool, causing boot failures.

Implementation: - Label MachineConfigs with `node-platform: s390x` or `node-platform: hci` - MachineConfigPool selectors exclude incompatible platforms - Each platform gets appropriate storage/network configuration

Code:

```
# MachineConfigPool selector
machineConfigSelector:
  matchExpressions:
    - key: machineconfiguration.openshift.io/role
      operator: In
      values: [worker, hci-vm]
    - key: node-platform
      operator: NotIn
      values: [s390x]  # Exclude s390x-specific configs
```

Reference: `main-cluster/configuration/base/machineconfig-pool-hci-vm.yaml`

6.1.2 Technical Decisions

Static IP Assignment via Cloud-Init and DHCP

Decision: Use cloud-init with MAC address matching for the gateway VM, and static DHCP reservations on the gateway for worker VMs.

Why: - **Gateway VM**: Configured via cloud-init network configuration (`gateway-vm-cloudinit.j2`). MAC address matching configures static IPs on the physical interface (`172.16.1.100/16`) and internal UDN interface (`192.168.100.1/24`) without relying on OVN IPAM. - **Worker VMs**: Receive deterministic IPs and hostnames from `dnsmasq` on the gateway VM via static reservations in `/etc/dnsmasq.d/pxe.conf` based on MAC prefix `52:54:01` . No cloud-init or ignition network injection needed on workers.

Gateway Cloud-Init Implementation:

```
network:
  version: 2
  ethernets:
    external:
      match:
        macaddress: "52:54:01:00:00:01"
      addresses: [172.16.1.100/16]
      gateway4: 172.16.0.1
      nameservers:
        addresses: [172.16.0.1]
    internal:
      match:
        macaddress: "52:54:01:00:00:02"
      addresses: [192.168.100.1/24]
```

Kustomize for Deployment Variants

Decision: Use Kustomize base + overlays for mediated vs direct topologies (with `generate-setup` providing automated rendering).

Why: - DRY principle - base configs defined once - Clear visibility of variant differences - Easy to add new variants (ARM, Power) - GitOps-ready - Industry standard

Structure:

```
main-cluster/configuration/
├── base/                    # Common MachineConfigs
│   ├── kustomization.yaml
│   ├── machineconfig-pool-hci-vm.yaml
│   └── machineconfig-hci-vm-kargs.yaml
└── overlays/
    ├── mediated/            # Mediated Topology (private network)
    │   └── kustomization.yaml
    └── direct/              # Direct Topology (shared/public network)
        ├── kustomization.yaml
        ├── 99-master-s390x-hci-network-route.yaml
        └── 99-worker-s390x-hci-network-route.yaml
```

OVN Networking Topology

Decision: Use OVN localnet (external) + layer2 (internal) topologies.

Why: - **Localnet**: Direct physical network access without overlay encapsulation - **Layer2**: Complete isolation for internal worker network - **Performance**: No double encapsulation overhead - **Security**: Internal network completely isolated from HCI pod network

Alternatives Rejected: - Pure overlay networks - Too much encapsulation overhead - Physical bridge - Requires node-level configuration - Router VM with NAT - Adds complexity, breaks cluster networking

6.1.3 Common Pitfalls & Solutions

1. NAD Namespace References

Problem: NetworkAttachmentDefinition references pointing to wrong namespace.

Symptom: VMs fail to start with "failed to get pod annotation" errors.

Solution: Always use fully qualified NAD references:

```
# Wrong
networks:
  - name: router-external-network

# Correct
networks:
  - name: <namespace>/router-external-network
```

Prevention: The operator automatically formats the network attachment as `<namespace>/router-external-network`.

2. ClusterUserDefinedNetwork Conflicts

Problem: Multiple deployments trying to manage the same ClusterUserDefinedNetwork.

Symptom: Second deployment fails or overwrites the cluster-scoped network.

Solution: - Cluster-admin creates `ClusterUserDefinedNetwork` once - Integration namespace must be labeled: `hci-gateway-enabled=true` - Provisioner operator references the generated NAD and manages only namespace-scoped resources

3. Master and Worker Node Routing (Direct Topology)

Problem: Master and s390x worker nodes cannot reach HCI internal network in direct topology deployments.

Symptom: CSR approval fails, cross-architecture pod communication fails.

Solution: Add static routes on master and worker nodes via MachineConfigs:

```
# Master route
ip route add 192.168.100.0/24 via <gateway-external-ip> dev br-ex

# Worker route
ip route add 192.168.100.0/24 via <gateway-external-ip>
```

Implementation: `main-cluster/configuration/overlays/direct/99-master-s390x-hci-network-route.yaml` and `99-worker-s390x-hci-network-route.yaml`

4. DNS Resolution

Problem: Workers using wrong DNS server (HCI cluster instead of main cluster).

Symptom: Can't resolve main cluster API, etcd endpoints.

Solution: - Gateway VM `dnsmasq` serves main cluster DNS (`172.16.0.1`) via DHCP - Disable `enp1s0` via kernel arguments to prevent HCI Pod network DNS from taking precedence - Verify with: `oc debug node/<worker> -- chroot /host cat /etc/resolv.conf`

6.1.4 Best Practices

Configuration Management

1. **Use single source of truth**: `integration-setup.yaml` processed by `./generate-setup`
2. **Never commit credentials**: Use `.gitignore` and example files
3. **Validate before apply**: Check generated artifacts in `workspace/`
4. **Test both topologies**: Mediated (private network) and Direct (shared network)

Deployment Workflow

1. **Gateway first**: The operator provisions and boots the gateway VM before worker VMs
2. **Verify gateway**: Ensure RHCOS boot files are downloaded and services are active
3. **Enable workers gradually**: Start with 1 canary worker, verify Ready state, then scale
4. **Monitor MachineConfigPool**: Watch `oc get mcp hci-vm` for `UPDATED: True`

Troubleshooting Approach

1. **Check MachineConfigPool status**: `oc --context ${MAIN_CLUSTER_CONTEXT} get mcp hci-vm`
2. **Verify network attachments**: `oc --context ${HCI_CLUSTER_CONTEXT} get nad -n ${HCI_INTEGRATION_NAMESPACE}`
3. **Inspect VM status**: `oc --context ${HCI_CLUSTER_CONTEXT} get vmi -n ${HCI_INTEGRATION_NAMESPACE}`
4. **Debug node**: `oc --context ${MAIN_CLUSTER_CONTEXT} debug node/<worker-name>`
5. **Check gateway status**: `ssh core@<gateway-ip>` (or `virtctl console`)

6.1.5 Performance Considerations

Network Performance

- **OVN localnet**: Near-native performance (no overlay encapsulation)
- **MTU**: Ensure consistent MTU across all networks (typically 1500)
- **Gateway VM resources**: 2 vCPU, 4GB RAM sufficient for 10-20 workers

Storage Performance

- **Worker VMs**: Use fast storage for etcd-heavy workloads
- **Gateway VM**: Standard storage sufficient (mostly network I/O)
- **Avoid multipath**: HCI virtual disks don't need multipath

6.1.6 Security Considerations

Network Isolation

- Internal worker network (192.168.100.0/24) completely isolated via OVN layer2
- No direct access from HCI pod network to workers
- Gateway VM provides controlled routing

Access Control

- Cluster-admin required for ClusterUserDefinedNetwork
- Namespace-level RBAC for VM deployment
- SSH key from main cluster required for VM access

Firewall Rules

Gateway VM implements defense-in-depth: - Explicit allow rules for cluster traffic - NAT for internet access - Drop all other traffic

6.1.7 Future Improvements

Operator Development

See Operator Roadmap and Build & Deploy Operators.

Key benefits: - Declarative configuration via `WorkerPool` and `WorkerPoolProvisioning` CRDs - Automated lifecycle management (gateway, UDN, worker VMs, CSRs, node labeling) - Self-healing capabilities - Multi-cluster and multi-namespace support

Additional Architectures

Structure supports: - ARM-based main clusters - Power-based main clusters - Additional HCI platforms

Implementation: Create new Kustomize overlay for architecture-specific needs.

6.1.8 References

Internal Documentation

- Architecture Overview
- Network Design
- Troubleshooting

External Resources

- OVN-Kubernetes Documentation
- OpenShift Multi-Architecture
- Kustomize Documentation

6.1.9 Changelog

Date	Change	Author
2026-04-22	Dual network conflict solution	Team
2026-04-22	Platform labeling implementation	Team
2026-04-27	ClusterUserDefinedNetwork separation	Team
2026-06-08	Centralized configuration system	Team
2026-06-09	Repository consolidation	Team

Note: This document is living documentation. Update it when you discover new patterns, solutions, or pitfalls.

6.2 Building and Deploying the Operators

The integration relies on two Ansible operators that run on the **main cluster**:

Operator	CRD watched	Responsibility
hci-consumer	`WorkerPool`	Scale worker VMs up/down; CSR approval and node labeling via sidecars
hci-provisioner	`WorkerPoolProvisioning`	Bootstrap/teardown: UDN, gateway VM, ignition, MachineConfig validation

Both operators must be built and deployed before you create a `WorkerPool` CR. This page covers everything from configuring the registry to verifying the running pods.

Prerequisite

You must be logged in to the **main cluster** with `cluster-admin` and have `kustomize` on your `PATH`.

Reconnecting an existing cluster?

If the operators were previously deployed on this main cluster:

- **Step 2 (BuildConfigs)** — skip unless the `BuildConfig` definitions changed
- **Step 4 (CRDs)** — skip unless the CRD schemas changed
- **Step 1 (registry config)** — verify `config.env` still matches your `setup_env.sh` values (rendered from `integration-setup.yaml`); confirm the push secret exists with `oc get secret ${REGISTRY_SECRET_NAME} -n ${OPERATOR_NAMESPACE}`

For a code-only rebuild the minimum is: **Step 3 → Step 5** (add `make rollout` if the image tag did not change — see Updating the Operators).

6.2.1 Step 1 — Configure the Registry

All image coordinates live in a single file: `operator/build/config.env`. Open it and fill in the values that match your environment:

```
# Namespace where everything lives (BuildConfigs, ImageStreams, operator Deployments)
OPERATOR_NAMESPACE ?= hci-node-manager

# ── Push target (build output) ──────────────────────────────────────
# Leave REGISTRY_PUSH empty to use the cluster's internal registry (default).
# Set it to an external / bastion-proxy registry hostname + project if needed.
REGISTRY_PUSH        ?=
REGISTRY_SECRET_NAME ?=

# ── Pull source (Deployment image references) ───────────────────────
# Internal registry — the namespace is added as a path segment automatically:
REGISTRY_PULL ?= image-registry.openshift-image-registry.svc:5000/hci-node-manager
```

`REGISTRY_SECRET_NAME` is the single push+pull secret — the same credential used by the BuildConfig (push) and the operator Deployments (pull). It matches the variable of the same name exported by `setup_env.sh` (rendered from `integration-setup.yaml`), so if your shell environment is already set up, no manual editing of this file is needed.

Registry strategy

Internal registry (default)	External / bastion-proxy registry

Leave `REGISTRY_PUSH` empty and keep the `REGISTRY_PULL` default. The build writes directly into the cluster's ImageStream and the Deployment pulls through the in-cluster registry service.

No push secret is required.

Set your bastion coordinates in `config.env` — or just export them in your shell before running `make` and the `?=` defaults will be overridden automatically:

```
REGISTRY_PUSH        = bastion.example.com:5443/myproject
REGISTRY_SECRET_NAME = hci-registry-push
REGISTRY_PULL        = bastion.example.com:5443/myproject
```

The push+pull secret (`REGISTRY_SECRET_NAME`) must already exist in `${OPERATOR_NAMESPACE}` on the main cluster. It is created by `setup-registry-access.sh` (Getting Started step C.6) — if you have completed that step, the secret is already in place.

You can override any variable on the `make` command line without editing the file:

```
make build IMAGE_TAG=v1.2.3
make deploy IMAGE_TAG=v1.2.3
```

6.2.2 Step 2 — Apply BuildConfigs (one-time)

```
cd operator/
make apply-buildconfigs
```

This single command:

1. Creates the `hci-node-manager` namespace (if absent)
2. Creates `ImageStream` objects for both operator images (internal-registry mode) **or** generates the external push patch (external-registry mode)
3. Applies both `BuildConfig` objects to the cluster

> 🔥 **You only need to re-run this step when the BuildConfig definition itself changes.**
>
> Day-to-day rebuilds of the operator images skip straight to Step 3.

6.2.3 Step 3 — Build the Images

```
make build
```

This runs `oc start-build --from-dir` for both operators. The build executes **on a cluster node** (s390x architecture), so no local container toolchain is needed.

Build a single image if only one operator changed:

```
make build-consumer     # hci-consumer only
make build-provisioner  # hci-provisioner only
```

Monitor a running build:

```
oc logs -f bc/hci-consumer-operator   -n hci-node-manager
oc logs -f bc/hci-provisioner-operator -n hci-node-manager
```

6.2.4 Step 4 — Install the CRDs

```
make install
```

Applies the `WorkerPool` and `WorkerPoolProvisioning` CRD manifests to the cluster. Run this once per cluster, or again after a CRD schema update.

Verify:

```
oc get crd | grep hci
# workerpoolprovisionings.hci.openshift.io
# workerpools.hci.openshift.io
```

6.2.5 Step 5 — Deploy the Operators

```
make deploy
```

`make deploy` generates `deploy/overlays/active/kustomization.yaml` (git-ignored) from your shell environment and runs `oc apply -k` against it. There is nothing to edit — the active overlay is produced entirely from the variables already set in your environment.

To inspect what will be applied before committing:

```
make print-config
```

If `REGISTRY_SECRET_NAME` is set, `make deploy` automatically adds an `imagePullSecrets` patch for both Deployments. No `OVERLAY` flag is needed.

> **`make deploy` does not restart running pods on its own**
>
> `make deploy` is a declarative `oc apply`: it only changes the cluster if the rendered manifests differ from what is already there. `imagePullPolicy: Always` only takes effect when a pod is (re)created. If you rebuilt the `:latest` image and re-run `make deploy`, the Deployment spec is identical, so **nothing happens and the old image keeps running**. Run `make rollout` afterwards (or deploy with a new `IMAGE_TAG`, which changes the spec and triggers a rollout automatically).

> **MAIN_CLUSTER_DOMAIN — no action required**
>
> The `csr-approver` sidecar derives the cluster domain automatically at startup by querying the `DNS/cluster` config object on the main cluster. No env var or patch file needs to be set before deploying. `MAIN_CLUSTER_DOMAIN` exists only as a fallback if that API call fails.

> **What gets deployed**
>
> - `Deployment/hci-consumer-operator` — three containers: `operator`, `csr-approver`, `node-labeler`
> - `Deployment/hci-provisioner-operator` — one container: `operator`
> - `ClusterRole` + `ClusterRoleBinding` for both operators (RBAC)

6.2.6 Step 6 — Verify

```
oc get pods -n hci-node-manager
# NAME                                  READY   STATUS    RESTARTS   AGE
# hci-consumer-operator-xxxx-yyyy       3/3     Running   0          2m
# hci-provisioner-operator-xxxx-yyyy    1/1     Running   0          2m
```

Inspect individual container logs:

```
# hci-consumer — three containers
oc logs deployment/hci-consumer-operator   -c operator      -n hci-node-manager
oc logs deployment/hci-consumer-operator   -c csr-approver  -n hci-node-manager
oc logs deployment/hci-consumer-operator   -c node-labeler  -n hci-node-manager

# hci-provisioner — one container
oc logs deployment/hci-provisioner-operator -c operator     -n hci-node-manager
```

Both operators start idle (no `WorkerPool` CR exists yet). Once you apply the `WorkerPool` CR, the operators begin their automated reconcile loop.

6.2.7 All Make Targets

Run `make help` inside `operator/` for a quick reference at any time.

Target	What it does
`apply-buildconfigs`	Create namespace, ImageStreams, BuildConfigs in-cluster (one-time)
`build`	Build both operator images in-cluster via `oc start-build`
`build-consumer`	Build `hci-consumer` image only
`build-provisioner`	Build `hci-provisioner` image only
`install`	Apply both CRDs to the cluster
`uninstall`	Remove both CRDs (**deletes all CRs**)
`deploy`	Generate active overlay from `config.env`, then `oc apply -k` (does not restart unchanged Deployments)
`rollout`	`oc rollout restart` both operator Deployments and wait for completion (use after `make build` with an unchanged tag)
`undeploy`	Delete operator Deployments (CRDs and namespace kept)
`print-config`	Show resolved image references and overlay path

6.2.8 Updating the Operators

To roll out a code change to either operator:

```
# 1. Build the new image (no need to re-apply BuildConfigs)
make build                # or build-consumer / build-provisioner

# 2. Restart both Deployments to pull the new :latest image, wait until ready
make rollout
```

`make rollout` runs `oc rollout restart` followed by `oc rollout status` for both operators (timeout via `ROLLOUT_TIMEOUT`, default `300s`). To restart only one operator:

```
oc rollout restart deployment/hci-consumer-operator -n hci-node-manager
oc rollout status  deployment/hci-consumer-operator -n hci-node-manager
```

For a tagged release:

```
make build IMAGE_TAG=v1.2.3
make deploy IMAGE_TAG=v1.2.3
```

6.2.9 Tearing Down

```
# Remove Deployments (leaves CRDs and namespace intact)
make undeploy

# Also remove CRDs (WARNING: deletes all WorkerPool and WorkerPoolProvisioning objects)
make uninstall

# Remove the namespace entirely
oc delete namespace hci-node-manager
```

> ⚠ **Removing CRDs while WorkerPools exist**
>
> Running `make uninstall` while active `WorkerPool` CRs exist bypasses the finalizer that drains nodes and deletes VMs. Always scale down / delete all `WorkerPool` objects first:
>
> ```
> oc delete workerpool --all
> # wait for finalizer to complete, then:
> make uninstall
> ```

6.2.10 Local Debug Workflow (No Image Build)

Each operator directory contains a `run-local.sh` that runs the Ansible role directly on the bastion against a live cluster — useful for iterating on role logic without a full image build cycle:

```
# Test hci-consumer reconcile against a live WorkerPool CR
cd operator/hci-consumer
./run-local.sh

# Test hci-provisioner reconcile against a live WorkerPoolProvisioning CR
cd operator/hci-provisioner
./run-local.sh
```

See the individual operator READMEs for prerequisites.

6.3 Operator Optimization and Go Migration Guide

This document captures architectural analysis, performance tuning recommendations for the current Ansible-based operators, and the roadmap for migrating to native Golang operators.

6.3.1 1. Executive Summary

The Fusion HCI integration currently runs two Ansible-based operators built with the Operator SDK Ansible plugin: * `hci-consumer`: Watches `WorkerPool` CRs on the main cluster, scales worker VMs on the HCI cluster, and manages node join operations. * `hci-provisioner`: Watches `WorkerPoolProvisioning` CRs, orchestrating UDN creation, gateway VM deployment, SSH readiness probes, and ignition generation.

While functional and verified end-to-end, the Ansible implementation exhibits latency during state transitions due to synchronous playbook execution and timer-based reconciliation loops.

6.3.2 2. Immediate Optimizations for Ansible Operators

The following improvements can be implemented directly within the existing Ansible operator architecture without rewriting the codebase in Go.

2.1 Enable Event-Driven Dependent Resource Watches

In `operator/hci-consumer/watches.yaml` and `operator/hci-provisioner/watches.yaml`, `watchDependentResources` is currently set to `false`. Reconcile loops rely almost entirely on the timer tick (`reconcilePeriod: 60s`).

Recommended Configuration:

```
- version: v1alpha1
  group: hci-integration.openshift.io
  kind: WorkerPool
  role: hci_consumer
  reconcilePeriod: 30s
  manageStatus: false
  watchDependentResources: true
  watchClusterScopedResources: true
  gatheredFacts: false
```

- **Mechanism**: Because `WorkerPoolProvisioning` CRs carry an `ownerReference` pointing to the owning `WorkerPool`, enabling `watchDependentResources: true` triggers an instant reconcile on `WorkerPool` when provisioning step statuses change (e.g. from `Pending` → `InProgress` → `Complete`).
- **Impact**: Eliminates up to 60 seconds of idle wait time when bootstrap completes.

2.2 Disable OS Fact Gathering (`gatheredFacts: false`)

Both operators currently set `gatheredFacts: true` in `watches.yaml` solely to obtain `ansible_date_time` timestamps.

- **Problem**: Ansible executes the complete `setup` module inside the container on **every single reconcile cycle**, gathering host/container hardware, network, and disk metrics. This adds **1 to 3 seconds of CPU overhead** per reconcile pass.
- **Fix**: Set `gatheredFacts: false` in `watches.yaml` and generate timestamps on-demand in tasks:

```
lastTransitionTime: "{{ lookup('pipe', 'date -u +%Y-%m-%dT%H:%M:%SZ') }}"
```

2.3 Cascade Sequential Steps in `advance_steps.yml`

In `hci-provisioner`, each step transition currently terminates the playbook via `ansible.builtin.meta: end_play`. When Step 1 (`UserDefinedNetwork`) completes, the operator waits for the next 60s timer tick before evaluating Step 2 (`GatewayVM`).

- **Fix**: Allow `advance_steps.yml` to evaluate downstream steps whose dependencies are immediately met, executing them in the same reconcile pass if no asynchronous external wait is required.
- **Impact**: Shaves 1 to 2 minutes off the initial bootstrap flow.

2.4 Server-Side API Filtering in `observe_world.yml`

In `observe_world.yml`, listing all cluster nodes and iterating through pods on cordoned nodes produces multiple sequential API queries.

- **Fix**: Add `label_selectors` directly to `k8s_info` calls so filtering occurs on the Kubernetes API server rather than across large Jinja2 lists in the Ansible runner:

```yaml
- name: "Observe | List pool worker nodes"
  kubernetes.core.k8s_info:
    api_version: v1
    kind: Node
    label_selectors:
      - "node-role.kubernetes.io/{{ consumer.machine_config_pool }}"
  register: _obs_all_nodes
```

2.5 Operator Runner Environment Tuning

Configure runtime environment variables in `operator/deploy/base/` deployments: * `WORKERS: "2"` or `MAX_CONCURRENT_RECONCILES: "2"` — Prevents queue blockages across multiple CR events. * `ANSIBLE_VERBOSITY: "0"` — Reduces runner JSON parsing and disk I/O in the container during steady state. * `ANSIBLE_RETRY_FILES_ENABLED: "0"` — Disables retry file creation.

6.3.3 3. Comparison: Ansible vs. Golang Operators

Metric / Feature	Ansible Operator	Golang Operator (`controller-runtime`)
Event Reaction Time	Seconds (sequential playbook execution)	Milliseconds (informer work queues & event filters)
Memory Footprint	~400 MB – 800 MB per pod (`ose-ansible-operator` + Python)	~30 MB – 60 MB per pod (single static binary)
Sidecar Containers	Requires 3 containers (`operator`, `csr-approver`, `node-labeler`)	Single container; approver & labeler run as integrated informers
Type Safety	Runtime YAML string interpretation; Ansible 2.9 quirks	Compile-time Go type safety for KubeVirt, CDI, and OpenShift APIs
Cross-Cluster Auth	Writes temporary kubeconfig files to `/tmp` per reconcile	In-memory `client.Client` instances with pooled HTTP/2 connections
External Operations	Python scripts, shell-outs for SSH/SCP	Native Go packages (`golang.org/x/crypto/ssh`, `crypto/ed25519`)
Testing	Live cluster or bastion runner (`run-local.sh`)	Fast unit/integration tests with `envtest` (no live cluster required)
Ecosystem Support	Standalone `ansible-operator` binary deprecated in SDK v1.38+	First-class, standard Kubernetes & OpenShift operator ecosystem

---

6.3.4 4. Golang Operator Migration Blueprint

When migrating to Go, the architecture simplifies into a single controller binary managing multiple controllers and informers:

```
cmd/main.go
├── WorkerPoolReconciler (replaces hci-consumer role)
│   ├── Remote HCI KubeVirt / CDI Client Manager
│   └── Local Node Lifecycle & Drain Manager
├── WorkerPoolProvisioningReconciler (replaces hci-provisioner role)
│   ├── State Machine Engine (UDN -> GatewayVM -> Ignition -> Bootfiles)
│   └── Native SSH/SFTP Client (gateway file transfer & probes)
├── CSRAutoApproverInformer (replaces node-auto-approver.py sidecar)
└── NodeLabelerInformer (replaces node-labeler.py sidecar)
```

Key Migration Steps

1. **Scaffold Project**:
2. Initialize using `operator-sdk init --plugins=go/v4 --domain=openshift.io`.
3. Create APIs for `WorkerPool` and `WorkerPoolProvisioning`.
4. **Implement Multi-Cluster Client Cache**:
5. Implement a manager that reads the credentials Secret and instantiates/caches a remote `client.Client` using `clientcmd.RESTConfigFromKubeConfig`.
6. **Port Reconciliation Logic**:
7. Use typed KubeVirt (`kubevirt.io/api/core/v1`) and CDI (`cdi.kubevirt.io/v1beta1`) client packages.
8. Replace manual node drain logic with the standard `k8s.io/kubectl/pkg/drain` package.
9. **Integrate Node Manager Sidecars into Informers**:
10. Add a `CertificateSigningRequest` controller that approves kubelet and bootstrap CSRs matching the pool naming pattern.
11. Add a `Node` controller that applies role labels based on internal IP subnet checks.
12. **Multi-Architecture Builds**:
13. Leverage Go cross-compilation (`GOARCH=s390x go build`) to produce LinuxONE binaries directly from standard CI build pipelines.

6.3.5 5. Trade-offs and Recommendation

- **Trade-offs**: Rewriting in Go requires Go/controller-runtime expertise and an upfront development investment (~20–25 story points), temporarily diverting focus from upcoming functional epics (such as Epic 3: HCI Provider Operator).
- **Recommendation**:
- **Phase 1 (Immediate)**: Apply the Ansible optimizations in Section 2 (`watchDependentResources: true`, `gatheredFacts: false`, cascading step execution) to achieve immediate responsiveness gains with minimal effort (~4 points).
- **Phase 2 (Production Hardening)**: Transition to Go when preparing for multi-tenant production deployments or publishing to OperatorHub.

6.4 Contributing to Fusion HCI Multi-Architecture Integration

Thank you for your interest in contributing! This document provides guidelines for contributing to the project.

6.4.1 Code of Conduct

- Be respectful and inclusive
- Focus on constructive feedback
- Help others learn and grow
- Maintain a professional environment

6.4.2 How to Contribute

Reporting Issues

1. **Search existing issues** to avoid duplicates
2. **Use issue templates** when available
3. **Provide details**:
4. OpenShift versions (main cluster and HCI cluster)
5. Deployment variant (z/VM or KVM)
6. Error messages and logs
7. Steps to reproduce

Suggesting Enhancements

1. **Check the roadmap** in Operator Roadmap
2. **Open a discussion** for major changes
3. **Describe the use case** and benefits
4. **Consider backwards compatibility**

Contributing Code

GETTING STARTED

1. **Fork the repository**
2. **Clone your fork**:

```
git clone https://github.com/your-username/fusion-hci-integration.git
cd fusion-hci-integration
```

3. **Create a branch**:

```
git checkout -b feature/your-feature-name
```

DEVELOPMENT GUIDELINES

Documentation: - Update documentation for any user-facing changes - Use clear, concise language - Include examples where appropriate - Test documentation builds: `mkdocs serve`

Configuration Files: - Never commit credentials or sensitive data - Use `.example` suffix for template files - Update `.gitignore` if needed - Validate YAML syntax

Kustomize Overlays: - Keep base resources generic - Use overlays for variant-specific changes - Test with `kubectl kustomize` - Document overlay purpose in README

Scripts: - Use bash with error handling (`set -euo pipefail`) - Add usage documentation - Make scripts idempotent where possible - Test on both Mediated and Direct topology variants

TESTING

Before submitting: 1. Test your changes in a real environment 2. Verify both Mediated and Direct topology variants (if applicable) 3. Check documentation builds without errors 4. Run any existing tests

Test checklist: - [ ] Gateway VM deploys successfully - [ ] Worker VMs join the cluster - [ ] Network connectivity works - [ ] Documentation is accurate - [ ] No credentials in commits

COMMIT MESSAGES

Follow conventional commits format:

```
<type>(<scope>): <subject>

<body>

<footer>
```

Types: - `feat` : New feature - `fix` : Bug fix - `docs` : Documentation changes - `refactor` : Code refactoring - `test` : Test additions/ changes - `chore` : Maintenance tasks

Examples:

```
feat(operator): add support for custom storage class

Add storageClassName parameter to worker VM template
to allow users to specify custom storage classes.

Closes #123
```

```
docs(direct): clarify master route configuration

Improve documentation for Direct Topology master node
routing setup with troubleshooting steps.
```

PULL REQUEST PROCESS

1. **Update documentation** for any user-facing changes
2. **Update CHANGELOG.md** (if applicable)
3. **Ensure tests pass** (manual testing required)
4. **Request review** from maintainers
5. **Address feedback** promptly
6. **Squash commits** if requested

PR Description Template:

```
## Description
Brief description of changes

## Type of Change
- [ ] Bug fix
- [ ] New feature
- [ ] Documentation update
- [ ] Refactoring

## Testing
- [ ] Tested on Mediated Topology variant
- [ ] Tested on Direct Topology variant
- [ ] Documentation builds successfully
- [ ] No credentials in commits

## Related Issues
Closes #<issue-number>

## Screenshots (if applicable)
```

Contributing Documentation

Documentation is crucial! We welcome: - Typo fixes and clarifications - Additional examples - Troubleshooting guides - Architecture diagrams - Tutorial improvements

Documentation structure:

```
docs/
├── index.md                    # Landing page
├── getting-started.md          # Quick start guide
├── prerequisites.md            # Requirements
├── deployment-variants.md      # z/VM vs KVM
├── architecture/               # Architecture docs
├── configuration/              # Configuration guides
├── operations/                 # Operational guides
└── development/                # Developer guides
```

Building docs locally:

```
pip install mkdocs-material
mkdocs serve
# Open http://localhost:8000
```

Areas Needing Contribution

HIGH PRIORITY

- [ ] Additional deployment examples
- [ ] Troubleshooting scenarios
- [ ] Performance tuning guides
- [ ] Security hardening documentation

MEDIUM PRIORITY

- [ ] Monitoring and alerting setup
- [ ] Backup and recovery procedures
- [ ] Upgrade procedures
- [ ] Multi-cluster examples

FUTURE WORK

- [ ] Operator development (see Operator Roadmap)
- [ ] ARM architecture support
- [ ] Power architecture support
- [ ] Automated testing framework

6.4.3 Development Environment

Required Tools

- `oc` or `kubectl` CLI
- `kustomize` (or `kubectl kustomize`)
- `mkdocs` with material theme
- Access to OpenShift clusters (main + HCI)

Recommended Tools

- `yamllint` for YAML validation
- `shellcheck` for shell script linting
- `pre-commit` hooks (optional)

6.4.4 Style Guidelines

YAML

- Use 2 spaces for indentation
- Quote strings when necessary
- Use `---` document separator
- Add comments for complex configurations

Markdown

- Use ATX-style headers (`#`)
- Add blank lines around code blocks
- Use fenced code blocks with language tags
- Keep lines under 120 characters (soft limit)

Shell Scripts

- Use `#!/bin/bash` shebang
- Add `set -euo pipefail` for safety
- Use meaningful variable names
- Add usage documentation
- Quote variables: `"${VAR}"`

6.4.5 Security

Reporting Security Issues

Do not open public issues for security vulnerabilities.

Contact maintainers privately: - Email: OpenShift-on-Z team (via GitHub Issues marked as security) - Include detailed description and reproduction steps

Security Best Practices

- Never commit credentials
- Use secrets management (Sealed Secrets, Vault, etc.)
- Follow principle of least privilege
- Keep dependencies updated
- Review security implications of changes

6.4.6 Questions?

- **Documentation**: Check the documentation site
- **Discussions**: Use GitHub Discussions
- **Issues**: Search existing issues
- **Chat**: Join our community chat (if available)

6.4.7 License

By contributing, you agree that your contributions will be licensed under the Apache License 2.0.

Thank you for contributing to make multi-architecture OpenShift deployments easier!

6.5 Operator Roadmap

This document describes the vision and implementation plan for the Kubernetes Operator that manages the Fusion HCI multi-architecture integration. For the three-actor model see provider-consumer-model.md.

6.5.1 Current State

Implementation: Ansible Operator (operator-sdk Ansible plugin)
Language: Ansible (roles, tasks) + Jinja2
Operator directories: `operator/hci-consumer/`, `operator/hci-provisioner/`

What Is Implemented

EPIC 1 — WORKER STEADY-STATE OPERATOR (`hci-consumer`)

The core reconcile loop is **fully implemented and confirmed e2e on a live test cluster pair**:

Component	File	Status
`WorkerPool` CRD (`hci-integration.openshift.io/v1alpha1`)	`config/crd/workerpool.yaml`	✅ Done
Operator scaffold: `watches.yaml`, `requirements.yml`	`operator/hci-consumer/`	✅ Done
RBAC: minimal-permission ServiceAccount	`config/rbac/role.yaml`	✅ Done
Observe phase: world-state fact gathering	`roles/hci_consumer/tasks/observe_world.yml`	✅ Done
Diff phase: set arithmetic, action lists	`roles/hci_consumer/tasks/diff_desired_actual.yml`	✅ Done
Scale-up: create DataVolume + VM on HCI	`roles/hci_consumer/tasks/create_worker_vm.yml`	✅ Done
Scale-down: cordon + drain gate → delete node → delete VM	`roles/hci_consumer/tasks/scale_down.yml`	✅ Done
Scale-down: protect nodes with local-storage annotation	`roles/hci_consumer/tasks/observe_world.yml`	✅ Done
Cleanup / finalizer role	`roles/hci_consumer_cleanup/tasks/main.yml`	✅ Done
`maxPoolSize` CEL admission rule on CRD	`config/crd/workerpool.yaml`	✅ Done
Status conditions: `ProviderCredentialsReady`, `ScaleDownBlocked`, `CountExceedsMaxPoolSize`	`roles/hci_consumer/tasks/`	✅ Done
Status counts: `deployedWorkers`, `readyWorkers`, `pendingWorkers`	`roles/hci_consumer/tasks/main.yml`	✅ Done
Phase sub-states: `ScalingUp`, `ScalingDown`, `ScalingDownBlocked`, `Ready`, `Deprovisioned`	`roles/hci_consumer/tasks/main.yml`	✅ Done
Bootfiles version gate (`sync_bootfiles_version.yml`)	`roles/hci_consumer/tasks/sync_bootfiles_version.yml`	✅ Done
Development helper: `run-local.sh`	`operator/hci-consumer/run-local.sh`	✅ Done

EPIC 2 — HCI PROVISIONING OPERATOR (`hci-provisioner`)

Full bootstrap and teardown lifecycle via `WorkerPoolProvisioning` CRD:

Component	File	Status
`WorkerPoolProvisioning` CRD (`wpp`)	`config/crd/workerpoolprovisioning.yaml`	✅ Done
Operator scaffold: `watches.yaml` , `requirements.yml`	`operator/hci-provisioner/`	✅ Done
Step: validate MachineConfigPool on consumer cluster	`roles/hci_provisioner/tasks/step_machineconfig.yml`	✅ Done
Step: create `UserDefinedNetwork` on HCI (+ teardown)	`roles/hci_provisioner/tasks/step_udn.yml`	✅ Done
Step: deploy gateway VM + DataVolume on HCI (+ teardown)	`roles/hci_provisioner/tasks/step_gateway_vm.yml`	✅ Done
Gateway VM readiness probe (SSH)	`roles/hci_provisioner/tasks/step_gateway_vm_readiness.yml`	✅ Done
Step: generate + SCP ignition to gateway	`roles/hci_provisioner/tasks/step_ignition.yml`	✅ Done
Step: teardown all consumer worker VMs before UDN teardown	`roles/hci_provisioner/tasks/step_workers_teardown.yml`	✅ Done
Provisioner SSH key management	`roles/hci_provisioner/tasks/ensure_provisioner_ssh_key.yml`	✅ Done
Gateway VM cloud-init template	`roles/hci_provisioner/templates/gateway-vm-cloudinit.j2`	✅ Done
Cleanup / finalizer role	`roles/hci_provisioner_cleanup/tasks/main.yml`	✅ Done
SA credential setup script (manual)	`operator/setup-provider-credentials.sh`	✅ Done
Development helper: `run-local.sh`	`operator/hci-provisioner/run-local.sh`	✅ Done

What Is Not Yet Implemented

Epic	What	Stories
Epic 2 remaining	MachineConfig setup scripting for LinuxONE admin	BS-02b
Epic 2 remaining	Optional: image registry mirror configuration	BS-07
Epic 3	HCI Provider Operator	HCI-01 to HCI-05
Epic 4	Negotiation flow (LO-initiated requests)	NEG-01 to NEG-03

6.5.2 Vision

Goal: Fully automated, declarative lifecycle management of multi-architecture worker nodes.

An operator administrator applies one `WorkerPool` CR; the system: 1. Provisions UDN, gateway VM, and ignition via the `hci-provisioner` operator (Bootstrap) 2. Continuously reconciles `workers.count` → VMs on HCI and nodes on the LO cluster (Steady-state) 3. Handles scale-up and scale-down safely, with drain timing controlled by LOOps 4. Tears down all infrastructure cleanly on CR deletion

HCI-side provisioning (namespace, network, quota, ticket generation) will be automated by the HCI Provider Operator (Epic 3, future work).

Target Users: Platform administrators, SREs, DevOps teams
Key Principles: Declarative, self-healing, observable, privilege-minimal

---

6.5.3 Framework and Technology Choices

Operator Framework

Choice: Ansible Operator (operator-sdk Ansible plugin)

Rationale: - Most of the existing integration work already uses Ansible and `kubernetes.core` modules - Ansible roles map directly to the observe/diff/act reconcile structure - Low barrier to contribution from the existing team - Excellent `kubernetes.core.k8s` and `k8s_info` modules for both clusters - No Go cross-compilation concern for s390x (runs on bastion natively)

`ansible-operator` **binary**: standalone download removed in operator-sdk v1.38+. Development uses `ansible-runner` directly on the cluster bastion.

Note: For performance tuning recommendations and the planned roadmap to migrate to native Golang operators, see Operator Optimization and Go Migration Guide.

Cross-Cluster Communication

Choice: Direct API calls with kubeconfig (stored as Kubernetes Secret)

Rationale: Simple, standard, no additional infrastructure. The kubeconfig is written to a temp file per reconcile and cleaned up after. The namespace-scoped SA limits blast radius.

CRD API Group

Choice: `hci-integration.openshift.io`

Used for `WorkerPool`, `WorkerPoolProvisioning`, and all future CRDs in this project.

6.5.4 Implementation Phases

Phase 1 — Steady-State Core ✅ COMPLETE

Delivered: - `WorkerPool` CRD with CEL admission validation - Observe/diff/act reconcile loop - Scale-up and scale-down with set-difference logic (gap-fill correct) - `ScaleDownBlocked` + `CountExceedsMaxPoolSize` + `ProviderCredentialsReady` conditions - Status counts: `deployedWorkers`, `readyWorkers`, `pendingWorkers` - Phase sub-states: `ScalingUp`, `ScalingDown`, `ScalingDownBlocked`, `Ready`, `Deprovisioned` - Drain gate: cordon + no non-system pods before VM deletion - Local-storage node protection via annotation - Finalizer/cleanup role with workload-safety gate - Minimal RBAC ServiceAccount - Bastion-based dev workflow with `run-local.sh`

Confirmed e2e on a live test cluster pair: full lifecycle (create → scale-up → scale-down → delete) verified including status counts and phase transitions.

Phase 2 — Bootstrap Operator (Epic 2) ✅ LARGELY COMPLETE

Delivered: - `WorkerPoolProvisioning` CRD with step-based state machine (`wpp`) - `WorkerPool` CRD split into `spec.provider` + `spec.consumer` (BS-00b refactor) - UDN step: create/delete `UserDefinedNetwork router-internal-network` on HCI (BS-03) - Gateway VM step: full cloud-init stack (DHCP, PXE/iPXE, nginx, ignition, RHCOS sync) (BS-04) - Ignition step: generate worker-hci.ign, SCP to gateway nginx root (BS-06a) - Bootfiles version gate: `desired-version.txt` drift detection, scale-up gate (BS-06b) - MachineConfig step: validate MachineConfigPool exists and is healthy (BS-02) - Workers teardown step: delete all consumer VMs before UDN teardown (BS-08) - Single credential secret (`hci-provisioner-credentials`) via `setup-provider-credentials.sh` - BS-05 deliberately descoped: manual script remains the documented handover tool

Confirmed e2e: apply one `WorkerPool` CR → gateway provisions → workers PXE-boot and join cluster → scale up/down → delete CR → full teardown. See `CURRENT-STATUS.md`.

Remaining: - [ ] BS-02b: Parameterised MachineConfig setup script for LinuxONE admin - [ ] BS-07: Optional image registry mirror (`ImageContentSourcePolicy`)

Phase 3 — HCI Provider Operator (Epic 3)

Goal: Remove the last manual step — HCI admin no longer needs to run scripts.

Deliverables: - [ ] HCI-01: `ComputeConsumerRequest` + `ComputeConsumerOrder` CRDs on HCI - [ ] HCI-02: HCI Provider Operator scaffold + watches - [ ] HCI-03: Process Order: namespace + CUDN label + ResourceQuota - [ ] HCI-04: Generate onboarding ticket (privileged bootstrap SA + time-limited token) - [ ] HCI-05: Quota enforcement and usage monitoring

Phase 4 — Negotiation Flow (Epic 4, Deferred)

Goal: LO-initiated requests — no out-of-band communication needed.

Design open. Requires a solution to the bootstrap discovery problem before stories can start. Deferred to iteration 2.

6.5.5 Demo Milestones

Milestone	Status	What it shows
Demo 1 — Core loop	✅ DONE	Change `workers.count` → VMs appear/disappear on HCI, nodes join/leave LO cluster. Credentials placed manually.
Demo 1b — Provisioned MVP	✅ DONE	Apply one `WorkerPool` CR → gateway VM boots with full PXE stack, bootfiles version gate, workers PXE-boot and join cluster. Scale up/down works end-to-end.
Demo 3 — Both operators	🔲 Phase 3	HCI admin approves request → full two-cluster automated provisioning.
Demo 4 — Self-service	🔲 Phase 4	LOOps submits request, no out-of-band communication.

6.5.6 Contributing

Development Setup (Bastion-Based)

```
# On the cluster bastion
git clone https://github.com/IBM/fusion-hci-integration
cd fusion-hci-integration

python3.11 -m venv --system-site-packages ~/ansible-operator-venv
source ~/ansible-operator-venv/bin/activate
pip install ansible-runner kubernetes openshift
ansible-galaxy collection install kubernetes.core redhat.openshift operator_sdk.util

# Consumer operator
cd operator/hci-consumer
./run-local.sh          # normal reconcile
./run-local.sh --cleanup  # simulate CR deletion

# Provisioner operator
cd operator/hci-provisioner
./run-local.sh          # normal reconcile
./run-local.sh --cleanup  # simulate CR deletion
```

See `operator/hci-consumer/README.md` for the full development guide.

Code Standards

- Ansible tasks: one logical step per task, descriptive `name:` field
- All API calls in `observe_world.yml` — zero API calls in diff or act files

- Conditions set/cleared symmetrically — every `True` has a paired `False` path
- Use `set_fact` for computed values; never compute inline in `when:` guards
- YAML strings: avoid backslash in `>-` blocks (use character classes like `([.]|$)`)

6.5.7 References

- Provider/Consumer Model
- operator-sdk Ansible plugin
- kubernetes.core collection